

Shor's Algorithm

Factoring and the Elliptic-Curve Discrete Logarithm

Derivation, circuit construction, and resource analysis

Prerequisites: linear algebra, elementary group theory, modular arithmetic.

No quantum mechanics assumed; the formalism needed is built in Part III.

PART 1

Setup: the two problems and the chain

The two problems

Problem A — integer factoring (RSA)

Given a composite N , find a non-trivial factor.

Best known classical algorithm: the number field sieve,

$$\exp\left(O(n^{1/3} \log^{2/3} n)\right), \quad n = \log_2 N.$$

Problem B — ECDLP (ECC)

Given points P and $Q = [m]P$ on an elliptic curve over $\mathbb{F}_p$, find m .

Best known classical algorithm: Pollard rho,

$$\Theta(\sqrt{r}) \text{ group operations, } r = \text{ord}(P).$$

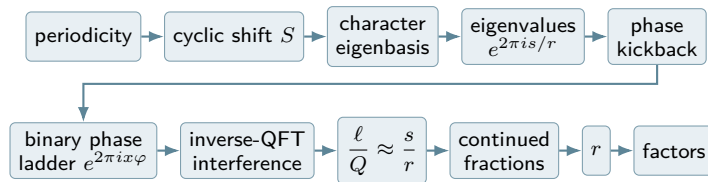
Both are instances of the abelian hidden subgroup problem

A group translation is diagonalised by characters; its eigenvalues are roots of unity $e^{2\pi i s/r}$, which carry the secret in a phase; controlled powers write that phase across a register; the inverse Fourier transform turns phase into probability; classical post-processing reads off the answer.

Factoring is the one-dimensional case, $A = \mathbb{Z}/r\mathbb{Z}$. The ECDLP is the two-dimensional case, $A = \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$. Everything else — registers, circuits, arithmetic — is engineering around this core.

The chain: nine links

Every arrow is a theorem; the table on the next slide gives each one.



The ECDLP branch: three links change

Links 5–7 (kickback, ladder, interference) are identical. What changes:

- ▶ Links 0–1 become $f(k, \ell) = [k]P + [\ell]Q = [k + m\ell]P$, which hides the subgroup $L = \langle (-m, 1) \rangle \leq \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$.
- ▶ Link 3 becomes QFT $\otimes$ QFT: the group is two-dimensional and the dual group factors.
- ▶ Links 8–9 become $y_2 \equiv my_1 \pmod{r}$, so $m \equiv y_2 y_1^{-1}$. No continued fractions, because r is public.

The chain, link by link

Statement establishing each arrow, and the one-line reason

#	Link	Statement	Reason
0	factoring $\leftarrow$ order finding	$x^2 \equiv 1, x \not\equiv \pm 1 \Rightarrow \gcd(x \mp 1, N)$ splits N ; take $x = a^{r/2}$	$N \mid (x - 1)(x + 1)$ but divides neither factor
1	periodicity = cyclic shift	on the orbit $\{1, a, \dots, a^{r-1}\}$, $U_a a^k = a^{k+1}$ is the shift S on $\mathbb{C}^r$	the orbit is a cycle of length r ; $\times a$ walks it one step
2	shift $\rightarrow$ roots of unity	$Sv = \lambda v$ forces $\lambda^r = 1$; $ u_s\rangle = \frac{1}{\sqrt{r}} \sum_k \omega_r^{-sk} k\rangle$	$c_{k-1} = \lambda c_k$ gives $c_k = \lambda^{-k} c_0$; closing the cycle needs $\lambda^{-r} = 1$
3	eigenvectors are characters	$S = F_r^\dagger D F_r$; r distinct eigenvalues $\Rightarrow$ eigenbasis unique up to phase	a character turns argument-addition into value-multiplication
4	the input is free	$ 1\rangle = \frac{1}{\sqrt{r}} \sum_s u_s\rangle$	orthogonality collapses $\sum_s \omega_r^{-sk}$ to $r\delta_{k,0}$
5	eigenvalue $\rightarrow$ relative phase	$c \cdot U(\alpha 0\rangle + \beta 1\rangle) u\rangle = (\alpha 0\rangle + \beta e^{2\pi i \varphi} 1\rangle) u\rangle$	linearity: the gate applies U^c ; the eigenvalue is a scalar and floats to the control
6	relative phase $\rightarrow$ ladder	$\bigotimes_s \frac{ 0\rangle + e^{2\pi i 2^{t-s} \varphi} 1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x \varphi} x\rangle$	expanding the product multiplies phases, and $\sum_s x_s 2^{t-s} = x$
7	ladder $\rightarrow$ peak	$A(y) = \frac{1}{Q} \sum_x e^{2\pi i x \delta y}, \delta_y = \varphi - \frac{y}{Q};$ $\Pr[y] = \frac{1}{Q^2} \frac{\sin^2(\pi Q \delta)}{\sin^2(\pi \delta)}$	Q unit arrows with common increment: an arc if $\delta \approx 0$, a closed polygon if $\delta = k/Q$
8	peak $\rightarrow$ exact fraction	Legendre: $ x - p/q < \frac{1}{2q^2} \Rightarrow p/q$ is a convergent of x	forces $Q \geq N^2$, i.e. $t = 2n + O(1)$ counting qubits
9	$r \rightarrow$ factors	verify $a^{r'} \equiv 1$; then $\gcd(a^{r/2} \pm 1, N)$	back to link 0; verification is what makes the algorithm Las Vegas

Structure of the talk

Part	Topic	Content
I	Classical reduction	the $\sqrt{1}$ lemma, order finding, the two conditions, success over random bases, verification
II	Algebra (no quantum)	the orbit is a cycle; solve $c_{k-1} = \lambda c_k$; $\lambda^r = 1$; the Fourier basis is forced
III	Quantum formalism	amplitudes vs. probabilities; global vs. relative phase; tensor product; interference
IV	Order-finding subroutine	$ 1\rangle = \frac{1}{\sqrt{r}} \sum_s u_s\rangle$; kickback; the ladder; the Fejér peak
V	Circuit construction	elementary gates $\rightarrow$ controlled- $U \rightarrow$ QFT/QFT $^\dagger \rightarrow$ reversible modular arithmetic $\rightarrow$ full circuit
VI	Post-processing	$\ell/Q \approx s/r$; Legendre; why $t = 2n + O(1)$; continued fractions
VII	Worked example	$N = 21$, $a = 2$, end to end, by hand
VIII	Qiskit implementation	$N = 15$ in code; synthesis choices; transpilation; hardware run; what the published demonstrations show
IX	ECDLP	the two-dimensional hidden subgroup; $y_2 \equiv my_1$; the elliptic oracle down to Toffoli gates
X	Summary	what each ingredient contributes, and where the mechanism stops

One statement that will not be used

The phrase “it tries all answers at once” will not appear, and Part X explains why it is wrong. An unmeasured uniform superposition carries zero information; the algorithm’s content is the interference that is arranged before measurement.

Notation

Fourier convention

For $Q \geq 2$, indices in $\mathbb{Z}/Q\mathbb{Z}$, and $\omega_Q := e^{2\pi i/Q}$:

$$\text{QFT}_Q |x\rangle = \frac{1}{\sqrt{Q}} \sum_{y=0}^{Q-1} \omega_Q^{xy} |y\rangle, \quad \text{QFT}_Q^\dagger |x\rangle = \frac{1}{\sqrt{Q}} \sum_{y=0}^{Q-1} \omega_Q^{-xy} |y\rangle$$

The character of $\mathbb{Z}/Q\mathbb{Z}$ indexed by y is $\chi_y(x) = \omega_Q^{xy}$.

Bit order (big-endian)

For $x = x_1 x_2 \cdots x_t \in \{0, 1\}^t$,

$$x = \sum_{s=1}^t x_s 2^{t-s}, \quad |x\rangle = |x_1\rangle \otimes \cdots \otimes |x_t\rangle,$$

so x_1 is the most significant bit and sits on the *top* wire.

Binary fractions: $0.a_1 a_2 \cdots a_m := \sum_{s \geq 1} a_s 2^{-s}$.

All phases live modulo 1 inside $e^{2\pi i(\cdot)}$.

Symbol

Meaning

N

modulus to factor

a

base, $\gcd(a, N) = 1$

$r = \text{ord}_N(a)$

order of a modulo N

$n = \lceil \log_2 N \rceil$

qubits of the work register

$Q = 2^t$

size of the counting register

$\omega_M = e^{2\pi i/M}$

principal M -th root of unity

H, X, Y, Z

Hadamard and Pauli gates

$P(\theta) = \text{diag}(1, e^{i\theta})$

phase gate

$R_k = P(2\pi/2^k)$

QFT rotation gate

$c\text{-}U$

controlled- U

S

cyclic shift $|k\rangle \mapsto |k+1\rangle$

U_a

$|y\rangle \mapsto |ay \bmod N\rangle$

$|u_s\rangle$

eigenvector of U_a , eigenvalue $e^{2\pi i s/r}$

φ

an eigenphase, $U|u\rangle = e^{2\pi i \varphi} |u\rangle$

$\delta_y = \varphi - y/Q$

detuning of candidate y

$E(\mathbb{F}_p), \mathcal{O}$

curve points; point at infinity

$[k]P$

k -fold sum $P + \cdots + P$

m

the discrete logarithm, $Q = [m]P$

Two uses of Q : $Q = 2^t$ is the counting-register size in Parts I–VIII; $Q = [m]P$ is the public point in Part IX. They never occur in the same formula; Part IX uses 2^{n+1} for register sizes.

Exercise 1 — the chain

3 minutes, blank page

Task

Write the nine-link chain from *periodicity* to *factors*. For each arrow, name the statement that establishes it. Then write the three links that change for the ECDLP.

Exercise 1 — the chain

3 minutes, blank page

Task

Write the nine-link chain from *periodicity* to *factors*. For each arrow, name the statement that establishes it. Then write the three links that change for the ECDLP.

Solution

The chain and its table are the two previous slides. Target: all nine links in order and at least seven of the nine statements.

The ECDLP fork:

1. the hiding function $f(k, \ell) = [k]P + [\ell]Q = [k + m\ell]P$ with $L = \langle (-m, 1) \rangle$;
2. two QFTs, because the dual group $\hat{A} \cong \widehat{\mathbb{Z}/r\mathbb{Z}} \times \widehat{\mathbb{Z}/r\mathbb{Z}}$ factors;
3. post-processing is $m \equiv y_2 y_1^{-1}$, with no continued fractions, because r is known in advance.

One-sentence version of the fork: *factoring is the one-dimensional hidden subgroup problem, where the unknown is the size of the hidden subgroup; the ECDLP is the two-dimensional one, where the size is known and the unknown is the slope. That is why factoring needs continued fractions and the ECDLP needs one modular inverse.*

PART 2

The classical reduction

Character orthogonality

The identity used throughout the derivations that follow

Lemma 1 (geometric sum / character orthogonality)

For $\ell \in \mathbb{Z}$ and $M \geq 1$,

$$\sum_{k=0}^{M-1} \omega_M^{k\ell} = \begin{cases} M, & \ell \equiv 0 \pmod{M}, \\ 0, & \text{otherwise,} \end{cases} \quad \omega_M = e^{2\pi i/M}.$$

Proof

If $\ell \equiv 0$ every term is 1. Otherwise $z := \omega_M^\ell \neq 1$ and $\sum_{k=0}^{M-1} z^k = \frac{1-z^M}{1-z} = 0$, since $z^M = \omega_M^{M\ell} = 1$. $\square$

What it generates

Unitarity of the QFT; inversion of the eigenvector formula; the identity $|1\rangle = \frac{1}{\sqrt{r}} \sum_s |u_s\rangle$; the exact zeros of the Fejér kernel; and the entire ECDLP measurement relation. It is the most frequently used identity in the derivations that follow.

Factoring reduces to a non-trivial square root of 1

Lemma 2 (square roots of 1 split N)

Let $N \geq 3$ and let x satisfy $x^2 \equiv 1$, $x \not\equiv 1$, $x \not\equiv -1 \pmod{N}$. Then $\gcd(x-1, N)$ and $\gcd(x+1, N)$ are non-trivial factors of N .

Proof

$x^2 - 1 = (x-1)(x+1) \equiv 0$, so $N \mid (x-1)(x+1)$. Put $d := \gcd(x-1, N)$.

- ▶ If $d = N$ then $N \mid x-1$, i.e. $x \equiv 1$ — excluded.
- ▶ If $d = 1$ then N and $x-1$ are coprime, so $N \mid x+1$, i.e. $x \equiv -1$ — also excluded.

Hence $1 < d < N$ and $d \mid N$. The same argument with $-x$ gives the claim for $\gcd(x+1, N)$. $\square$

Order finding manufactures such an x

Pick a coprime to N and let $r = \text{ord}_N(a)$, so $a^r \equiv 1$. If (i) r is even and (ii) $a^{r/2} \not\equiv -1 \pmod{N}$, put $x := a^{r/2}$. Then

- ▶ $x^2 = a^r \equiv 1$;
- ▶ $x \not\equiv 1$, else $r/2$ would already be a period, contradicting minimality of r ;
- ▶ $x \not\equiv -1$ by (ii).

So Lemma 2 applies, matching the identity $a^r - 1 = (a^{r/2} - 1)(a^{r/2} + 1)$.

A random base usually works

Theorem

Let N be odd and composite with $m \geq 2$ distinct prime factors, and draw a uniformly from $(\mathbb{Z}/N\mathbb{Z})^\times$, $r = \text{ord}_N(a)$. Then

$$\Pr \left[r \text{ even and } a^{r/2} \not\equiv -1 \pmod{N} \right] \geq 1 - 2^{-(m-1)} \geq \frac{1}{2}.$$

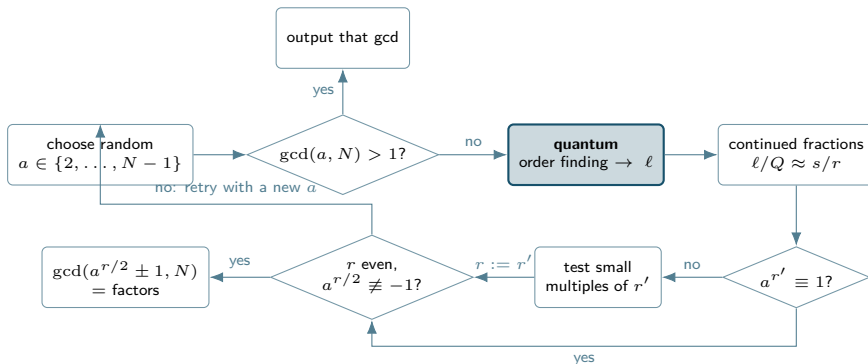
Proof sketch

By CRT, $(\mathbb{Z}/N\mathbb{Z})^\times \cong \prod_j (\mathbb{Z}/p_j^{\alpha_j}\mathbb{Z})^\times$, and drawing a uniformly is the same as drawing its components independently. Write $r_j = \text{ord}_{p_j^{\alpha_j}}(a)$ and $d_j = v_2(r_j)$, the 2-adic valuation. The bad event (r odd, or $a^{r/2} \equiv -1$) forces all d_j to coincide. Each $(\mathbb{Z}/p^\alpha\mathbb{Z})^\times$ is cyclic for odd p , so $\Pr[d_j = d] \leq \frac{1}{2}$ for each fixed d ; independence over the $m-1$ indices $j \neq j_0$ bounds the bad event by $2^{-(m-1)}$. $\square$

Two preconditions that are handled classically first

Strip factors of 2, and test whether N is a perfect power p^k for $k \leq \log_2 N$: the theorem needs $m \geq 2$ distinct odd primes.

The pipeline, with the quantum part isolated



Reading the diagram

Only the shaded box is quantum. Everything else — gcd, continued fractions, verification, factor extraction — is cheap classically and verifies its own output, so the procedure never returns a wrong factor: it is Las Vegas, not Monte Carlo. Two independent failure probabilities, both bounded away from 0, must not be conflated: the choice of a may be unlucky, and a given quantum run may return a useless outcome.

Two running examples

$N = 15, a = 2$ — the dyadic case

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 1 \pmod{15}$, so $r = \text{ord}_{15}(2) = 4$.

(i) $r = 4$ even ✓

(ii) $2^2 = 4 \not\equiv -1 \equiv 14$ ✓

► $\gcd(3, 15) = 3, \gcd(5, 15) = 5$

$r = 4$ is a power of two, so phase estimation with $Q = 8$ is *exact* and the continued-fraction step is vacuous.

$N = 21, a = 2$ — the non-dyadic case

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 11 \rightarrow 1 \pmod{21}$, so $r = \text{ord}_{21}(2) = 6$.

(i) $r = 6$ even ✓

(ii) $2^3 = 8 \not\equiv -1 \equiv 20$ ✓

► $\gcd(7, 21) = 7, \gcd(9, 21) = 3$

$6 \nmid Q$ for any power of two, so the peaks broaden into a diffraction pattern and the continued-fraction step is non-trivial.

Why $N = 15$ is not a representative test case

$(\mathbb{Z}/15\mathbb{Z})^\times \cong \mathbb{Z}/2 \times \mathbb{Z}/4$, so $\text{ord}_{15}(a) \in \{1, 2, 4\}$ always: the period is a power of two whichever base is chosen. The continued-fraction step is therefore vacuous for $N = 15$. The analytical worked example below uses $N = 21$.

Exercise 2 — the classical reduction

5 minutes

Task

(a) Prove: $x^2 \equiv 1, x \not\equiv \pm 1 \pmod{N} \Rightarrow \gcd(x - 1, N)$ is a non-trivial factor. (b) Show order finding produces such an x , stating both conditions. (c) State the success probability over a random base, and why the algorithm never returns a wrong answer.

Exercise 2 — the classical reduction

5 minutes

Task

(a) Prove: $x^2 \equiv 1, x \not\equiv \pm 1 \pmod{N} \Rightarrow \gcd(x-1, N)$ is a non-trivial factor. (b) Show order finding produces such an x , stating both conditions. (c) State the success probability over a random base, and why the algorithm never returns a wrong answer.

Solution

(a) $x^2 - 1 = (x-1)(x+1) \equiv 0$, so $N \mid (x-1)(x+1)$. Let $d = \gcd(x-1, N)$. If $d = N$ then $x \equiv 1$ — excluded. If $d = 1$ then $N \mid x+1$, i.e. $x \equiv -1$ — excluded. Hence $1 < d < N$ and $d \mid N$.

(b) Let $r = \text{ord}_N(a)$. If (i) r is even and (ii) $a^{r/2} \not\equiv -1$, put $x = a^{r/2}$. Then $x^2 = a^r \equiv 1$; $x \not\equiv 1$ by minimality of r ; $x \not\equiv -1$ by (ii).

(c) For odd composite N with $m \geq 2$ distinct prime factors and a uniform in $(\mathbb{Z}/N\mathbb{Z})^\times$,

$\Pr[(i) \text{ and } (ii)] \geq 1 - 2^{-(m-1)} \geq \frac{1}{2}$. The algorithm is Las Vegas: every candidate r' is checked by $a^{r'} \stackrel{?}{\equiv} 1$ and every candidate factor by division, so a wrong value is rejected, never returned.

Two preconditions people omit: strip factors of 2 first, and test whether N is a perfect power p^k ($k \leq \log_2 N$) — the theorem needs $m \geq 2$ distinct primes.

PART 3

The algebra: periodicity as a cyclic shift

Periodicity is cyclic translation

Let $G = \mathbb{Z}/r\mathbb{Z}$ and $\mathbb{C}^G \cong \mathbb{C}^r$ with standard basis $\{|k\rangle\}_{k \in \mathbb{Z}/r\mathbb{Z}}$; all index arithmetic is mod r .

Definition (cyclic translation)

$S \in \mathbb{C}^{r \times r}$, $S|k\rangle = |k+1 \bmod r\rangle$, i.e. $S = \sum_k |k+1\rangle\langle k|$, $S_{k\ell} = \delta_{k-\ell,1}$. More generally $S^t|k\rangle = |k+t\rangle$.

Proposition (elementary properties)

S is unitary, $S^r = \mathbb{I}$, $S^\dagger = S^{-1} = S^{r-1}$ with $S^\dagger|k\rangle = |k-1\rangle$, and $t \mapsto S^t$ is a group homomorphism $(\mathbb{Z}/r\mathbb{Z}, +) \rightarrow U(r)$.

Proof. S permutes an orthonormal basis, hence is unitary; $S^r|k\rangle = |k+r\rangle = |k\rangle$. $\square$

Circulants

For $a = (a_0, \dots, a_{r-1})$ the circulant C_a has $(C_a)_{k\ell} = a_{k-\ell \bmod r}$; equivalently $C_a = \sum_m a_m S^m$, a polynomial in the shift.

Circulants are *exactly* the matrices commuting with S , i.e. the translation-invariant linear maps. So any basis diagonalising S diagonalises every translation-invariant operator at once.

Concretely

For $N = 15$, $a = 2$: the orbit of 1 is $\{1, 2, 4, 8\}$ and multiplication by 2 sends $|1\rangle \mapsto |2\rangle \mapsto |4\rangle \mapsto |8\rangle \mapsto |1\rangle$ — literally S on $\mathbb{C}^4$.

Diagonalising S by hand: roots of unity are forced

Seek $Sv = \lambda v$ with $v = \sum_{k=0}^{r-1} c_k |k\rangle$. Since $Sv = \sum_k c_k |k+1\rangle = \sum_k c_{k-1} |k\rangle$, the eigenvalue equation reads

$$\boxed{c_{k-1} = \lambda c_k \text{ for all } k \bmod r} \quad \implies \quad c_k = \lambda^{-k} c_0.$$

Consistency around the cycle, $c_0 = c_r = \lambda^{-r} c_0$, forces

$$\lambda^r = 1 \quad \implies \quad \lambda = \lambda_s = \omega_r^s = e^{2\pi i s/r}, \quad s = 0, 1, \dots, r-1.$$

Normalising ($|c_k| = 1/\sqrt{r}$) gives the eigenvectors

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} \omega_r^{-sk} |k\rangle, \quad S|u_s\rangle = \omega_r^s |u_s\rangle.$$

Check

$$S|u_s\rangle = \frac{1}{\sqrt{r}} \sum_k \omega_r^{-sk} |k+1\rangle = \frac{1}{\sqrt{r}} \sum_{k'} \omega_r^{-s(k'-1)} |k'\rangle = \omega_r^s |u_s\rangle. \quad \square$$

What has been achieved

The hidden period r has been rewritten as *the denominator of an eigenvalue's phase*. Finding r is now the problem of measuring a phase s/r . Parts III–VII do nothing else. No quantum mechanics has appeared yet.

The eigenvectors are the characters; the DFT is the change of basis

Characters

A character of $(\mathbb{Z}/r\mathbb{Z}, +)$ is a homomorphism $\chi : \mathbb{Z}/r\mathbb{Z} \rightarrow \mathbb{C}^\times$; they are exactly $\chi_s(k) = \omega_r^{sk}$, $s \in \mathbb{Z}/r\mathbb{Z}$. The eigenvector $|u_s\rangle$ has coordinates $\overline{\chi_s(k)}/\sqrt{r}$.

Why characters must be eigenvectors, in one line: a character turns *addition of its argument* into *multiplication of its value*, $\chi(k+t) = \chi(k)\chi(t)$; so translating it can only multiply it by the scalar $\chi(t)$.

Theorem (diagonalisation)

With F_r the DFT matrix, $(F_r)_{yx} = \omega_r^{xy}/\sqrt{r}$, the matrix F_r is unitary and

$$S = F_r^\dagger D F_r, \quad D = \text{diag}(1, \omega_r, \omega_r^2, \dots, \omega_r^{r-1}),$$

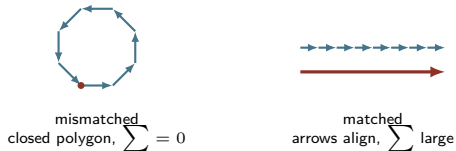
since the matrix whose columns are the $|u_s\rangle$ is $F_r^\dagger$. Consequently every circulant $C_a = \sum_m a_m S^m$ is diagonalised by the same basis, with eigenvalues $\hat{a}(s) = \sum_m a_m \omega_r^{ms}$.

The basis is determined, not chosen

S is unitary with r *distinct* eigenvalues ω_r^s , so by the spectral theorem its eigenbasis is unique up to phases. Any procedure that wants to see the period must land in the Fourier basis. The QFT is therefore not a design choice but a consequence of the spectral theorem.

What a Fourier coefficient computes

For $f : \mathbb{Z}/Q\mathbb{Z} \rightarrow \mathbb{C}$, the coefficient $\hat{f}(y) = \frac{1}{\sqrt{Q}} \sum_x f(x) \omega_Q^{-xy}$ takes each sample $f(x)$, rotates it backwards by the angle $2\pi xy/Q$, and sums.



The mechanism

Multiplying by a conjugate rotating mode and summing is a *test for phase matching*.

- ▶ If the unwinding rate y/Q matches the signal's rate, every arrow points the same way and they add coherently: $|\hat{f}(y)|$ is large.
- ▶ If it mismatches by a non-zero multiple of $1/Q$, the arrows are the Q -th roots of unity taken k at a time — a closed polygon — and the sum is *exactly* 0. This is Lemma 1.
- ▶ If it nearly matches, the arrows form a short arc: the sum is large but not maximal.

Where this is used again

Part IV shows that the inverse QFT is exactly this picture, with Q arrows and common increment $2\pi\delta_y$, $\delta_y = \varphi - y/Q$. The closed-form intensity is the Fejér kernel.

Exercise 3 — diagonalise the shift from scratch

6 minutes

Task

Define S on $\mathbb{C}^r$. Solve $Sv = \lambda v$ by hand. Derive $\lambda^r = 1$ and the normalised eigenvectors. Then answer in one sentence each: why is $|\lambda| = 1$ automatic, and why is the Fourier basis forced rather than chosen?

Exercise 3 — diagonalise the shift from scratch

6 minutes

Task

Define S on $\mathbb{C}^r$. Solve $Sv = \lambda v$ by hand. Derive $\lambda^r = 1$ and the normalised eigenvectors. Then answer in one sentence each: why is $|\lambda| = 1$ automatic, and why is the Fourier basis forced rather than chosen?

Solution

$S|k\rangle = |k+1 \bmod r\rangle$, i.e. $S = \sum_k |k+1\rangle\langle k|$. Write $v = \sum_k c_k |k\rangle$; then $Sv = \sum_k c_k |k+1\rangle = \sum_k c_{k-1} |k\rangle$, so the eigen-equation is $c_{k-1} = \lambda c_k$, giving $c_k = \lambda^{-k} c_0$. Consistency around the cycle, $c_0 = c_r = \lambda^{-r} c_0$, forces

$$\lambda^r = 1 \Rightarrow \lambda_s = e^{2\pi i s/r}, \quad |u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |k\rangle.$$

$|\lambda| = 1$: S permutes an orthonormal basis, hence is unitary and norm-preserving: $|\lambda| \|v\| = \|Sv\| = \|v\|$. Every eigenvalue of a unitary is a pure phase — which is what allows an eigenvalue to be kicked back as a phase.

Forced: S is normal with r distinct eigenvalues, so by the spectral theorem its eigenbasis is unique up to phases; any procedure that wants to see the period must land in that basis. (Bonus: circulants are exactly the matrices commuting with S , so the same basis simultaneously diagonalises every translation-invariant operator.)

PART 4

The quantum formalism used

Defined here

- ▶ complex Hilbert space, Dirac notation
- ▶ qubit, superposition, Born rule
- ▶ global vs. relative phase
- ▶ tensor product and why $\dim = 2^n$
- ▶ entanglement
- ▶ unitaries, controlled gates, measurement
- ▶ interference and its four rules
- ▶ reversible evaluation of a classical f

Not needed

- ▶ Schrödinger's equation, Hamiltonians
- ▶ infinite-dimensional analysis
- ▶ density matrices, mixed states
- ▶ any physical interpretation of measurement

Working definition

For present purposes, quantum computation is linear algebra over $\mathbb{C}$ together with one rule for reading out: the outcome x occurs with probability $|\alpha_x|^2$.

States, amplitudes, and the Born rule

Definition (qubit, register)

A qubit has state space $\mathbb{C}^2$ with computational basis $|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$, $|1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$. A pure state of n qubits is a *unit* vector

$$|\psi\rangle = \sum_{x \in \{0,1\}^n} \alpha_x |x\rangle \in \mathcal{H}_n = (\mathbb{C}^2)^{\otimes n}, \quad \sum_x |\alpha_x|^2 = 1.$$

The $\alpha_x \in \mathbb{C}$ are *amplitudes*; a state with at least two non-zero amplitudes is a superposition.

Born rule

Measuring $|\Psi\rangle = \sum_x \alpha_x |x\rangle |\chi_x\rangle$ in the computational basis of the first register returns x with $\Pr[x] = |\alpha_x|^2$ and leaves the state $|x\rangle |\chi_x\rangle$. This is the only non-unitary ingredient used.

An amplitude is not a probability

α_x is complex; it may be negative or imaginary. Only $|\alpha_x|^2$ is a probability. Two amplitudes can cancel, which two probabilities cannot. This is the property the algorithm exploits.

Precise phrasing for a qubit

A qubit's state is a unit vector in $\mathbb{C}^2$. The two coefficients are complex amplitudes; only their squared moduli are probabilities, and the *relative phase* between them is physical — that is the part with no classical analogue.

Dimension counting

$\mathbb{C}^d$ has real dimension $2d$, but a pure state costs $2d$ reals minus one for normalisation and minus one for the unobservable global phase: $2d - 2$ physical parameters. For a qubit that is two, the Bloch angles. The factor of two is bookkeeping for imaginary parts, not a hidden copy of the system.

Global phase is invisible; relative phase is not

Global phase. For any projector P and unitary U , $\langle\psi| e^{-i\gamma} P e^{i\gamma} |\psi\rangle = \langle\psi| P |\psi\rangle$ and $U(e^{i\gamma} |\psi\rangle) = e^{i\gamma} U |\psi\rangle$: an overall $e^{i\gamma}$ can never be detected, now or later.

Relative phase. A phase *between two amplitudes of one superposition* is entirely physical:

$$|+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}}.$$

Both give computational-basis probabilities $(\frac{1}{2}, \frac{1}{2})$: a measurement *now* cannot tell them apart. But with $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$,

$$H |+\rangle = |0\rangle, \quad H |-\rangle = |1\rangle,$$

and they become perfectly distinguishable.

The same structure at large scale

The relative sign was invisible to one measurement and decisive after one gate. The inverse QFT plays the role of that second Hadamard at scale: it converts hidden relative phases into visible probabilities. The algorithm consists of manufacturing the right relative phases and then applying this conversion.

Tensor product: what it buys, and what it does not

Construction

For $\mathcal{H}_A = \mathbb{C}^{d_A}$ with basis $\{|i\rangle\}$ and $\mathcal{H}_B = \mathbb{C}^{d_B}$ with basis $\{|j\rangle\}$, the space $\mathcal{H}_A \otimes \mathcal{H}_B$ has basis $\{|i\rangle \otimes |j\rangle\}$, so $\dim = d_A d_B$. Operators act factorwise, $(M \otimes N)(|\psi\rangle \otimes |\phi\rangle) = M|\psi\rangle \otimes N|\phi\rangle$. Shorthand: $|i\rangle \otimes |j\rangle = |ij\rangle$.

Why $\otimes$ and not $\oplus$: two independent classical bits have $2 \times 2 = 4$ joint configurations, not $2 + 2$. The joint state space must contain all four basis configurations *and their superpositions*, so dimensions multiply: $\dim \mathcal{H}_n = 2^n$.

One layer of Hadamards

$$H^{\otimes n} |0\rangle^{\otimes n} = \bigotimes_{s=1}^n \frac{|0\rangle + |1\rangle}{\sqrt{2}} = \frac{1}{2^{n/2}} \sum_{x=0}^{2^n-1} |x\rangle.$$

This is not yet a speed-up

Every amplitude in $2^{-n/2} \sum_x |x\rangle$ has the same modulus $2^{-n/2}$, so measuring it returns a uniform random x — exactly what a classical coin gives. *An unmeasured uniform superposition carries zero information.*

The exponential space is necessary but not sufficient: the work is done by interference, which redistributes those amplitudes before measurement.

Scale

$n = 300$ qubits span a space of dimension $2^{300} \approx 10^{90}$, more than the number of atoms in the observable universe. But a measurement returns 300 classical bits, no more. The design problem is arranging for those 300 bits to be the informative ones.

Definition

$|\Psi\rangle \in \mathcal{H}_A \otimes \mathcal{H}_B$ is *separable* if $|\Psi\rangle = |\alpha\rangle \otimes |\beta\rangle$; otherwise it is *entangled*.

Example

Let $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$ and suppose $|\Phi^+\rangle = (a_0|0\rangle + a_1|1\rangle) \otimes (b_0|0\rangle + b_1|1\rangle)$. Comparing coefficients, $a_0b_0 = a_1b_1 = \frac{1}{\sqrt{2}}$ and $a_0b_1 = a_1b_0 = 0$. The last two force $a_0 = 0$ or $b_1 = 0$, either of which kills one of the first two. Contradiction.

Equivalently: separability says the coefficient matrix $\begin{pmatrix} \alpha_{00} & \alpha_{01} \\ \alpha_{10} & \alpha_{11} \end{pmatrix}$ has rank 1, i.e. determinant 0.

The operational meaning used below

If a register holds $\sum_x \alpha_x |x\rangle |\chi_x\rangle$ with $\langle \chi_x | \chi_{x'} \rangle = 0$ for $x \neq x'$, the second register records *which branch* the first is in. That record is which-path information, and it destroys interference between the branches.

Where entanglement sits in Shor's algorithm

It appears exactly when the oracle correlates the two registers, and what it does is keep the r eigenphase branches from mixing. It is necessary bookkeeping, not the engine. Independently, it cannot be sufficient: the Gottesman–Knill theorem gives highly entangled circuits that are classically simulable in polynomial time.

Gates: unitary matrices

A closed system evolves by a unitary U : $U^\dagger U = U U^\dagger = \mathbb{I}$, acting as $|\psi\rangle \mapsto U |\psi\rangle$. Unitaries are exactly the linear maps preserving all inner products, hence all probabilities; every gate is invertible, with inverse $U^\dagger$ — which is why a circuit can always be run backwards.

Gate	Matrix	Action
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	$ b\rangle \mapsto b \oplus 1\rangle$ (NOT)
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	$ b\rangle \mapsto (-1)^b b\rangle$
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	$ b\rangle \mapsto \frac{1}{\sqrt{2}}(0\rangle + (-1)^b 1\rangle)$
$P(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	$ b\rangle \mapsto e^{i\theta b} b\rangle$
R_k	$\begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$	$P(2\pi/2^k)$; used by the QFT

$$R_1 = Z, R_2 = P(\pi/2) = \text{diag}(1, i), R_3 = T = P(\pi/4).$$

Controlled gates

On $1 + m$ qubits,

$$c\text{-}U := |0\rangle\langle 0| \otimes \mathbb{I}_{2^m} + |1\rangle\langle 1| \otimes U,$$

so $c\text{-}U |c\rangle |\psi\rangle = |c\rangle U^c |\psi\rangle$ for $c \in \{0, 1\}$. By linearity this determines the action on *superpositions* of the control, which is the content of phase kickback (Part IV).

CNOT = $c\text{-}X$; the Toffoli TOF = $cc\text{-}X$ maps $|a\rangle |b\rangle |c\rangle \mapsto |a\rangle |b\rangle |c \oplus ab\rangle$ and is universal for reversible classical computation.

Convention

Circuits are read left to right in time, but composed right to left as operators: a drawing G_1 then G_2 implements $G_2 G_1$. The top wire is most significant.

Interference: amplitudes add before probabilities are formed

Let $|\psi\rangle = \sum_x \alpha_x |x\rangle$ and V unitary with $V_{yx} = \langle y|V|x\rangle$. Then

$$V|\psi\rangle = \sum_y \underbrace{\left(\sum_x V_{yx} \alpha_x \right)}_{\beta_y} |y\rangle, \quad \text{Pr}[y] = |\beta_y|^2 = \left| \sum_x V_{yx} \alpha_x \right|^2.$$

For two contributions $A = \rho e^{i\theta}$, $B = \sigma e^{i\phi}$,

$$|A+B|^2 = |A|^2 + |B|^2 + \underbrace{2 \operatorname{Re}(\bar{A}B)}_{\text{interference term}} = \rho^2 + \sigma^2 + 2\rho\sigma \cos(\theta - \phi).$$

The cross term is positive when the contributions are nearly aligned, negative when nearly opposite, zero when orthogonal in phase. Only the *relative* phase enters.

The four rules

- (R1) Amplitudes are summed before $|\cdot|^2$; interference is the cross term in that square.
- (R2) Only contributions landing on the *same* final basis state $|y\rangle$ are added.
- (R3) If a label sits in an orthogonal unmeasured register, the cross term carries $\langle \chi_x | \chi_{x'} \rangle = 0$ and vanishes: which-path information kills interference.
- (R4) A relative phase becomes observable only when a later operation recombines the branches.

Naming discipline

Do not write “the phases interfere”. Write: *the contributions $V_{yx}\alpha_x$ and $V_{yx'}\alpha_{x'}$ to $|y\rangle$ add with relative phase $\theta - \phi$* . If the two contributions cannot be named, there is no interference to discuss.

Computing a classical function reversibly

A classical $f : \{0, 1\}^n \rightarrow \{0, 1\}^m$ is generally not injective, so $|x\rangle \mapsto |f(x)\rangle$ is not unitary. The standard fix uses an extra register:

$$U_f : |x\rangle |z\rangle \mapsto |x\rangle |z \oplus f(x)\rangle, \quad U_f^2 = \mathbb{I}.$$

This is unitary (a permutation of basis states) and is built from Toffoli and CNOT gates. Applied to a superposition:

$$U_f \left(\frac{1}{\sqrt{2^n}} \sum_x |x\rangle \right) |0\rangle = \frac{1}{\sqrt{2^n}} \sum_x |x\rangle |f(x)\rangle \quad \text{— one call, all inputs.}$$

Garbage must be erased, and this is not optional

A naive circuit leaves intermediate scratch bits behind: $|x\rangle |f(x)\rangle |g(x)\rangle$. By rule (R3) that garbage tags the branch and destroys the interference the algorithm depends on.

Bennett's compute-copy-uncompute. Compute f into scratch with a circuit C , copy the answer out with CNOTs, then run C^{-1} to clear the scratch:

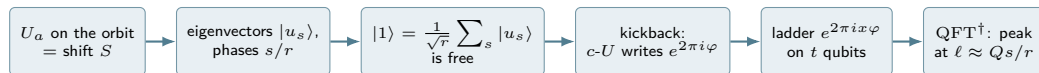
$$|x\rangle |y\rangle |0\rangle |0\rangle \xrightarrow{C} |x\rangle |y\rangle |f(x)\rangle |g(x)\rangle \xrightarrow{\text{copy}} |x\rangle |y \oplus f(x)\rangle |f(x)\rangle |g(x)\rangle \xrightarrow{C^{-1}} |x\rangle |y \oplus f(x)\rangle |0\rangle |0\rangle.$$

Every arithmetic block in Part V is engineered this way, and every one of them ends by recomputing its own flags from its output.

PART 5

The order-finding subroutine

The chain now built



Definition (U_a)

For $\gcd(a, N) = 1$ define U_a on $\mathbb{C}^{2^n}$, $n = \lceil \log_2 N \rceil$, by $U_a |y\rangle = |ay \bmod N\rangle$ ($0 \leq y < N$), $U_a |y\rangle = |y\rangle$ ($N \leq y < 2^n$).

Proposition (U_a is unitary, and its powers are cheap)

U_a permutes basis states (its inverse is $U_{a^{-1} \bmod N}$), hence is unitary. Moreover

$$U_a^{2^j} = U_{a^{2^j} \bmod N},$$

and the constant $a^{2^j} \bmod N$ is computed *classically* by j repeated squarings.

Theorem (restriction to the orbit)

Let $\mathcal{O} = \{1, a, a^2, \dots, a^{r-1}\}$ be the orbit of 1. Identifying $a^k \leftrightarrow |k\rangle$, $U_a|_{\mathcal{O}}$ is exactly the shift S on $\mathbb{C}^r$. Hence

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k / r} |a^k \bmod N\rangle,$$

$$U_a |u_s\rangle = e^{2\pi i s / r} |u_s\rangle, \quad s = 0, \dots, r-1.$$

Source of the exponential speed-up

$U_a^{2^j}$ costs *one* modular multiplication, not 2^j of them. This, not the Fourier transform, is the source of the advantage.

The input state $|1\rangle$ is free

Theorem

$$\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle = |1\rangle.$$

Proof

$$\frac{1}{\sqrt{r}} \sum_s |u_s\rangle = \frac{1}{r} \sum_s \sum_k e^{-2\pi i s k / r} |a^k\rangle = \frac{1}{r} \sum_k \underbrace{\left(\sum_s e^{-2\pi i s k / r} \right)}_{= r \delta_{k,0} \text{ (Lemma 1)}} |a^k\rangle = |a^0\rangle = |1\rangle. \quad \square$$

Why this matters

Standard phase estimation requires an *eigenvector* as input. But writing down a single $|u_s\rangle$ requires knowing r — the very quantity being computed. This identity removes the circularity: the uniform superposition of all r eigenvectors is the computational basis state $|0 \cdots 01\rangle$, which is free to prepare.

By linearity the circuit acts on each eigencomponent independently, so a run is statistically identical to running on a uniformly random $|u_s\rangle$, s uniform in $\{0, \dots, r-1\}$. The work register is never measured.

Exercise 4 — the free input

5 minutes

Task

Prove $|1\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle$, and explain in two sentences why this is the step that makes the algorithm possible. Then state what the two registers hold immediately before the inverse QFT, in both bases, and why they are the same state.

Exercise 4 — the free input

5 minutes

Task

Prove $|1\rangle = \frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} |u_s\rangle$, and explain in two sentences why this is the step that makes the algorithm possible. Then state what the two registers hold immediately before the inverse QFT, in both bases, and why they are the same state.

Solution

The computation is the previous slide; the collapse $\sum_s e^{-2\pi i s k / r} = r \delta_{k,0}$ is Lemma 1.

Why it matters: phase estimation wants an eigenvector, but writing down a single $|u_s\rangle$ needs r — the thing being computed. The identity dissolves that circularity, and by linearity the run is statistically identical to running on a uniformly random $|u_s\rangle$.

The state before QFT[†]:

$$\underbrace{\frac{1}{\sqrt{Q}} \sum_x |x\rangle |a^x \bmod N\rangle}_{\text{periodic-function form}} = \underbrace{\frac{1}{\sqrt{r}} \sum_s \left(\frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x s / r} |x\rangle \right) |u_s\rangle}_{\text{eigenphase form}}.$$

Same state, two bases of the work register: substitute $|a^x\rangle = \frac{1}{\sqrt{r}} \sum_s e^{2\pi i s x / r} |u_s\rangle$ (Lemma 1 again). Period finding and phase estimation are therefore two descriptions of one mechanism and give identical counting-register statistics.

Lemma 3

Let $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, $\varphi \in [0, 1)$. Then for every one-qubit control state,

$$c\text{-}U[(\alpha|0\rangle + \beta|1\rangle) \otimes |u\rangle] = (\alpha|0\rangle + \beta e^{2\pi i\varphi}|1\rangle) \otimes |u\rangle.$$

Proof

By linearity and $c\text{-}U|c\rangle|\psi\rangle = |c\rangle U^c|\psi\rangle$,

$$c\text{-}U(\alpha|0\rangle + \beta|1\rangle)|u\rangle = \alpha|0\rangle U^0|u\rangle + \beta|1\rangle U^1|u\rangle = \alpha|0\rangle|u\rangle + \beta e^{2\pi i\varphi}|1\rangle|u\rangle,$$

and the scalar $e^{2\pi i\varphi}$ moves from the second tensor factor to the first because scalars commute with everything; factoring $|u\rangle$ out gives the claim. $\square$

What the gate does, and what it does not

Nothing is moved and nothing is copied. The eigenvalue was already a property of the pair $(U, |u\rangle)$; the gate re-attaches it to the control, where it is a *relative* phase and therefore observable, unlike a global phase on $|u\rangle$. The target is returned unchanged, so it can drive the entire ladder.

Kickback: four correct statements and one incorrect inference

Correct

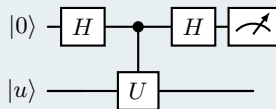
- ▶ The eigenvalue becomes a relative phase precisely because it multiplies the $|1\rangle$ branch and not the $|0\rangle$ branch.
- ▶ The target factors out *only* because $|u\rangle$ is an eigenstate; then $U|u\rangle$ is $|u\rangle$ up to a scalar.
- ▶ If the target is $|\psi\rangle = \sum_s c_s |u_s\rangle$, then $c\text{-}U(|+\rangle \otimes |\psi\rangle) = \frac{1}{\sqrt{2}} \sum_s c_s (|0\rangle + e^{2\pi i \varphi_s} |1\rangle) |u_s\rangle$, which is entangled unless all φ_s coincide.
- ▶ Entanglement appears exactly when the target mixes eigenstates with distinct eigenphases — which is what occurs in Shor's algorithm.

Incorrect

“ $c\text{-}U$ forces or projects the target into an eigenstate.”

It does not. $c\text{-}U$ is a fixed linear gate; it acts on whatever target it is given. Nothing is measured, projected, or selected. The factorisation of $|u\rangle$ in Lemma 3 works only because $|u\rangle$ was already an eigenstate.

Smallest complete example



$$\Pr[0] = \cos^2(\pi\varphi), \quad \Pr[1] = \sin^2(\pi\varphi).$$

Eigenvalue $\rightarrow$ relative phase $\rightarrow$ recombination
 $\rightarrow$ probability, on a single qubit.

Exercise 5 — kickback, and what it does not say

4 minutes

Task

State and prove phase kickback. Then write down (a) what happens if the target is a superposition of eigenstates with distinct eigenphases, and (b) the false statement that people infer from kickback, with a one-line refutation.

Exercise 5 — kickback, and what it does not say

4 minutes

Task

State and prove phase kickback. Then write down (a) what happens if the target is a superposition of eigenstates with distinct eigenphases, and (b) the false statement that people infer from kickback, with a one-line refutation.

Solution

Lemma and proof: as on the previous slides.

(a) With $|\psi\rangle = \sum_s c_s |u_s\rangle$: $c\text{-}U(|+\rangle \otimes |\psi\rangle) = \frac{1}{\sqrt{2}} \sum_s c_s (|0\rangle + e^{2\pi i \varphi_s} |1\rangle) |u_s\rangle$, entangled unless all φ_s coincide.

(b) False: **$c\text{-}U$ forces the target into an eigenstate**. It does not: $c\text{-}U$ is a fixed linear gate that acts on whatever target it is given; nothing is measured, projected or selected.

Precise phrasing: nothing is moved and nothing is copied — the eigenvalue was already a property of the pair $(U, |u\rangle)$; the controlled gate re-attaches it to the control, where it becomes an observable relative phase instead of an unobservable global one. The target is returned unchanged.

Quantum phase estimation: building the phase ladder

Let $Q = 2^t$ and $U|u\rangle = e^{2\pi i\varphi}|u\rangle$. Initialise t counting qubits in $|0\rangle^{\otimes t}$ and apply $H^{\otimes t}$: the state is $\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \otimes |u\rangle$.

For $s = 1, \dots, t$ apply $c-U^{2^{t-s}}$ with the s -th counting qubit as control (most significant first). Since $U^{2^{t-s}}|u\rangle = e^{2\pi i 2^{t-s}\varphi}|u\rangle$, kickback puts $e^{2\pi i 2^{t-s}\varphi}$ on the $|1\rangle$ branch of qubit s :

$$\bigotimes_{s=1}^t \frac{|0\rangle + e^{2\pi i 2^{t-s}\varphi} |1\rangle}{\sqrt{2}} \otimes |u\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} e^{2\pi i x\varphi} |x\rangle \otimes |u\rangle.$$

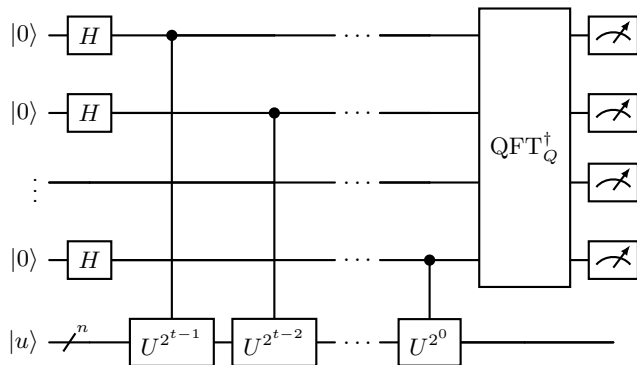
Proposition (product form = ramp) — proof

Expand the tensor product over $x = x_1 \cdots x_t$ with $x = \sum_s x_s 2^{t-s}$:

$$\bigotimes_{s=1}^t \frac{|0\rangle + e^{2\pi i 2^{t-s}\varphi} |1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{Q}} \sum_{x_1, \dots, x_t \in \{0,1\}} \prod_s e^{2\pi i x_s 2^{t-s}\varphi} |x_1 \cdots x_t\rangle = \frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x\varphi} |x\rangle,$$

using $\prod_s e^{2\pi i x_s 2^{t-s}\varphi} = e^{2\pi i \varphi \sum_s x_s 2^{t-s}} = e^{2\pi i x\varphi}$. □

The phase-estimation circuit



What each rung of the ladder inspects

The phases on the qubits are $\varphi, 2\varphi, \dots, 2^{t-1}\varphi \pmod{1}$: doubling shifts the binary expansion of φ one place left, so qubit s inspects it one bit further along. $U^{2^{t-s}}$ comes from $t-s$ *classical* repeated squarings, not 2^{t-s} applications.

Uniform magnitudes before the inverse QFT

Every amplitude of $\frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x \varphi} |x\rangle$ has modulus $1/\sqrt{Q}$. Measuring *now* gives probability $1/Q$ for every x and tells you nothing about φ . All the information is in relative phases; the inverse QFT is what converts phase into magnitude.

Exercise 6 — the ladder

6 minutes

Task

Starting from $|0\rangle^{\otimes t} |u\rangle$, derive the state after the Hadamard layer and the controlled powers. Prove the product form equals the ramp. Then state, in one sentence each: why $U^{2^{t-s}}$ is cheap, and what a measurement at this point would return.

Exercise 6 — the ladder

6 minutes

Task

Starting from $|0\rangle^{\otimes t} |u\rangle$, derive the state after the Hadamard layer and the controlled powers. Prove the product form equals the ramp. Then state, in one sentence each: why $U^{2^{t-s}}$ is cheap, and what a measurement at this point would return.

Solution

After $H^{\otimes t}$: $\frac{1}{\sqrt{Q}} \sum_x |x\rangle \otimes |u\rangle$. Applying $c-U^{2^{t-s}}$ from qubit s kicks $e^{2\pi i 2^{t-s} \varphi}$ onto that qubit's $|1\rangle$ branch, leaving the target untouched; the product-form-equals-ramp computation is the previous slide.

Cheap: $U_a^{2^j} = U_{a^{2^j \bmod N}}$, and the constant $a^{2^j \bmod N}$ is obtained by j classical repeated squarings, so the gate is one modular multiplication, not 2^j of them.

Measure now: every amplitude has modulus $Q^{-1/2}$, so the outcome is a uniformly random x — zero information about φ . All the information is in the relative phases; the inverse QFT is what converts phase into magnitude.

The state before the inverse QFT, in two bases

Theorem

After the controlled modular multiplications and before the inverse QFT,

$$\underbrace{\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle \otimes |a^x \bmod N\rangle}_{\text{periodic-function form}} = \underbrace{\frac{1}{\sqrt{r}} \sum_{s=0}^{r-1} \left(\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} e^{2\pi i x s / r} |x\rangle \right) \otimes |u_s\rangle}_{\text{eigenphase form}}.$$

Proof

Inverting the eigenvector formula with Lemma 1 gives $|a^x \bmod N\rangle = \frac{1}{\sqrt{r}} \sum_s e^{2\pi i s x / r} |u_s\rangle$. Substituting and exchanging the sums gives the right-hand side. $\square$

Consequence

In the orbit basis the work register holds the periodic sequence $a^x \bmod N$, whose period the inverse QFT will detect. In the character basis it holds one eigenphase branch $|u_s\rangle$, each tagged with a ramp of frequency s/r . Because it is the same state, both descriptions give identical counting-register statistics. Whether the work register is measured (period finding) or ignored (phase estimation) changes the description, not the mechanism.

The inverse QFT as coherent unwinding

Take one eigenphase branch, the ramp $|\Phi_\varphi\rangle = \frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x \varphi} |x\rangle$, and apply $\text{QFT}_Q^\dagger |x\rangle = \frac{1}{\sqrt{Q}} \sum_y e^{-2\pi i x y / Q} |y\rangle$.

Proposition (inverse-QFT amplitudes)

$\text{QFT}_Q^\dagger |\Phi_\varphi\rangle = \sum_{y=0}^{Q-1} A(y) |y\rangle$ with

$$A(y) = \frac{1}{Q} \sum_{x=0}^{Q-1} e^{2\pi i x \varphi} e^{-2\pi i x y / Q} = \frac{1}{Q} \sum_{x=0}^{Q-1} e^{2\pi i x \delta_y}, \quad \delta_y := \varphi - \frac{y}{Q}.$$

Summing the geometric series (with value 1 at $\delta_y = 0$ by continuity),

$$A(y) = \frac{1}{Q} \cdot \frac{1 - e^{2\pi i Q \delta_y}}{1 - e^{2\pi i \delta_y}}, \quad \Pr[y] = |A(y)|^2 = \frac{1}{Q^2} \frac{\sin^2(\pi Q \delta_y)}{\sin^2(\pi \delta_y)}$$

— the Fejér kernel, i.e. the intensity pattern of a Q -slit diffraction grating.

What the inverse QFT does not do

It does not run and store Q separate classical tests. One unitary creates all the $A(y)$ coherently; the Born rule then returns a single y with probability $|A(y)|^2$. The full spectrum is never made classically accessible — which is also why the QFT is not a fast DFT.

Three regimes

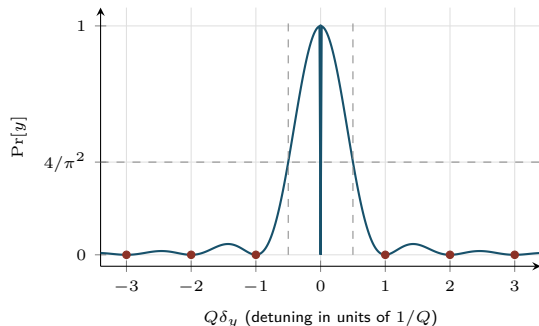
The contributions to $A(y)$ are the Q unit vectors $1, e^{2\pi i \delta_y}, \dots, e^{2\pi i (Q-1) \delta_y}$.

1. **Exact match**, $\delta_y = 0$: every vector is 1, so $A(y) = 1$ — all the probability.
2. **Exact non-matching mode**, $\delta_y = k/Q$ with $k \not\equiv 0$: the vectors are the Q -th roots of unity taken k at a time, a balanced closed polygon, so $A(y) = 0$ *exactly*. This is Lemma 1.
3. **Near match**, $|\delta_y|$ small: the vectors rotate by a small angle and form an arc, so the sum stays large. Concretely $|\delta_y| \leq \frac{1}{2Q}$ gives

$$\Pr[y] \geq 4/\pi^2 \approx 0.405,$$

from $\sin \pi \theta \leq \pi \theta$ and $\sin \theta \geq 2\theta/\pi$ on $[0, \pi/2]$.

One measurement therefore lands within 1 of $Q\varphi$ with probability at least about $8/\pi^2$, counting both neighbours.



Fejér kernel in the large- Q limit, $\Pr \approx \text{sinc}^2(\pi Q \delta_y)$. Red dots: the exact zeros at integer detunings. The dashed vertical lines bound the nearest grid point, $|\delta_y| \leq 1/(2Q)$, where the $4/\pi^2$ guarantee applies.

Exercise 7 — the peak

8 minutes

Task

Derive $A(y)$ and the closed form for $\Pr[y]$. Identify the three regimes and prove the middle one exactly. State the near-match bound. Finally, explain what the inverse QFT does not do.

Exercise 7 — the peak

8 minutes

Task

Derive $A(y)$ and the closed form for $\Pr[y]$. Identify the three regimes and prove the middle one exactly. State the near-match bound. Finally, explain what the inverse QFT does not do.

Solution

Apply $\text{QFT}^\dagger |x\rangle = \frac{1}{\sqrt{Q}} \sum_y e^{-2\pi i xy/Q} |y\rangle$ to the ramp: $A(y) = \frac{1}{Q} \sum_x e^{2\pi i x \delta_y}$ with $\delta_y = \varphi - y/Q$. Geometric series: $A(y) = \frac{1}{Q} \cdot \frac{1 - e^{2\pi i Q \delta_y}}{1 - e^{2\pi i \delta_y}}$, hence $\Pr[y] = \frac{1}{Q^2} \frac{\sin^2(\pi Q \delta_y)}{\sin^2(\pi \delta_y)}$.

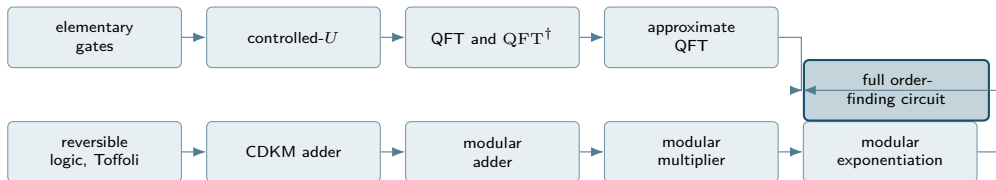
Regimes: (i) $\delta_y = 0$: all Q arrows equal 1, $A = 1$. (ii) $\delta_y = k/Q$, $k \not\equiv 0 \pmod{Q}$: the arrows are the Q -th roots of unity taken k at a time — a balanced closed polygon — so $A(y) = 0$ exactly, which is Lemma 1. (iii) $|\delta_y| \leq \frac{1}{2Q}$: the arrows form a short arc, giving $\Pr[y] \geq 4/\pi^2 \approx 0.405$.

Does not do: it does not run and store Q classical tests. One unitary creates all the $A(y)$ coherently; the Born rule then hands you one y . The spectrum is never classically accessible — which is also why the QFT is not a fast DFT.

PART 6

Building the circuit

What has to be built, and in what order



Two levels of description

Algorithmic level. Explicit unitary matrices, controlled- U , QFT, controlled powers. This is the level at which the circuit is written in code.

Fault-tolerant cost level. Clifford+ T . Clifford gates are nearly free; T is not. All resource counts are quoted in Toffoli or T gates, never in raw gate count.

Where the gates go

The Fourier transform is $\Theta(n^2)$ exact, or $O(n \log n)$ approximate. The modular exponentiation is $\Theta(n^3)$. The QFT is asymptotically the *cheapest* part of Shor's algorithm; roughly 99.997% of the gates are arithmetic. Sections V.C–V.F are therefore the bulk of the engineering.

Gate	Matrix	Action on $ b\rangle$, $b \in \{0, 1\}$	Where it is used here
I	$\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$	$ b\rangle$	padding; the “do nothing” branch of $c\text{-}U$
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	$ b \oplus 1\rangle$	preparing $ 1\rangle$ in the work register; two’s-complement negation; folding a constant bit $a_i = 1$ into a circuit
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	$i(-1)^b b \oplus 1\rangle$	not used directly; needed only to define the Pauli group
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	$(-1)^b b\rangle$	$= R_1$; the $k = 1$ case of the QFT rotation
H	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	$\frac{1}{\sqrt{2}}(0\rangle + (-1)^b 1\rangle)$	the $H^{\otimes t}$ layer; the first and last gate of every QFT factor; the X -basis measurement in the temporary-AND
$P(\theta)$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\theta} \end{pmatrix}$	$e^{i\theta b} b\rangle$	the general phase gate; all QFT rotations are instances
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	$i^b b\rangle$	$= P(\pi/2) = R_2$; Clifford generator
$S^\dagger$	$\begin{pmatrix} 1 & 0 \\ 0 & -i \end{pmatrix}$	$(-i)^b b\rangle$	inverse QFT rotation, $k = 2$
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	$e^{i\pi b/4} b\rangle$	$= P(\pi/4) = R_3$; the unit of fault-tolerant cost
$T^\dagger$	$\begin{pmatrix} 1 & 0 \\ 0 & e^{-i\pi/4} \end{pmatrix}$	$e^{-i\pi b/4} b\rangle$	appears in the Toffoli decomposition
R_k	$\begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$	$e^{2\pi i b/2^k} b\rangle$	$= P(2\pi/2^k)$; the QFT rotation ladder, $k = 2, \dots, n$

Rotations, needed for the ABC construction of $c\text{-}U$: $R_x(\theta) = e^{-i\theta X/2}$, $R_y(\theta) = e^{-i\theta Y/2}$, $R_z(\theta) = e^{-i\theta Z/2} = \text{diag}(e^{-i\theta/2}, e^{i\theta/2})$, so $P(\theta) = e^{i\theta/2} R_z(\theta)$; every $U \in U(2)$ factors as $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$. Note $R_1 = Z$, $R_2 = S$, $R_3 = T$; beyond $k = 3$ the R_k are not in $\text{Clifford}+T$ and must be synthesised approximately — which is exactly why the approximate QFT exists.

Two- and three-qubit gates

Gate	Symbol	Matrix	Action	Where it is used
CNOT	$c\text{-}X$	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$	$ a\rangle b\rangle \mapsto a\rangle b \oplus a\rangle$	copying a basis value; every sum bit of the adder; the MAJ/UMA blocks; LOAD fan-outs
CZ	$c\text{-}Z$	$\text{diag}(1, 1, 1, -1)$	$ a\rangle b\rangle \mapsto (-1)^{ab} a\rangle b\rangle$	the free correction in the temporary-AND uncomputation
$c\text{-}P(\theta)$	$c\text{-}R_k$	$\text{diag}(1, 1, 1, e^{i\theta})$	$ a\rangle b\rangle \mapsto e^{i\theta ab} a\rangle b\rangle$	every controlled rotation of the QFT and QFT [†]
SWAP	—	$\begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$	$ a\rangle b\rangle \mapsto b\rangle a\rangle$	the QFT's closing bit reversal; the in-place modular multiplier; binary shifts ($\times 2$, $/2$)
TOF	$cc\text{-}X$	8×8 permutation	$ a\rangle b\rangle c\rangle \mapsto a\rangle b\rangle c \oplus ab\rangle$	every carry in every adder; the unit in which arithmetic cost is reported
FREDKIN	$c\text{-}SWAP$	8×8 permutation	$ a\rangle b\rangle c\rangle \mapsto a\rangle a?c:b\rangle a?b:c\rangle$	controlling the swap layer of the in-place multiplier
$C^k(X)$	—	2^{k+1} perm.	X on the target iff all k controls are 1	doubly-controlled modular additions inside a controlled U_a

Three-bit gates and universality

Theorem (two-bit reversible gates are not universal)

Every reversible circuit built from gates acting on at most two bits computes an affine map $\mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$, $x \mapsto Mx + c$. Consequently none computes $(x, y, z) \mapsto (x, y, z \oplus xy)$, nor $\wedge$ — even with ancillas and garbage allowed.

Proof

Step 1: every 1- and 2-bit reversible gate is affine. For one bit the only bijections of $\mathbb{F}_2$ are $x \mapsto x$ and $x \mapsto x + 1$. For two bits, count: the reversible 2-bit gates form S_4 , of order 24, and $|\text{AGL}_2(\mathbb{F}_2)| = (2^2 - 1)(2^2 - 2) \cdot 4 = 24$ as well. Every affine bijection permutes the four points, so $\text{AGL}_2(\mathbb{F}_2) \leq S_4$; equal orders force equality.

Step 2: affineness is preserved under embedding and composition, $M_2(M_1x + c_1) + c_2 = (M_2M_1)x + (M_2c_1 + c_2)$.

Step 3. Every output bit is therefore a degree ≤ 1 polynomial over $\mathbb{F}_2$; but xy has degree 2. $\square$

The order coincidence, and its consequence

$|S_4| = |\text{AGL}_2(\mathbb{F}_2)| = 24$ is a coincidence of small numbers. For three bits $|S_8| = 40\,320$ while $|\text{AGL}_3(\mathbb{F}_2)| = 168 \cdot 8 = 1344$, so the overwhelming majority of 3-bit reversible gates are *not* affine — and the Toffoli gate is one of them. That is exactly why the step from two bits to three buys universality: non-linearity is what computation needs.

Universality, in two forms

Reversible-classical: $\{\text{TOF}\}$ suffices, since $\text{TOF}(x, y, 1) = (x, y, \text{nand}(x, y))$ and $\text{TOF}(1, x, 0) = (1, x, x)$ give NAND and copy. Replacing each gate of a size- s classical circuit by its reversible counterpart gives $O(s)$ gates and $O(s)$ ancillas.

Quantum: $\{H, S, \text{CNOT}, T\}$ and $\{H, \text{TOF}\}$ are both approximately universal. A universal set must (i) create superpositions, (ii) create entanglement, and (iii) reach an irrational relative phase; the Clifford group and $\{H, \text{CNOT}\}$ fail (iii), $\{\text{TOF}\}$ fails (i).

Solovay–Kitaev: precision is cheap, $O(\log^c(1/\epsilon))$ gates with $c \approx 4$. Circuit *width* is not: a generic n -qubit unitary needs $\Omega(4^n \log(1/\epsilon) / \log n)$ gates. Useful algorithms live in the measure-zero set with short circuits; V.D–V.E shows modular arithmetic is in it.

The Clifford group and the T gate

Definitions

The n -qubit Pauli group is $\mathcal{P}_n = \{i^k P_1 \otimes \cdots \otimes P_n\}$ with $P_j \in \{I, X, Y, Z\}$. The Clifford group is its normaliser,

$\mathcal{C}_n = \{C \in U(2^n) : CPC^\dagger \in \mathcal{P}_n \ \forall P \in \mathcal{P}_n\} / U(1)$,
generated by $\{H, S, \text{CNOT}\}$.

$T \notin \mathcal{C}_1$: $TXT^\dagger = \frac{1}{\sqrt{2}}(X + Y)$, which is not a scalar multiple of any Pauli. But $T^2 = S \in \mathcal{C}_1$.

Theorem (Gottesman–Knill)

A circuit built only from computational-basis preparation, Clifford gates, and computational-basis measurement (including adaptive Clifford corrections) can be simulated exactly in time polynomial in the number of qubits and gates.

Proof idea. Track the stabiliser group rather than 2^n amplitudes: for a stabiliser state it is generated by n commuting Paulis, storable in $O(n^2)$ bits. A Clifford gate maps generators to generators, $P \mapsto CPC^\dagger$, an $O(n)$ update each. Total $O(n^2)$ per gate.

Consequence for the architecture

Bell and GHZ states are stabiliser states, produced by Clifford circuits, and classically simulable — so entanglement alone cannot be the speed-up. Whatever makes the computation hard to simulate must live outside the Clifford group; for Clifford+ T that is the T gate.

In the surface code this is not a coincidence but the same fact seen twice: Paulis are bookkeeping, H and S are single-patch operations, CNOT is lattice surgery — all cheap. T is not transversal and must be injected from a magic state

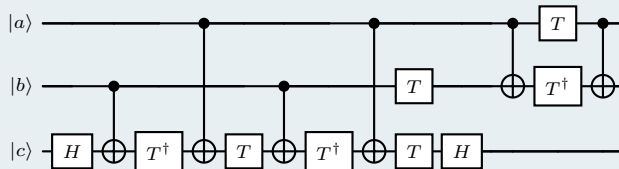
$$|A\rangle = \frac{1}{\sqrt{2}}(|0\rangle + e^{i\pi/4}|1\rangle),$$

produced by distillation or cultivation, which dominates the qubit budget.

The gate that places a circuit outside the classically simulable class is the same gate that dominates its fault-tolerant cost.

Toffoli and Fredkin in Clifford+T

Toffoli, T -count 7



Gate list: H_c ; $\text{CNOT}_{b \rightarrow c}$; $T_c^\dagger$; $\text{CNOT}_{a \rightarrow c}$; T_c ; $\text{CNOT}_{b \rightarrow c}$; $T_c^\dagger$; $\text{CNOT}_{a \rightarrow c}$; T_b , T_c ; H_c ; $\text{CNOT}_{a \rightarrow b}$; T_a , $T_b^\dagger$; $\text{CNOT}_{a \rightarrow b}$. Totals: 7 T -type gates, 6 CNOT s, 2 Hadamards, no ancilla. As drawn the T -depth is 5; Amy–Maslov–Mosca–Roetteler resynthesise to T -depth 3 with no ancilla, Selinger to T -depth 1 with 4.

Why exactly seven

TOF = $H_c C^2(Z) H_c$, and with $a, b, c \in \{0, 1\}$ the basis labels,

$$C^2(Z) = \exp \left[\frac{i\pi}{4} \left(a+b+c-(a\oplus b)-(a\oplus c)-(b\oplus c)+(a\oplus b\oplus c) \right) \right],$$

since the bracket is 0 except on $|111\rangle$, where it is 4. Each of the seven terms is a $\pi/4$ phase on a parity of the labels — a T gate conjugated by $CNOT$ s. Hence exactly seven.

Fredkin

FREDKIN = CNOT_{c→b} · TOF(*a*, *b* → *c*) · CNOT_{c→b}, so it also costs 7 *T*. It is *conservative* (it preserves the number of 1s), which is why it is the natural model for billiard-ball and optical computation. Here it appears only as the controlled SWAP in the in-place modular multiplier.

Multiply-controlled not, and the exchange rate

Proposition (decomposing $C^k(X)$, $k \geq 3$)

- (a) With $k - 2$ *clean* ancillas ($|0\rangle$ in, $|0\rangle$ out): $2k - 3$ Toffoli gates.
- (b) With one *borrowed* ancilla in an arbitrary unknown state: $O(k)$ Toffoli gates.
- (c) With no ancilla at all: $\Theta(k^2)$ elementary gates.

Proof of (a). With ancillas $b_1, \dots, b_{k-2}$, compute the conjunction in a binary chain

TOF($c_1, c_2; b_1$), TOF($b_1, c_3; b_2$), $\dots$, TOF($b_{k-3}, c_{k-1}; b_{k-2}$), so $b_{k-2} = c_1 \cdots c_{k-1}$; apply TOF(b_{k-2}, c_k ; target); run the chain backwards to restore the ancillas: $(k - 2) + 1 + (k - 2) = 2k - 3$. Compute-copy-uncompute, with the copy replaced by one controlled operation. $\square$

The exchange rate

Object	Cost
Clifford gate ($H, S, \text{CNOT}, CZ, \text{Paulis}$)	≈ 0
TOF standalone	$7T$, T -depth 3
TOF as a compute/uncompute pair	$4T$ for the pair
FREDKIN	$7T$
$C^k(X)$, clean ancillas	$(2k - 3) \times \text{TOF}$
$R_z(\theta)$ to precision ϵ	$3 \log_2(1/\epsilon) T$

The governing ratio

one arbitrary-angle rotation $\approx 3 \log_2(1/\epsilon) T$.

At $\epsilon = 10^{-6}$ that is $\approx 60 T$ gates; at $\epsilon = 10^{-15}$, ≈ 150 . A Toffoli costs 7, or 4 in a pair — so *an arbitrary rotation is one to two orders of magnitude more expensive than a Toffoli*.

This single fact decides both remaining design choices: the approximate QFT replaces the exact one, and ripple-carry arithmetic replaces the Fourier (Draper) adder.

What to count: two cost regimes

Circuit resources

For a circuit on q qubits: *qubit cost* $q = \text{inputs} + \text{ancillas}$; *gate count* and *depth*; *ancillas* (qubits not carrying input, initialised to a known state); *garbage* (outputs that are neither an input, nor a desired output, nor a restored ancilla). Two equivalent accountings, $q = \text{ancillas} + \text{inputs} = \text{garbage} + \text{outputs}$; solving the second for the garbage term is the standard way to check a design.

Reported costs often omit uncomputation

Garbage must be uncomputed before the qubits can be reused or before any interference step, so quoted counts for garbage-producing designs are optimistic, sometimes by a factor of two. *Every circuit quoted here includes its uncomputation and has zero garbage.*

	NISQ	Fault-tolerant
dominant error source	two-qubit gates, decoherence	magic-state production
count this	CNOT-count, CNOT-depth	T -count, T -depth, Toffoli count
free	— (nothing)	Clifford gates
expensive	every gate	T , TOF, rotations
volume proxy	KQ_{CNOT}	KQ_T , spacetime
preferred adder	minimise two-qubit gates	minimise Toffolis, avoid rotations

$KQ = \text{depth} \times q$ is a spacetime-volume fidelity proxy; KQ_{CNOT} and KQ_T restrict the depth to the dominant error source. A circuit is plausibly runnable without error correction when $1/KQ > \varepsilon$, the machine's per-operation failure rate.

T -count is a proxy

The honest currency is spacetime volume: logical qubits $\times$ logical time steps. T -count is a good proxy only while magic-state production is the bottleneck — with cultivation the factories shrink and idle logical qubits become a comparable share.

Gate reference: use of each gate in Shor's algorithm

Layer	Gates used	Built from	Cost
counting-register preparation	$H^{\otimes t}$	H	t Clifford gates
work-register preparation	X	X	1 Clifford gate
MAJ/UMA block	CNOT $\times 2$, TOF $\times 1$	Clifford + TOF	1 TOF; $4T$ per MAJ/UMA pair
CDKM adder (n bits)	$2n$ TOF, $(4n+1)$ CNOT	MAJ/UMA	$2n$ TOF; $4n T$ with temporary-AND
constant adder	as above, $\approx$ half deleted	compile-time folding	$\approx n$ TOF
comparator	MAJ chain, CNOT, MAJ $^{-1}$	MAJ	n TOF, $4n T$
modular adder	5 constant additions, X , CNOT	adder	$\Theta(n)$ TOF, 1 ancilla, cleared
modular doubler (p odd)	SWAP chain, $\pm p$	SWAP, adder	$\Theta(n)$ TOF, no persistent ancilla
modular multiplier	$n \times$ controlled modular add	modular adder	$\Theta(n^2)$ TOF
in-place multiplier U_a	2 multipliers, n SWAP	above + SWAP	$\Theta(n^2)$ TOF
controlled U_a	doubly-controlled adds, FREDKIN	$C^2(X)$, FREDKIN	same order, $\approx 2 \times$ constant
modular exponentiation	t controlled U_a	above	$\Theta(n^3)$ TOF; $\approx 0.3n^3$ windowed
QFT factor	H , then $c-R_k$, $k = 2..n-s+1$	H , $P(\theta)$, CNOT	1 H + $(n-s)$ rotations
$c-R_k$	$P(\pm\theta/2) \times 3$, CNOT $\times 2$	Clifford + P	$3 \log_2(1/\epsilon) T$ if $k > 3$
bit reversal	$\lfloor n/2 \rfloor$ SWAP	3 CNOT each	Clifford
QFT †	same, reversed, $R_k \mapsto R_k^\dagger$	—	same
approximate QFT	drop $c-R_k$ for $k > m$	—	$O(nm)$ gates, $m \approx \log_2 n + 2$
measurement	computational basis	—	—
AND uncomputation	H , measure, CZ	Clifford	zero T

Bit order

Mismatching the bit order between the controlled-power ladder and the QFT is the most common implementation bug. Fix the convention once (here: big-endian, x_1 on the top wire) and check it end to end on a known basis state.

Controlled- U for a single-qubit U

Definition

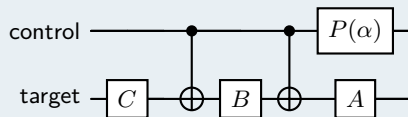
$c\text{-}U = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes U$; as a matrix on two qubits, $c\text{-}U = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & u_{00} & u_{01} \\ 0 & 0 & u_{10} & u_{11} \end{pmatrix}$.

Theorem (Barenco et al.: ABC decomposition)

For every $U \in U(2)$ there exist $A, B, C \in SU(2)$ and $\alpha \in \mathbb{R}$ with

$$ABC = \mathbb{I}, \quad U = e^{i\alpha} A X B X C.$$

Then $c\text{-}U$ is realised with two CNOTs and three single-qubit gates, plus a phase $P(\alpha)$ on the control:



Why it works. If the control is $|0\rangle$ the target sees $ABC = \mathbb{I}$. If it is $|1\rangle$ the target sees $AXBXC = e^{-i\alpha}U$, and the $P(\alpha)$ on the control supplies the missing $e^{i\alpha}$ — which is a relative phase, hence physical, hence necessary.

Constructing A, B, C

Write $U = e^{i\alpha} R_z(\beta) R_y(\gamma) R_z(\delta)$ (Euler / Z - Y - Z decomposition, always possible). Then take

$$A = R_z(\beta) R_y(\frac{\gamma}{2}), \quad B = R_y(-\frac{\gamma}{2}) R_z(-\frac{\delta+\beta}{2}), \quad C = R_z(\frac{\delta-\beta}{2}),$$

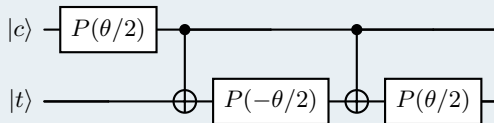
and check $ABC = \mathbb{I}$ and $AXBXC = R_z(\beta) R_y(\gamma) R_z(\delta)$ using $XR_y(\theta)X = R_y(-\theta)$ and $XR_z(\theta)X = R_z(-\theta)$.

Controlled phase gates: the case used by the QFT

The QFT only ever needs controlled versions of *diagonal* gates, and those have a short explicit form.

Decomposition of $c\text{-}P(\theta)$

$c\text{-}P(\theta) = \text{diag}(1, 1, 1, e^{i\theta})$. Then



Verification. The accumulated phase on $|c, t\rangle$ is $\exp\left(\frac{i\theta}{2}[c - (t \oplus c) + t]\right)$, and

$$\begin{array}{ll} (c, t) = (0, 0) : & 0 - 0 + 0 = 0 \\ (c, t) = (0, 1) : & 0 - 1 + 1 = 0 \\ (c, t) = (1, 0) : & 1 - 1 + 0 = 0 \\ (c, t) = (1, 1) : & 1 - 0 + 1 = 2 \Rightarrow e^{i\theta} \checkmark \end{array}$$

Cost: 2 CNOTs + 3 phase gates. For the QFT, set $\theta = 2\pi/2^k$, giving $c\text{-}R_k$ from R_{k+1} and its inverse.

Control and target are interchangeable

As a matrix, $c\text{-}R_k = \text{diag}(1, 1, 1, e^{2\pi i/2^k})$ is symmetric under exchanging the two qubits. This is a general feature of controlled-diagonal gates, and it means QFT diagrams may be drawn with either qubit carrying the control dot.

Controlling a large block

$U_a^{2^j}$ is not built by squaring a controlled gate. Three separate techniques are used.

1. Fold the power into the classical constant

$U_a^{2^j} = U_{a^{2^j \bmod N}}$. Compute $a_j := a^{2^j} \bmod N$ classically by j repeated squarings, then build *one* controlled modular multiplier by the constant a_j . Cost: one modular multiplication, independent of j .

2. Control only what must be controlled

A block $V = V_L \cdots V_1$ can be controlled by controlling every V_i , but that is wasteful. Two cheaper patterns:

- ▶ If $V = W^\dagger DW$ with D the only part that depends on the data, control only D : the conjugating W is applied and undone regardless.
- ▶ In a compute–copy–uncompute sandwich $C^{-1} \Sigma C$, control only the copy step Σ . The scratch computation C runs either way and is cleaned either way.

For the in-place modular multiplier below, this means controlling the modular *additions* (each CNOT-controlled by x_i becomes a Toffoli controlled by (ctrl, x_i)) and replacing the closing SWAPs by Fredkin gates.

3. Constant operands halve the gate count

Every addition inside Shor's algorithm has a compile-time classical operand. A CNOT controlled on a constant bit $a_i = 0$ is deleted; one controlled on $a_i = 1$ becomes an X ; a Toffoli with a constant control collapses to a CNOT or vanishes. This is not a special case but the normal one.

The product form, and the $\Theta(n^2)$ gate count

Theorem (product form)

For $Q = 2^n$ and a basis state $|j\rangle = |j_1 \cdots j_n\rangle$ (big-endian),

$$F_{2^n} |j_1 \cdots j_n\rangle = \frac{1}{2^{n/2}} \bigotimes_{\ell=1}^n \left(|0\rangle + e^{2\pi i (0.j_{n-\ell+1} \cdots j_n)} |1\rangle \right), \quad 0.j_m \cdots j_n = \sum_{s \geq 1} j_{m+s-1} 2^{-s}.$$

Proof

This is the ladder identity applied to $j/2^n = \sum_{\ell} j_{\ell} 2^{-\ell}$: expanding $\frac{1}{\sqrt{Q}} \sum_k e^{2\pi i j k / 2^n} |k\rangle$ over $k = k_1 \cdots k_n$, $k = \sum_t k_t 2^{n-t}$, factorises because $\chi_j(k) = \prod_t \chi_j(k_t 2^{n-t})$ — the multiplicativity of characters. $\square$

The structural point

The QFT of a *basis state* is always a product state — never entangled. This is what permits a small circuit. (Applied to a superposition, the QFT generally does produce entanglement.) Realising that the product form and the phase-estimation ladder are the same identity read in opposite directions halves what has to be remembered.

Building one factor

Lemma (one output factor)

Fix $s \in \{1, \dots, n\}$ and suppose qubits $s, s+1, \dots, n$ are in the basis state $|j_s j_{s+1} \dots j_n\rangle$. Apply to qubit s first a Hadamard, then, for each $k = 2, 3, \dots, n-s+1$, a controlled- R_k with control qubit $s+k-1$ and target qubit s . Then qubit s ends in

$$\frac{1}{\sqrt{2}} \left(|0\rangle + e^{2\pi i (0.j_s j_{s+1} \dots j_n)} |1\rangle \right),$$

and qubits $s+1, \dots, n$ are unchanged.

Proof

$H|j_s\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^{j_s} |1\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i 0.j_s} |1\rangle)$, since $0.j_s = j_s/2$. Controlled- R_k with control $s+k-1$ (a basis state, so no back-action) multiplies the $|1\rangle$ component by $e^{2\pi i j_{s+k-1}/2^k}$. Accumulating $k = 2, \dots, n-s+1$:

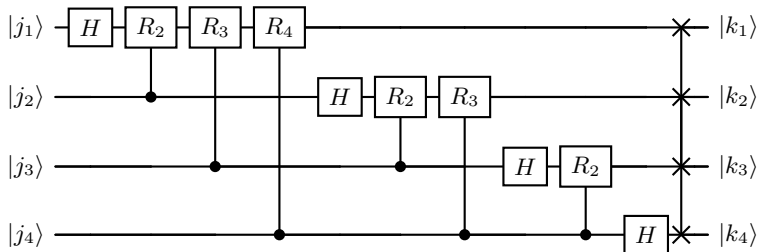
$$\exp\left(2\pi i \sum_{k=2}^{n-s+1} \frac{j_{s+k-1}}{2^k}\right) = \exp\left(2\pi i \sum_{p=1}^{n-s} \frac{j_{s+p}}{2^{p+1}}\right) = e^{2\pi i (0.0j_{s+1}j_{s+2}\dots j_n)}.$$

Combined with $e^{2\pi i 0.j_s}$ and $0.j_s + 0.0j_{s+1} \dots j_n = 0.j_s j_{s+1} \dots j_n$, the claim follows. $\square$

Order matters

The lemma requires qubits $s+1, \dots, n$ to *still be basis states* when they act as controls. So the circuit processes qubit 1 first, then qubit 2, and so on: once qubit s has been Hadamarded it is no longer a basis state and may not be used as a control.

The QFT circuit



Algorithm (QFT on n qubits)

1. For $s = 1, 2, \dots, n$:
 - (a) apply H to qubit s ;
 - (b) for $k = 2, 3, \dots, n - s + 1$, apply $c\text{-}R_k$ with control qubit $s + k - 1$, target qubit s .
2. Reverse the qubit order with $\lfloor n/2 \rfloor$ SWAPs (qubit $s \leftrightarrow$ qubit $n + 1 - s$).

Cost and the closing swaps

n Hadamards, $\binom{n}{2}$ controlled rotations, $\lfloor n/2 \rfloor$ swaps:

$$\frac{n(n+1)}{2} + \left\lfloor \frac{n}{2} \right\rfloor = \Theta(n^2) \text{ gates,}$$

depth $\Theta(n)$ if commuting rotations are parallelised.

Why the swaps. Output factor ℓ depends on the tail $j_{n-\ell+1} \cdots j_n$, so the natural circuit emits the qubits in *reversed* order: wire s carries output factor $t = n - s + 1$. The swaps repair that — or they may be dropped and the relabelling done classically.

QFT and $\text{QFT}^\dagger$ in code

```
# qubits[0] is the MOST significant (top wire).
#  $R_k = P(2\pi / 2^{**k})$ 

def qft(circ, q):
    n = len(q)
    for s in range(n):                #  $s = 0..n-1$ 
        circ.h(q[s])
        for k in range(2, n - s + 1):
            ctrl = q[s + k - 1]
            circ.cp(2*pi/2**k, ctrl, q[s])
    for s in range(n // 2):          # closing swaps
        circ.swap(q[s], q[n - 1 - s])

def iqft(circ, q):
    # exact inverse: reverse the order of every
    # operation and conjugate each rotation
    n = len(q)
    for s in range(n // 2):
        circ.swap(q[s], q[n - 1 - s])
    for s in reversed(range(n)):
        for k in reversed(range(2, n - s + 1)):
            ctrl = q[s + k - 1]
            circ.cp(-2*pi/2**k, ctrl, q[s])
        circ.h(q[s])
```

The inverse

$\text{QFT}^\dagger$ is the same circuit run backwards with $R_k \mapsto R_k^\dagger$, i.e. every rotation angle negated. There is nothing else to derive: $(\prod_i G_i)^\dagger = \prod_i G_{L-i}^\dagger$, and $H^\dagger = H$, $\text{SWAP}^\dagger = \text{SWAP}$.

$c\text{-}R_k$ from elementary gates

If the target framework has no native controlled-phase, expand each $\text{cp}(\theta, c, t)$ as

$$P(\theta/2)_c, \text{CNOT}_{c \rightarrow t}, P(-\theta/2)_t, \text{CNOT}_{c \rightarrow t}, P(\theta/2)_t.$$

Two checks to run first

- Build the $2^n \times 2^n$ matrix of qft for $n = 3, 4$ and compare with $(F_{2^n})_{kj} = \omega^{kj} / \sqrt{2^n}$ entrywise.
- Check $\text{QFT}^\dagger \circ \text{QFT} = \mathbb{I}$ numerically. A failure here is almost always the swap layer or the bit order.

The approximate QFT

The rotation R_k is exponentially close to the identity for large k : $\|R_k - \mathbb{I}\| = |e^{2\pi i/2^k} - 1| \leq 2\pi/2^k$. Coppersmith observed that these gates may simply be dropped.

Definition and cost

AQFT $_m$ is the QFT algorithm with every $c\text{-}R_k$ for $k > m$ omitted. Then

$$\|\text{QFT}_{2^n} - \text{AQFT}_m\| \leq \frac{2\pi n}{2^m},$$

and AQFT $_m$ uses $O(nm)$ gates. Taking $m = \lceil \log_2(2\pi n/\epsilon) \rceil$ gives error at most ϵ with $O(n \log(n/\epsilon))$ gates.

Proof of the bound

For products of unitaries, $\|V - \tilde{V}\| \leq \sum_{\ell} \|V_{\ell} - \tilde{V}_{\ell}\|$. There are fewer than n omitted rotations per wire and n wires; each contributes at most $2\pi 2^{-k} \leq 2\pi 2^{-m}$, and the geometric tail sums to less than twice the first term. $\square$

Effect on the cost model

In the Clifford+ T cost model an arbitrary-angle rotation costs $\approx 3 \log_2(1/\epsilon)$ T gates, an order of magnitude more than a Toffoli. Dropping rotations with $k \gtrsim \log_2 n + 2$ is what turns the QFT from $\Theta(n^2)$ expensive rotations into $O(n \log n)$ gates with negligible distortion. Every serious resource estimate uses the approximate QFT.

Exercise 8 — the QFT circuit

5 minutes

Task

State the product form. Draw the 3-qubit circuit with the correct gate order. Give the gate count. Explain the closing swaps and how to build $\text{QFT}^\dagger$.

Exercise 8 — the QFT circuit

5 minutes

Task

State the product form. Draw the 3-qubit circuit with the correct gate order. Give the gate count. Explain the closing swaps and how to build $\text{QFT}^\dagger$.

Solution

$F_{2^n} |j_1 \cdots j_n\rangle = 2^{-n/2} \bigotimes_{\ell=1}^n \left(|0\rangle + e^{2\pi i(0.j_{n-\ell+1} \cdots j_n)} |1\rangle \right)$, which is the ladder identity applied to $j/2^n = \sum_{\ell} j_{\ell} 2^{-\ell}$. The QFT of a basis state is always a product state; that is what makes a small circuit possible.

Circuit ($n = 3$, $R_k = \text{diag}(1, e^{2\pi i/2^k})$): wire 1: H , then R_2 controlled by wire 2, then R_3 controlled by wire 3; wire 2: H , then R_2 controlled by wire 3; wire 3: H ; then $\lfloor n/2 \rfloor$ swaps. Pattern: each H writes the leading bit of a binary fraction, each controlled R_k appends the k -th bit without disturbing the wire supplying it.

Count: n Hadamards, $\binom{n}{2}$ controlled rotations, $\lfloor n/2 \rfloor$ swaps $\Rightarrow \Theta(n^2)$. Dropping rotations with $k \gtrsim \log_2 n + 2$ gives the approximate QFT at $O(n \log n)$ gates.

Swaps: output factor ℓ depends on the tail $j_{n-\ell+1} \cdots j_n$, so the natural circuit emits qubits in reversed order; the swaps repair that, or one relabels classically. $\text{QFT}^\dagger$ is the same circuit reversed with $R_k \mapsto R_k^\dagger$.

Addition: the carries are the cost

Schoolbook binary addition of a and b :

$$s_i = a_i \oplus b_i \oplus c_i, \quad c_{i+1} = \text{MAJ}(a_i, b_i, c_i) := a_i b_i \oplus a_i c_i \oplus b_i c_i, \quad c_0 = 0.$$

The sum bits are $\mathbb{F}_2$ -linear given the carries, so they are free (CNOTs). The carries are quadratic, so each costs at least one Toffoli-like gate. And the carries form a chain of length n , so the naive depth is $\Theta(n)$.

Definition (maj and uma blocks)

On an ordered triple of wires (c, b, a) define, in this order:

$$\text{MAJ} : \quad \text{CNOT}(a \rightarrow b), \quad \text{CNOT}(a \rightarrow c), \quad \text{TOF}(c, b \rightarrow a)$$

$$\text{UMA} : \quad \text{TOF}(c, b \rightarrow a), \quad \text{CNOT}(a \rightarrow c), \quad \text{CNOT}(c \rightarrow b)$$

Each block contains exactly one Toffoli gate.

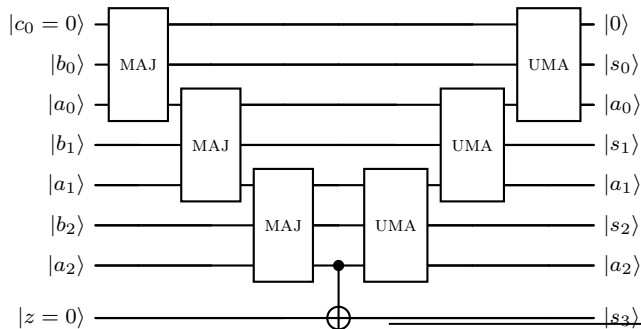
Lemma (what the blocks do)

$$\text{MAJ} : (c, b, a) \mapsto (c \oplus a, b \oplus a, \text{MAJ}(a, b, c)), \quad \text{UMA} : (c \oplus a, b \oplus a, \text{MAJ}(a, b, c)) \mapsto (c, a \oplus b \oplus c, a).$$

Proof of MAJ. Track the assignments: $b \mapsto b \oplus a$, then $c \mapsto c \oplus a$, then

$a \mapsto a \oplus (b \oplus a)(c \oplus a) = a \oplus bc \oplus ab \oplus ac \oplus a = ab \oplus ac \oplus bc = \text{MAJ}(a, b, c)$, using $a^2 = a$ and $a \oplus a = 0$ over $\mathbb{F}_2$. UMA reverses the same computation and leaves the sum bit. $\square$

The CDKM ripple-carry adder



Theorem

Interleave wires $c_0, b_0, a_0, b_1, a_1, \dots, b_{n-1}, a_{n-1}, z$; run a MAJ chain up, one CNOT copying the top carry to z , then a UMA chain down: $|0\rangle |b\rangle |a\rangle |0\rangle \mapsto |0\rangle |(a+b) \bmod 2^n\rangle |a\rangle |s_n\rangle$, an in-place $(n+1)$ -bit sum with a preserved and *no garbage*.

qubits	$2n + 2$
Toffoli gates	$2n$
CNOTs	$4n + 1$
T -count	$14n$ (7 T per Toffoli)
T -count with temporary-AND	$4n$
garbage	0

Variants

Dropping z gives a mod- 2^n adder on $2n + 1$ qubits; freeing c_0 gives an adder with carry-in; *running the circuit backwards subtracts*; a constant addend deletes or fixes roughly half the Toffolis at compile time.

The temporary logical-and: Toffolis in pairs cost $4 T$

Theorem (measurement-based and uncomputation)

Let $|\Psi\rangle = \sum_{x,y \in \{0,1\}} \alpha_{xy} |x\rangle |y\rangle |xy\rangle$ be any state in which the third qubit holds the AND of the first two. Measure the third qubit in the X basis $\{|+\rangle, |-\rangle\}$. Then

1. each outcome occurs with probability exactly $\frac{1}{2}$;
2. on outcome $+$, the remaining state is $\sum_{x,y} \alpha_{xy} |x\rangle |y\rangle$ — the AND is erased;
3. on outcome $-$, the state is $\sum_{x,y} (-1)^{xy} \alpha_{xy} |x\rangle |y\rangle$, which a single CZ corrects.

The whole uncomputation uses *zero* T gates: CZ is Clifford.

Proof

Projecting the third qubit onto $|\pm\rangle$ gives, up to normalisation, $\frac{1}{\sqrt{2}} \sum_{x,y} (\pm 1)^{xy} \alpha_{xy} |x\rangle |y\rangle$, using $\langle +|t\rangle = 1/\sqrt{2}$ and $\langle -|t\rangle = (-1)^t/\sqrt{2}$. The squared norm is $\frac{1}{2}$ either way, giving (1). For $+$ all signs are $+1$, giving (2). For $-$ the factor $(-1)^{xy}$ is exactly the action of $CZ = \text{diag}(1, 1, 1, -1)$, and $CZ^2 = I$, giving (3). $\square$

Consequence

Computing the AND costs $4 T$; uncomputing it costs none. A Toffoli occurring as a compute/uncompute pair therefore costs $4 T$ in total rather than $2 \times 7 = 14$. Each carry in the CDKM adder is computed by a MAJ block and uncomputed by the matching UMA block, so *almost every Toffoli in Shor's algorithm is half of such a pair*: the n -bit adder costs $4n + O(1) T$ gates, not $14n$.

Subtraction, comparison, and controlled addition

Subtraction

Running the CDKM adder backwards computes $|a\rangle |b\rangle \mapsto |a\rangle |b - a \bmod 2^n\rangle$ at the same cost. Its carry-out wire holds the *borrow*: $z = 1$ exactly when $b < a$. Equivalently, use two's complement, $b - a = b + \neg a + 1$, at the price of $2n$ extra X gates.

Comparator

$|a\rangle |b\rangle |0\rangle \mapsto |a\rangle |b\rangle [|a \geq b]\rangle$ using n Toffolis and no garbage: run the MAJ chain, copy the final carry to the output qubit with one CNOT, and run the MAJ chain backwards. *Proof.* The MAJ chain leaves c_n on the top a -wire, and c_n is the carry-out of $a + \neg b + 1 = a - b + 2^n$, which is 1 iff $a - b \geq 0$, i.e. iff $a \geq b$. (Dropping the $+1$, i.e. computing $a + \neg b$, gives $[a > b]$ instead.) Since the chain is computed then uncomputed, each of its n Toffolis is half of a temporary-AND pair, so the T -count is $4n$.

Controlled adder

$|k\rangle |a\rangle |b\rangle \mapsto |k\rangle |a\rangle |b + ka\rangle$, $k \in \{0, 1\}$: control only the UMA half, giving $2n$ Toffolis for the carries plus n for the controlled sum bits. Note that the carry chain itself need *not* be controlled — it is computed and uncomputed either way.

Constant operands

Since every addition inside Shor's algorithm has a compile-time classical operand, roughly half the gates disappear before the circuit is emitted. Report gate counts *after* this folding, and always including uncomputation — quoted counts for garbage-producing designs are optimistic, sometimes by a factor of two.

Modular addition

Fix an n -bit modulus $N < 2^n$ and use registers of $n + 1$ qubits, so intermediate values up to $2N$ fit.

Theorem (modular adder, classical constant $a < N$)

- | | |
|--|---|
| 1. ADD(a) | $ x\rangle \mapsto x + a\rangle$, with $x + a < 2N$ |
| 2. SUB(N) | $\mapsto x + a - N\rangle$, possibly negative |
| 3. CNOT from the sign qubit to an ancilla t | $t = \lceil x + a < N \rceil$ |
| 4. ADD(N) controlled on t | $\mapsto (x + a) \bmod N\rangle$ |
| 5. SUB(a) | $\mapsto (x + a) \bmod N - a\rangle$ |
| 6. X on t , then CNOT from the sign qubit to t | $t \mapsto 0$ |
| 7. ADD(a) | $\mapsto (x + a) \bmod N\rangle$ |

This implements $|x\rangle \mapsto |(x + a) \bmod N\rangle$ for all $0 \leq x < N$, using one ancilla which is returned to $|0\rangle$, and five additions of classical constants plus $O(1)$ Clifford gates.

Proof (the part worth checking: clearing the ancilla)

Steps 1–2 give $x + a - N$, negative exactly when $x + a < N$; in two's complement the sign qubit is $\lceil x + a < N \rceil$, so step 3 records it without disturbing the value, and step 4 restores $R := (x + a) \bmod N \in [0, N)$. Now

$$t = 1 \iff x + a < N \iff R = x + a \geq a, \quad t = 0 \iff R = x + a - N < a,$$

the last because $x < N$. Hence $t = \lceil R \geq a \rceil$, *recomputable from the register alone*. Step 5 forms $R - a$, whose sign qubit is $\lceil R < a \rceil = \neg t$; applying X to t makes it $\neg t$, and the CNOT then sets $t \mapsto \neg t \oplus \neg t = 0$. Step 7 restores R . □

Theorem (out-of-place multiplication by a constant)

Let $\gcd(a, N) = 1$. The circuit

$$\prod_{i=0}^{n-1} \left(\text{ADD}_{\text{mod } N} (2^i a \bmod N) \text{ controlled on } x_i \right)$$

implements $|x\rangle |0\rangle \mapsto |x\rangle |ax \bmod N\rangle$ using n controlled modular additions, hence $\Theta(n^2)$ Toffoli gates.

Proof. $ax \bmod N = \left(\sum_i x_i (2^i a \bmod N) \right) \bmod N$, and the constants $2^i a \bmod N$ are computed classically at compile time. Each term is added exactly when $x_i = 1$. All partial sums are reduced mod N , so the accumulator never exceeds N . $\square$

Theorem (in-place multiplication)

With $\bar{a} := a^{-1} \bmod N$ computed classically by the extended Euclidean algorithm,

$$|x\rangle |0\rangle \xrightarrow{\text{out-of-place with } a} |x\rangle |ax\rangle \xrightarrow{n \text{ SWAPS}} |ax\rangle |x\rangle \xrightarrow{\text{inverse of out-of-place with } \bar{a}} |ax\rangle |0\rangle.$$

So $U_a : |x\rangle \mapsto |ax \bmod N\rangle$ costs two out-of-place modular multiplications, n SWAPS, and one n -qubit ancilla register.

Proof. The third step is the inverse of $|y\rangle |0\rangle \mapsto |y\rangle |\bar{a}y\rangle$ applied with $y = ax$: it maps $|ax\rangle |z\rangle \mapsto |ax\rangle |z - \bar{a}(ax)\rangle$, and $\bar{a}(ax) \equiv x$, so $|x\rangle \mapsto |0\rangle$. Invertibility of a is exactly what makes an in-place circuit possible, and it holds because $\gcd(a, N) = 1$ is checked first. $\square$

Modular exponentiation: the controlled-power ladder

Theorem

Let $\gcd(a, N) = 1$ and let t be the exponent-register size. Define the classical constants $a_\ell := a^{2^\ell} \bmod N$ for $\ell = 0, \dots, t-1$, computed by t classical modular squarings. Then

$$\prod_{\ell=0}^{t-1} \left(U_{a_\ell} \text{ controlled on exponent qubit } \ell \right) : |k\rangle |1\rangle \mapsto |k\rangle |a^k \bmod N\rangle$$

for every $k \in \{0, \dots, 2^t - 1\}$, where U_{a_ℓ} is the in-place modular multiplier.

Proof. Writing $k = \sum_\ell k_\ell 2^\ell$, the circuit applies $\prod_\ell U_{a_\ell}^{k_\ell} = U_{a^k \bmod N}$, since $a \mapsto U_a$ is a group homomorphism ($U_a U_b = U_{ab \bmod N}$ on basis states, and both sides are linear). Then $U_{a^k} |1\rangle = |a^k \bmod N\rangle$. $\square$

This circuit and the kickback ladder are the same

The ladder of controlled powers $c \cdot U^{2^\ell}$ that phase estimation calls for is the modular exponentiation that Shor's original formulation calls for. Each box is one in-place modular multiplication by a classically precomputed constant, $\Theta(n^2)$ Toffoli gates.

Schoolbook cost

$$\underbrace{\Theta(n)}_{\text{multiplications}} \times \underbrace{\Theta(n)}_{\text{modular additions each}} \times \underbrace{\Theta(n)}_{\text{Toffolis each}} = \Theta(n^3) \text{ Toffoli gates on } \Theta(n) \text{ qubits.}$$

Reducing the constant factor: windowing

The idea

A modular multiplication is n controlled modular additions, one per bit. If instead c bits are consumed at a time and the appropriate multiple of a is looked up from a classically precomputed table of size 2^c , there is one addition per c bits. A lookup of a table of size 2^c costs $\approx 2^c$ Toffolis while an addition costs $\approx n$, so windows of size $c \approx \log_2 n$ are effectively free and the number of additions drops by a factor $\log n$.

Table lookup (QROM / unary iteration)

$|j\rangle |0^n\rangle \mapsto |j\rangle |T[j]\rangle$ for $j < 2^c$ costs $2^c - 1$ Toffoli gates and $c - 1$ ancillas: a binary tree of Toffolis computes the indicator of each j in turn, each indicator controlling a layer of CNOTs that xors $T[j]$ into the output. Uncomputing a lookup is cheaper still — measurement-based uncomputation costs $O(2^{c/2})$.

Effect

Source of the constant	Mechanism	Rough gain
constant-operand folding	compile-time deletion of Toffolis	$\approx 2\times$
temporary logical-AND	$4T$ per pair instead of 14	$3.5\times$ in T
windowing, $c_e + c_m \lesssim \log_2 n$	fewer additions per multiplication	$\approx 25\times$ at $n = 2048$
combined	$\approx 10 n^3 \rightarrow \approx 0.3 n^3$ Toffoli	$\approx 40\times$

The Fourier (Draper) adder

The identity $\text{QFT}_N^\dagger \text{diag}(\omega_N^{ak}) \text{QFT}_N = T_a$, $T_a |b\rangle = |b + a \bmod N\rangle$, read left to right, is an algorithm: to add, Fourier transform, apply phases, transform back.

Theorem (Draper adder)

With $\Phi\text{ADD}(a) := \bigotimes_{j=1}^n P(2\pi a/2^j)$ acting on qubit j (MSB first),

$$\text{QFT}_{2^n}^\dagger \Phi\text{ADD}(a) \text{QFT}_{2^n} |b\rangle = |(a + b) \bmod 2^n\rangle.$$

No ancilla qubits, and for classical a exactly n single-qubit phase gates between the two transforms.

Proof. $\text{QFT} |b\rangle = 2^{-n/2} \sum_k \omega^{bk} |k\rangle$; $\Phi\text{ADD}(a)$ multiplies $|k\rangle$ by $\prod_j e^{2\pi i a k_j / 2^j} = \omega^{ak}$, giving $\text{QFT} |a + b\rangle$; apply $\text{QFT}^\dagger$. $\square$

n -bit adder	anc.	$n=32$	$n=256$	$n=2048$
CDKM, $7T/\text{TOF}$ ($14n$)	2	448	3 584	28 672
CDKM, temporary-AND ($4n$)	n	128	1 024	8 192
Draper, classical addend	0	3 830	30 600	245 000
Draper, quantum addend	0	126 000	7.9×10^6	5.0×10^8

T -count, rotation synthesis at $\epsilon = 10^{-12}$ per rotation.

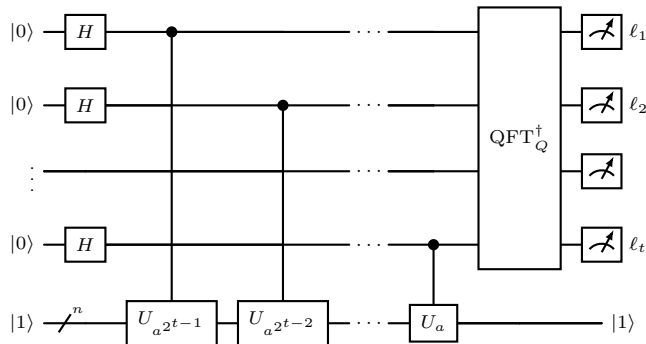
The trade-off

The Draper adder is the direct algorithmic reading of the identity that makes the QFT interesting, and it is the wrong circuit to build fault-tolerantly: rotations cost $3 \log_2(1/\epsilon) T$ and Toffolis cost 4.

- ▶ On a NISQ machine, where ancillas are the scarcest resource, the Draper adder can win — it needs zero.
- ▶ On a fault-tolerant machine ripple-carry wins by $\approx 30\times$.

Beauregard's $2n + 3$ -qubit circuit was state of the art when qubits were the binding constraint; every modern estimate uses ripple-carry.

The complete order-finding circuit



Reading the circuit

The top (most significant) counting wire drives $U_a^{2^{t-1}}$, the bottom drives U_a . Each $U_a^{2^j} = U_{a^{2^j \bmod N}}$ is *one* modular multiplication by a classical constant. The work register is a catalyst: it returns to $|1\rangle$ and is never measured.

Sizes: $n = \lceil \log_2 N \rceil$ work qubits; $t = 2n + O(1)$ counting qubits with $N^2 \leq Q = 2^t < 2N^2$ (Part VI). For $N = 21$: $n = 5$, and $441 \leq Q < 882$ forces $Q = 512$, $t = 9$.

Semiclassical variant: one ancilla instead of t

Every control of the $\text{QFT}^\dagger$ is measured at the end and never reused, so by deferred measurement each two-qubit rotation can be replaced by a *classically* controlled one-qubit rotation applied *after* measuring its control (Griffiths–Niu). Processing bit $k = t - 1, \dots, 0$, the correction is $P(\theta_k)$ with $\theta_k = -\pi \sum_{j>k} 2^{k-j} \mu_j$, the μ_j being the already-measured higher bits. Exact, not an approximation.

The algorithm in code

```
def order_finding_circuit(N, a):
    n = ceil(log2(N))
    t = (N*N - 1).bit_length()      #  $Q=2^t$  in  $[N^2, 2N^2)$ 
    ctr = QuantumRegister(t)        # counting
    work = QuantumRegister(n + 1)   # work + overflow
    anc = QuantumRegister(n + 2)    # adder scratch
    circ = QuantumCircuit(ctr, work, anc)
    circ.h(ctr)                      # superposition
    circ.x(work[n])                  # work := |1>
    for j in range(t):               # controlled powers
        const = pow(a, 2**(t-1-j), N) # CLASSICAL
        c_mult_mod(circ, ctrl=ctr[j], const=const,
                    N=N, x=work, anc=anc)
    iqft(circ, ctr)
    circ.measure(ctr, cbits)
    return circ, t

def shor(N):
    if N % 2 == 0: return 2
    if is_perfect_power(N): return base_of(N)
    while True:
        a = randint(2, N-1)
        g = gcd(a, N)
        if g > 1: return g          # lucky
        circ, t = order_finding_circuit(N, a)
        ell = run(circ)
        r = recover_order(ell, 2**t, N, a) # Part VI
        if r is None or r % 2: continue
        x = pow(a, r//2, N)
        if x == N - 1: continue
        f = gcd(x - 1, N)
        if 1 < f < N: return f
```

```
def c_mult_mod(circ, ctrl, const, N, x, anc):
    """|ctrl>|x> -> |ctrl>|const*x mod N>"""
    inv = pow(const, -1, N)          # classical
    out_of_place(circ, ctrl, const, N, x, anc)
    cswap_register(circ, ctrl, x, anc)
    out_of_place_inv(circ, ctrl, inv, N, x, anc)

def out_of_place(circ, ctrl, c, N, x, acc):
    """|x>|0> -> |x>|c*x mod N>, ctrl-gated"""
    for i in range(len(x)):
        k = (c * 2**i) % N           # classical
        # doubly controlled: ctrl AND x[i]
        cc_add_mod(circ, [ctrl, x[i]], k, N, acc)

def cc_add_mod(circ, ctrls, k, N, acc):
    """acc -> (acc + k) mod N, on all ctrls"""
    cc_add_const(circ, ctrls, k, acc) # step 1
    sub_const(circ, N, acc)           # step 2
    circ.cx(acc.sign, flag)           # step 3
    c_add_const(circ, flag, N, acc)   # step 4
    cc_sub_const(circ, ctrls, k, acc) # step 5
    circ.x(flag); circ.cx(acc.sign, flag) # step 6
    cc_add_const(circ, ctrls, k, acc) # step 7
```

Implementation checklist

Verify $\text{QFT}^\dagger \circ \text{QFT} = \mathbb{I}$; then the adder exhaustively for $n \leq 6$; then `cc_add_mod` over all (x, k, ctrl) for small N , checking ancilla cleanliness; then the bit order end to end.

Resource summary for factoring

Asymptotics (logical)

Stage	Cost
$H^{\otimes t}$	$t = 2n + O(1)$ gates
modular exponentiation	$\Theta(n^3)$ Toffoli
QFT [†] exact / approximate	$\Theta(n^2)$ / $O(n \log n)$ gates
continued fractions	$O(n^3)$ bit ops (classical)
total quantum	$\Theta(n^3)$, qubits $\Theta(n)$

Concretely $\approx 0.3 n^3$ Toffoli with windowed arithmetic; $T \approx 4 \times$ Toffoli count.

Simulated counts (Häner–Roetteler–Svore)

One modular constant multiplication costs $32n^2(\log_2 n - 2) + 14.73 n^2$ Toffoli, and the whole factoring run, $2n$ of them, costs $(64(\log_2 n - 2) + 29.46)n^3$.

$\lceil \log_2 N \rceil$	qubits	Toffoli gates
512	1 026	6.41×10^{10}
1 024	2 050	5.81×10^{11}
2 048	4 098	5.20×10^{12}
3 072	6 146	1.86×10^{13}
7 680	15 362	3.30×10^{14}
15 360	30 722	2.87×10^{15}

Twenty years of constant factors

	Beauregard '03	Gidney–Ekerå '21	Gidney '25
arithmetic	Fourier (Draper)	ripple-carry, windowed	approximate residues
logical qubits	$2n + 3$	$\approx 3n$	$\approx 0.7n$
Toffoli	$O(n^3 \log n)$	$\approx 0.3n^3$	higher, traded for qubits
<i>Physical, RSA-2048; error 10^{-3}, cycle $1 \mu\text{s}$:</i>			
physical qubits	—	20 million	< 1 million
runtime	—	8 hours	< 1 week

Two caveats

The Ekerå–Håstad variant reduces the phase-estimation rounds from $2n$ to $n/2$ at the cost of a lattice-enumeration post-processing, dividing every Toffoli count by 4 without saving qubits.

These figures assume a planar grid with nearest-neighbour connectivity, a 10^{-3} physical error rate below threshold, a $1 \mu\text{s}$ cycle time, and fast classical decoding. No existing device meets all of these at scale. They answer “how much hardware of this quality would be needed”, not “when will it exist”.

PART 7

Post-processing: from ℓ to r

From ℓ/Q to r : Legendre's theorem

The peak concentrates ℓ on the grid point nearest Qs/r , so $|\ell/Q - s/r| \leq \frac{1}{2Q}$. Choosing $Q \geq N^2$ makes this error smaller than $1/(2r^2)$ — exactly the threshold at which the rational is *uniquely* recoverable.

Theorem (Legendre)

Let $x \in \mathbb{R}$ and let p/q be a rational in lowest terms, $q \geq 1$, with $|x - \frac{p}{q}| < \frac{1}{2q^2}$. Then p/q is one of the convergents of the continued-fraction expansion of x , computable in $O(\log^3 q)$ bit operations.

Corollary (why $t = 2n + O(1)$)

If $r < N$ and $|\ell/Q - s/r| \leq \frac{1}{2Q}$ with $Q \geq N^2$, then running continued fractions on ℓ/Q and keeping the last convergent with denominator $< N$ returns s/r in lowest terms.

Proof. Write $s/r = s'/r'$ in lowest terms, $r' \leq r < N$. Then

$$\left| \frac{\ell}{Q} - \frac{s'}{r'} \right| \leq \frac{1}{2Q} \leq \frac{1}{2N^2} < \frac{1}{2r'^2},$$

so Legendre applies with $p/q = s'/r'$. □

The register-size rule

The nearest grid point satisfies $|\delta_y| \leq \frac{1}{2Q}$, and Legendre needs $\frac{1}{2Q} < \frac{1}{2r'^2}$ with $r' \leq r < N$. So $Q \geq N^2$ suffices, and the standard choice is

$$N^2 \leq Q = 2^t < 2N^2, \quad t = 2n + O(1).$$

This answers “why $Q \approx N^2$ and not $Q \approx N$ ”: the denominator can be as large as $r < N$, so the precision needed is better than $1/(2N^2)$.

Given a measured ℓ

1. form ℓ/Q and expand it as a continued fraction;
2. list the convergents p_k/q_k ;
3. keep those with $q_k < N$;
4. for each candidate $r' := q_k$, test $a^{r'} \bmod N$;
5. reject r' if $a^{r'} \not\equiv 1$;
6. note that s/r reduces to $\frac{s/\gcd(s,r)}{r/\gcd(s,r)}$, so the recovered denominator is $r/\gcd(s,r)$ — a *divisor* of r ;
7. when only a divisor is found, test small multiples $2r', 3r', \dots$;
8. or combine runs: lcm of two recovered denominators;
9. verify the order before extracting factors.

```
def convergents(num, den):
    cf, n, d = [], num, den
    while d:
        cf.append(n // d)
        n, d = d, n - (n // d) * d
    h1, h2, k1, k2 = 1, 0, 0, 1
    out = []
    for ai in cf:
        h, k = ai*h1 + h2, ai*k1 + k2
        out.append((h, k))
        h2, h1, k2, k1 = h1, h, k1, k
    return out # (numerator, denominator)

def recover_order(ell, Q, N, a):
    cand = [k for (h, k) in convergents(ell, Q)
            if 0 < k < N]
    for rp in reversed(cand): # largest first
        for m in range(1, 8): # small multiples
            cand = rp * m
            if cand < N and pow(a, cand, N) == 1:
                return cand
    return None
```

The recovered denominator is a divisor of r

Statement

The machine samples s uniformly in $\{0, \dots, r-1\}$, and the convergent is returned *in lowest terms*. What is recovered is therefore

$$\frac{r}{\gcd(s, r)},$$

a divisor of r . The value r itself is obtained outright exactly when $\gcd(s, r) = 1$, with probability $\phi(r)/r = \Omega(1/\log \log r)$. For $r = 4$ this is $\frac{1}{2}$; for $r = 6$ it is $\frac{1}{3}$.

This is not a failure mode

All other outcomes still yield a divisor, repaired classically by testing small multiples or taking the lcm of two runs. And because step 9 verifies, a wrong r is never used: the algorithm is Las Vegas — it may take a few runs, but it never returns a wrong answer. That is why a high per-run success probability is not required.

Two related questions, answered precisely

Why continued fractions rather than rounding to the nearest small fraction? Continued fractions are the optimal way to do that rounding, and Legendre is the theorem guaranteeing the answer appears among the convergents. Ad-hoc rounding has no uniqueness guarantee, and the convergents are computable in $O(\log^3 q)$ bit operations.

What if r is odd, or $a^{r/2} \equiv -1$? That base yields nothing; draw a new a . For odd composite N with $m \geq 2$ distinct prime factors the good-base probability is at least $1 - 2^{-(m-1)} \geq \frac{1}{2}$.

Cost

The whole classical stage — continued fractions, verification, gcd — is $O(n^3)$ bit operations, negligible beside the quantum circuit and beside the number field sieve.

Exercise 9 — why $t = 2n + O(1)$

6 minutes

Task

Justify the register size from Legendre's theorem. Then describe the post-processing procedure and explain precisely what denominator is recovered, including its probability.

Exercise 9 — why $t = 2n + O(1)$

6 minutes

Task

Justify the register size from Legendre's theorem. Then describe the post-processing procedure and explain precisely what denominator is recovered, including its probability.

Solution

The peak concentrates ℓ on the grid point nearest Qs/r , so $|\ell/Q - s/r| \leq \frac{1}{2Q}$. Legendre: if $|x - p/q| < \frac{1}{2q^2}$ with p/q in lowest terms, then p/q is a convergent of the continued fraction of x . Writing $s/r = s'/r'$ in lowest terms, $r' \leq r < N \leq 2^n$, one needs $\frac{1}{2Q} < \frac{1}{2N^2}$, i.e. $Q \geq N^2$; the standard choice is $N^2 \leq Q < 2N^2$, i.e. $t = 2n + O(1)$.

Procedure: form ℓ/Q ; expand as a continued fraction; list convergents; keep those with denominator $< N$; for each candidate r' test $a^{r'} \bmod N$; reject if $\neq 1$; if only a divisor was found, test small multiples $2r', 3r', \dots$ or combine two runs by lcm; verify before extracting factors.

The recovered denominator: the machine samples s uniformly in $\{0, \dots, r-1\}$ and the convergent comes back in lowest terms, so what is recovered is $r/\gcd(s, r)$, a divisor of r . One gets r outright exactly when $\gcd(s, r) = 1$, with probability $\phi(r)/r$: for $r = 4$, $\frac{1}{2}$; for $r = 6$, $\frac{1}{3}$.

PART 8

Worked example: $N = 21$, $a = 2$

Setup

The instance

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 11 \rightarrow 1 \pmod{21}$, so

$$\mathcal{O} = \{1, 2, 4, 8, 16, 11\}, \quad r = \text{ord}_{21}(2) = 6,$$

and U_2 acts on $\mathcal{O}$ as the 6-cycle S . The eigenphases are

$$0, \frac{1}{6}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{5}{6}.$$

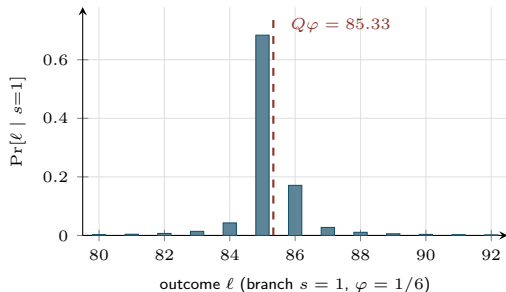
Only $s = 0$ ($\varphi = 0$) and $s = 3$ ($\varphi = \frac{1}{2} = \frac{256}{512}$) are dyadic; the other four are not representable as $j/2^t$, so their peaks must broaden.

Register size

$n = \lceil \log_2 21 \rceil = 5$; the bound $N^2 \leq Q < 2N^2$ reads $441 \leq Q < 882$, so

$$Q = 512, \quad t = 9 \text{ counting qubits.}$$

Since $6 \nmid 512$, four of the six branches must broaden.



Branch $s = 1$, by hand

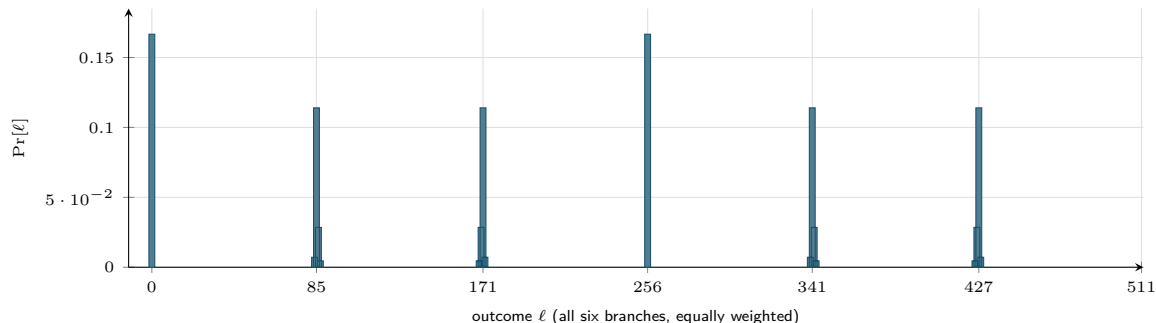
$Q\varphi = 512/6 = 85.33$, not an integer; the nearest grid point is $y = 85$ with

$$\delta_{85} = \frac{1}{6} - \frac{85}{512} = \frac{512-510}{3072} = \frac{1}{1536}, \quad Q\delta = \frac{1}{3}.$$

$$\Pr[85] = \frac{\sin^2(\pi/3)}{512^2 \sin^2(\pi/1536)} \approx \frac{0.75}{1.0966} \approx 0.684.$$

For $y = 86$: $Q\delta = -\frac{2}{3}$ and $\Pr[86] \approx 0.171$. The two adjacent outcomes hold 85.5% of the branch.

The full measured distribution



Reading the peaks

Six peaks at $\ell \approx Qs/6$ for $s = 0, \dots, 5$. The two dyadic branches ($s = 0$ at $\ell = 0$, $s = 3$ at $\ell = 256$) are single spikes of height $\frac{1}{6} = 0.1667$. The four non-dyadic branches spread: height 0.1140 at the nearest grid point plus 0.0285 at the neighbour, with a small tail beyond.

Where r is encoded

The peak *spacing* is Q/r . That, not any single amplitude, is where r is encoded — which is exactly why one measurement gives a rational approximation and needs continued fractions to be promoted to an exact fraction.

Reconstruction, run by run

s	$\varphi = s/6$	nearest ℓ	ℓ/Q	continued fraction $\rightarrow$ convergent with denom. < 21	r'	verdict
0	0	0	0	$[0] \rightarrow 0/1$	1	$2^1 = 2 \neq 1$: retry
1	$1/6$	85	$85/512$	$[0; 6, 42, 2] \rightarrow 1/6$	6	$2^6 = 64 \equiv 1$: success
2	$1/3$	171	$171/512$	$[0; 2, 1, 170] \rightarrow 1/3$	3	$2^3 = 8 \neq 1$: divisor
3	$1/2$	256	$256/512$	$[0; 2] \rightarrow 1/2$	2	$2^2 = 4 \neq 1$: divisor
4	$2/3$	341	$341/512$	$[0; 1, 1, 1, 170] \rightarrow 2/3$	3	divisor
5	$5/6$	427	$427/512$	$[0; 1, 5, 42, 2] \rightarrow 5/6$	6	$2^6 \equiv 1$: success

Success probability, exactly

Branches $s = 1, 5$ have $\gcd(s, 6) = 1$ and recover $r = 6$ directly: $\phi(6)/6 = \frac{1}{3}$ of branches. Summing the exact Fejér weights of *all* ℓ whose convergent has denominator 6 gives

$$\Pr[\text{one run yields } r = 6 \text{ directly}] \approx 0.321.$$

Allowing the classical repair step (testing small multiples of the recovered divisor) raises this to ≈ 0.815 per run.

Finishing the factorisation

$r = 6$ is even and $2^3 = 8 \not\equiv -1 \equiv 20 \pmod{21}$, so

$$\gcd(2^3 - 1, 21) = \gcd(7, 21) = 7,$$

$$\gcd(2^3 + 1, 21) = \gcd(9, 21) = 3,$$

giving $21 = 3 \times 7$.

Branches $s = 2, 3, 4$ give only divisors $(3, 2, 3)$, repaired by testing multiples ($2^{2 \cdot 3} = 2^6 \equiv 1$) or by $\text{lcm}(3, 2) = 6$ from two runs.

Following $\ell = 171$ through the post-processing

Continued fraction of $171/512$

$$512 = 2 \cdot 171 + 170,$$

$$171 = 1 \cdot 170 + 1, \quad \Rightarrow \quad \frac{171}{512} = [0; 2, 1, 170].$$

$$170 = 170 \cdot 1 + 0,$$

$$\text{Convergents: } \frac{0}{1}, \frac{1}{2}, \frac{1}{3}, \frac{171}{512}.$$

The last with denominator $< N = 21$ is $\frac{1}{3}$, so $r' = 3$.

Verification and repair

$2^3 = 8 \not\equiv 1 \pmod{21}$ — fails. So 3 is only a divisor of r : this run came from branch $s = 2$, and $\frac{2}{6}$ reduced to $\frac{1}{3}$.

Test small multiples: $2 \cdot 3 = 6$, and

$2^6 = 64 = 3 \cdot 21 + 1 \equiv 1 \checkmark$, so $r = 6$. Alternatively, combine with a run giving $r' = 2$: $\text{lcm}(3, 2) = 6$.

Why the denominator is never simply trusted

The measured ℓ carries s/r in lowest terms; the reduction is invisible from ℓ alone. The classical verification $a^{r'} \stackrel{?}{\equiv} 1$ is what makes the whole algorithm never return a wrong answer.

Contrast with $N = 15$

For $N = 15$, $a = 2$: $r = 4$ divides $Q = 8$, so every eigenphase $s/4 = 2s/8$ is an exact grid point. Phase estimation is exact — four perfectly sharp peaks at $\ell \in \{0, 2, 4, 6\}$, each of probability $\frac{1}{4}$ — and continued fractions do literally no work.

$(\mathbb{Z}/15\mathbb{Z})^\times \cong \mathbb{Z}/2 \times \mathbb{Z}/4$, so every order is a power of two, no matter which base is chosen. It is exactly the case in which the hardest step disappears. Test an implementation on a non-dyadic period.

The state through the circuit

Stage	Joint state (counting $\otimes$ work)	Basis	Ent.?	Where r lives	Measure now $\rightarrow$
1. initial	$ 0\rangle^{\otimes t} \otimes 1\rangle$	comp.	no	nowhere yet	0; work $ 1\rangle$
2. after $H^{\otimes t}$	$\frac{1}{\sqrt{Q}} \sum_x x\rangle \otimes 1\rangle$	comp.	no	nowhere	uniform x
3. after the $c\text{-}U_a$ ladder	$\frac{1}{\sqrt{Q}} \sum_x x\rangle a^x \bmod N\rangle$	orbit	yes	period of a^x	correlated pair (x, a^x)
4. same state, eigenbasis	$\frac{1}{\sqrt{r}} \sum_s \left(\frac{1}{\sqrt{Q}} \sum_x e^{2\pi i x s / r} x\rangle \right) u_s\rangle$	character	yes	denominators of s/r	same distribution
5. ladder reading	each branch: ramp $e^{2\pi i x s / r}$, uniform $ \cdot $	character	yes	ramp frequency s/r	still uniform
6. after QFT †	$\frac{1}{\sqrt{r}} \sum_s \sum_y A_s(y) y\rangle u_s\rangle$	comp./char.	yes	peaks at $y \approx Qs/r$	$\ell \approx Qs/r$
7. measure counting	$ \ell\rangle \otimes u_s\rangle$ (one branch)	comp./char.	no	in $\ell/Q \approx s/r$	ℓ obtained

Reading the table

Entanglement appears at stage 3, when the oracle correlates the registers, and disappears at stage 7 only because measurement projects onto one branch. r is never in a single amplitude: it is a global property — the period of the sequence, equivalently the common denominator of the eigenphases, equivalently the spacing of the peaks.

Exercise 10 — $N = 21$ end to end

12 minutes

Task

With $a = 2$: find the orbit and r ; choose t and Q with justification; list the six eigenphases and say which are dyadic; compute $Q\varphi$ and Pr for the two most likely outcomes of branch $s = 1$; run continued fractions on $\ell = 171$; and finish the factorisation. State the per-run success probability.

Exercise 10 — $N = 21$ end to end

12 minutes

Task

With $a = 2$: find the orbit and r ; choose t and Q with justification; list the six eigenphases and say which are dyadic; compute $Q\varphi$ and Pr for the two most likely outcomes of branch $s = 1$; run continued fractions on $\ell = 171$; and finish the factorisation. State the per-run success probability.

Solution

Orbit: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 11 \rightarrow 1 \pmod{21}$, so $r = \text{ord}_{21}(2) = 6$, $n = \lceil \log_2 21 \rceil = 5$. **Register:** $N^2 \leq Q < 2N^2$ reads $441 \leq Q < 882$, so $Q = 512$, $t = 9$. **Eigenphases:** $0, \frac{1}{6}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{5}{6}$; only $s = 0$ and $s = 3$ ($\varphi = \frac{1}{2} = \frac{256}{512}$) are dyadic, so the other four peaks broaden.

Branch $s = 1$: $Q\varphi = 512/6 = 85.3$; nearest grid point $y = 85$ with $\delta_{85} = \frac{1}{6} - \frac{85}{512} = \frac{2}{3072} = \frac{1}{1536}$, so $Q\delta = \frac{1}{3}$ and

$\text{Pr}[85] = \frac{\sin^2(\pi/3)}{512^2 \sin^2(\pi/1536)} \approx \frac{0.75}{1.0966} \approx 0.684$. For $y = 86$: $Q\delta = -\frac{2}{3}$, $\text{Pr}[86] \approx 0.171$. Two adjacent outcomes hold 85.5% of the branch.

$\ell = 171$: $512 = 2 \cdot 171 + 170$, $171 = 1 \cdot 170 + 1$, so $171/512 = [0; 2, 1, 170]$ with convergents $\frac{0}{1}, \frac{1}{2}, \frac{1}{3}, \frac{171}{512}$. The last with denominator < 21 is $\frac{1}{3}$, so $r' = 3$; verify $2^3 = 8 \not\equiv 1$ — fails, so 3 is only a divisor (this run came from $s = 2$, and $\frac{2}{6}$ reduced to $\frac{1}{3}$). Test $2 \cdot 3 = 6$: $2^6 = 64 \equiv 1 \checkmark$, so $r = 6$.

Finish: $r = 6$ even, $2^3 = 8 \not\equiv -1 \equiv 20$, so $\gcd(7, 21) = 7$ and $\gcd(9, 21) = 3$: $21 = 3 \times 7$. **Per-run success:** $\phi(6)/6 = \frac{1}{3}$ of branches reconstruct 6 directly; summing the exact Fejér weights gives ≈ 0.321 (and ≈ 0.815 if the classical multiple-testing repair is allowed).

Why not $N = 15$: there every order is a power of two, phase estimation is exact, and continued fractions do no work at all. $N = 21$ is the smallest example that exercises every step.

PART 9

Running it: a complete Qiskit implementation

The four-step implementation workflow

The preceding derivation maps onto a standard four-step pattern for running an algorithm on hardware. This part covers all four steps for $N = 15$, $a = 2$.



Where the deck's parts fit

Step	Established in
1. Map	Parts I–IV: reduction, shift, kickback, ladder
2. Optimise	Part V: gates, QFT, arithmetic, gate count
3. Execute	this part
4. Post-process	Part VI: Legendre, continued fractions

Why $N = 15$ here, having recommended $N = 21$

$N = 15$ needs only 4 target qubits and its multipliers are pure permutations, so it is the largest instance that runs end to end on current hardware with a recognisable signal. It is also, for exactly the same reason, the case in which continued fractions do no work. Both facts are stated below; Part VII treats the non-dyadic case analytically.

Step 1 — the multiplier is a permutation

$M_2 \bmod 15$ on four qubits

$M_a |x\rangle = |ax \bmod N\rangle$ permutes basis states, so it is a permutation matrix. For $N = 15$, $a = 2$ there is a shortcut: $2^4 = 16 \equiv 1 \pmod{15}$, so multiplication by 2 is a *cyclic left rotation* of the 4-bit string,

$$|x_3 x_2 x_1 x_0\rangle \mapsto |x_2 x_1 x_0 x_3\rangle.$$

Check: $1 = 0001 \mapsto 0010 = 2$; $8 = 1000 \mapsto 0001 = 1$ (indeed $16 \equiv 1$); $5 = 0101 \mapsto 1010 = 10$. A rotation by one position on four wires is three SWAPS; $M_4 = M_2^2$ is a rotation by two, i.e. two SWAPS.

```
def M2mod15():                # x -> 2x mod 15
    U = QuantumCircuit(4)
    U.swap(2, 3); U.swap(1, 2); U.swap(0, 1)
    U = U.to_gate(); U.name = "M_2"
    return U

def M4mod15():                # x -> 4x mod 15
    U = QuantumCircuit(4)
    U.swap(1, 3); U.swap(0, 2)
    U = U.to_gate(); U.name = "M_4"
    return U

def controlled_M(gate):
    return gate.control()
```

The exponent constants, computed classically

```
def a2kmodN(a, k, N):
    """a^(2^k) mod N by repeated squaring."""
    for _ in range(k):
        a = (a * a) % N
    return a

k_list = range(8)
b_list = [a2kmodN(2, k, 15) for k in k_list]
# -> [2, 4, 1, 1, 1, 1, 1, 1]
```

Reading that list

$a^{2^k} \bmod 15$ collapses to 1 from $k = 2$ onwards, because $r = \text{ord}_{15}(2) = 4 = 2^2$. So six of the eight controlled multipliers are the *identity* and are simply omitted. This is not an optimisation but the dyadic structure of $N = 15$ appearing in the circuit, and it is why this instance is easy.

For a non-dyadic period the list never collapses, and all t controlled multipliers must be built.

Register sizes

$n = \lfloor \log_2(N - 1) \rfloor + 1 = 4$ target qubits; $t = 2n = 8$ control qubits, so $Q = 256$.

Step 2 — the effect of the synthesis method

The general route: build the permutation matrix

```
def mod_mult_gate(b, N):
    """Modular multiplication from its
       permutation matrix."""
    if gcd(b, N) > 1:
        raise ValueError("b not invertible")
    n = floor(log(N - 1, 2)) + 1
    U = np.zeros((2**n, 2**n), dtype=int)
    for x in range(N):
        U[(b * x) % N][x] = 1
    for x in range(N, 2**n):
        # act as identity
        U[x][x] = 1
        # outside Z_N
    G = UnitaryGate(U); G.name = f"M_{b}"
    return G
```

This is correct and completely general. On this instance it is an order of magnitude more expensive:

$M_2 \bmod 15$	2q-depth	2q-size
from permutation matrix	94	96
hand-synthesised (SWAPs)	9	9

Transpiled to a linear coupling map on 4 qubits.

The general principle

Passing a synthesiser a $2^n \times 2^n$ unitary discards the structure the derivation established: the synthesiser must recover, from a 16×16 matrix of zeros and ones, that the operation is three SWAPs, and it does not.

This is the same principle as Part V's "constant operands are cheaper": the classical constant b is known at compile time, so the circuit should be *generated* from it, never searched for. At cryptographic sizes the matrix route is not merely more expensive but infeasible: $2^{2048} \times 2^{2048}$ cannot be stored.

Transpiled for a real backend

The complete 12-qubit order-finding circuit, transpiled at

optimisation level 2:	2q-depth	187
	2q-size	260
	gate counts	sx: 521, rz: 354,
		cz: 260, x: 4

The native gate set here is $\{sx, rz, cz, x\}$, not Clifford+ T . T -counting is the *fault-tolerant* currency of Part V; on a NISQ device the currency is two-qubit count and depth, per the table in V.A.

The full order-finding circuit

```
N, a = 15, 2
num_target = floor(log(N - 1, 2)) + 1    # 4
num_control = 2 * num_target             # 8

b_list = [a2kmodN(a, k, N) for k in range(num_control)]

control = QuantumRegister(num_control, name="C")
target = QuantumRegister(num_target, name="T")
output = ClassicalRegister(num_control, name="out")
circuit = QuantumCircuit(control, target, output)

# work register := |1>
circuit.x(num_control)

# Hadamard layer + controlled powers
for k, qubit in enumerate(control):
    circuit.h(qubit)
    b = b_list[k]
    if b == 2:
        circuit.compose(M2mod15().control(),
                        qubits=[qubit] + list(target), inplace=True)
    elif b == 4:
        circuit.compose(M4mod15().control(),
                        qubits=[qubit] + list(target), inplace=True)
    else:
        continue          # M_1 is the identity

# inverse QFT on the control register, then measure
circuit.compose(QFT(num_control, inverse=True),
                qubits=control, inplace=True)
circuit.measure(control, output)
```

What each line corresponds to

Code	Derivation
circuit.x(...)	$ 1\rangle = \frac{1}{\sqrt{r}} \sum_s u_s\rangle$ is free
circuit.h(qubit)	$H^{\otimes t}$, the 2^t -way splitter
M...control()	kickback, $e^{2\pi i 2^k \varphi}$ per wire
a2kmodN	$U_a^{2^k} = U_{a^{2^k \bmod N}}$, classical squaring
QFT(inverse=True)	phase $\rightarrow$ probability
measure	Born rule, one sample ℓ

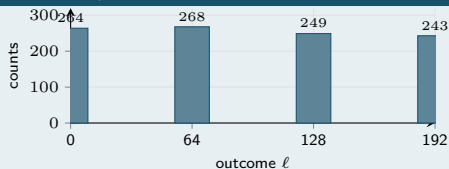
Bit order

This circuit indexes control qubit k with $b_k = a^{2^k}$, i.e. *least significant first*, paired with Qiskit's little-endian QFT. The deck's convention (Part 0) is big-endian, x_1 on the top wire driving $U^{2^{t-1}}$. Both are correct; mixing them is not.

This is the single most common implementation bug, and it fails *silently*: the circuit runs and returns plausible but incorrect output. Check it end to end on a known basis state before trusting any output.

Step 3 — execution, and Step 4 — post-processing

Ideal simulator, 1024 shots



An approximately *uniform* distribution over exactly four outcomes — $\ell \in \{0, 64, 128, 192\} = Q \cdot \frac{s}{4}$ for $Q = 256$, $r = 4$. Four perfectly sharp spikes, each of probability $\frac{1}{4}$, precisely as Part VII predicted for the dyadic case.

Reading the phases

bitstring	ℓ	$\varphi = \ell/2^8$	limit_denominator(15)
00000000	0	0.00	$0/1 \Rightarrow r' = 1$
01000000	64	0.25	$1/4 \Rightarrow r' = 4 \checkmark$
10000000	128	0.50	$1/2 \Rightarrow r' = 2$
11000000	192	0.75	$3/4 \Rightarrow r' = 4 \checkmark$

```
from fractions import Fraction
```

```
# Step 4: phase -> order -> factor
for bitstring in counts_keep:
    decimal = int(bitstring, 2)
    phase = decimal / 2**num_control      # = k / r
    if phase == 0:                        # uninformative
        continue
    frac = Fraction(phase).limit_denominator(N)
    r = frac.denominator
    if r % 2:                             # odd order: retry
        continue
    x = pow(a, r // 2, N)
    if x == N - 1:                         # a^(r/2) = -1: retry
        continue
    d = gcd(x - 1, N)
    if 1 < d < N:
        print(f"factor found: {d}")      # -> 3
        break
```

limit_denominator is the continued-fraction step

Fraction(0.666) returns 5998794703657501/9007199254740992 — the exact binary float. The .limit_denominator(15) call returns 2/3: it is precisely the “keep the last convergent with denominator $< N$ ” rule of Part VI, and Legendre’s theorem is what guarantees the answer is among the convergents.

Outcome

$\varphi = 0$ fails; $\varphi = 0.5$ gives the divisor 2; $\varphi \in \{0.25, 0.75\}$ gives $r = 4$, then $2^2 = 4$ and $\gcd(3, 15) = 3$. Two of four succeed — $\phi(4)/4 = \frac{1}{2}$, as predicted.

Error suppression and mitigation

```
sampler = Sampler(backend)

# dynamical decoupling: timed pulse sequences
# average out environmental interactions
sampler.options.dynamical_decoupling.enable = True
sampler.options.dynamical_decoupling.sequence_type = "XpXm"

# gate twirling: randomise gates so coherent
# errors become Pauli errors, which accumulate
# linearly rather than quadratically
sampler.options.twirling.enable_gates = True

job = sampler.run([transpiled_circuit], shots=1024)
counts = job.result()[0].data["out"].get_counts()

# statistical filter: keep peaks above half the max
threshold = max(counts.values()) / 2
counts_keep = {k: v for k, v in counts.items()
               if v > threshold}
# -> {'00000000': 58, '01000000': 41,
#     '10000000': 40, '11000000': 42}
```

What the hardware run shows

The four correct bitstrings still have the highest counts, but out of 1024 shots only about 181 land on them — the rest leak across the other 252 outcomes. The signal survives; the contrast does not.

That is the observed behaviour of a 12-qubit, 260-two-qubit-gate circuit on a current device, and it is why the threshold filter is needed.

Why this does not extrapolate to cryptographic sizes

The circuit here has 260 two-qubit gates. RSA-2048 needs $\approx 5 \times 10^{12}$ Toffoli gates and, with error correction, on the order of 10^6 physical qubits at circuit depth $\sim 10^9$. The gap is nine to ten orders of magnitude in both dimensions.

Note the role of the classical verification here: because every candidate is checked, noise can only waste runs, never produce a wrong factorisation. The Las Vegas property holds on hardware as well.

What the published demonstrations show

Work	Platform	N	Technique
Vandersypen et al. (2001)	7-qubit NMR	15	first demonstration
Martin-López et al. (2012)	photonic	15	qubit recycling; one control qubit reused, work register in H
Monz et al. (2016)	ion trap	15	Kitaev iterative phase estimation; 7 control + 4 cache qubits
Amico, Saleem, Kumph (2019)	superconducting	15, 21, 35	semiclassical (Kitaev) QFT to minimise qubits and gates
Skosana & Tame (2021)	superconducting	21	compiled QPE, approximate Toffolis with residual phase sh

The question to ask of a claimed record

Was the modular exponentiation circuit compiled using prior knowledge of r ?

Nearly every demonstration above uses a compiled circuit — multipliers simplified using the known order, or the known factorisation. That removes precisely the part that does the work. No cryptographically relevant number has been factored by Shor's algorithm, and none is close.

The demonstrations are valuable as engineering milestones, not as evidence about scaling.

Where the qubit savings in that table come from

Two ideas recur, and both are in Part V:

- **Semiclassical / iterative QFT** (Kitaev; Griffiths–Niu): replace the t control qubits by a single reused ancilla with classical feedback. Exact, not approximate — the controlled rotations can be deferred past the measurement of their controls. Cost: a sequential circuit with mid-circuit measurement and reset.
- **Compiling constants at synthesis time**: every multiplier is by a classical constant, so its gate pattern is generated, not searched for.

The first is legitimate at any scale; the second is legitimate only when the constant is genuinely public. Using knowledge of r is not.

Exercise 14 — build the $N = 21$ circuit

Open-ended; this is the honest instance

Task

Construct the order-finding circuit for $N = 21$ with a base a of your choice. Decide the bit accuracy, hence the number of control qubits; then design the modular multiplication operators $M_{a^{2^k}}$.

Exercise 14 — build the $N = 21$ circuit

Open-ended; this is the honest instance

Task

Construct the order-finding circuit for $N = 21$ with a base a of your choice. Decide the bit accuracy, hence the number of control qubits; then design the modular multiplication operators $M_{a^{2^k}}$.

Expected results, and how to check them

Sizes. $n = \lceil \log_2 21 \rceil = 5$ target qubits. Part VI gives $N^2 \leq Q < 2N^2$, i.e. $441 \leq Q < 882$, so $Q = 512$ and $t = 9$ control qubits: 14 qubits before ancillas.

The constants. With $a = 2$: $b_k = 2^{2^k} \bmod 21$ for $k = 0, \dots, 8$ gives 2, 4, 16, 4, 16, 4, 16, 4, 16 — it becomes periodic but *never collapses* to 1, because $r = 6$ is not a power of two. Every one of the nine controlled multipliers must be built. That is the substantive difference from $N = 15$.

The multipliers. $M_2 \bmod 21$ is a permutation of $\{0, \dots, 20\}$ with cycle structure $(1\ 2\ 4\ 8\ 16\ 11)(3\ 6\ 12)(5\ 10\ 20\ 19\ 17\ 13)(7\ 14)(9\ 18\ 15)(0)$ — *not* a single rotation, so no SWAP-only shortcut exists. Either synthesise each cycle from CNOTs and Toffolis, or build the general modular multiplier of Part V.E.

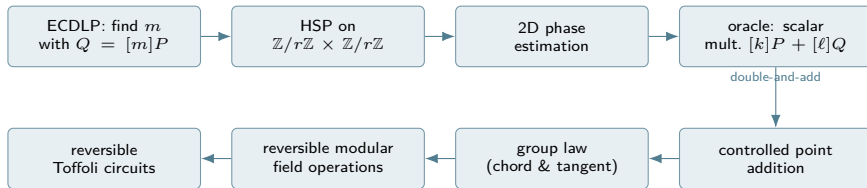
What you should see. Six peaks at $\ell \approx 0, 85, 171, 256, 341, 427$; two sharp (the dyadic branches $s = 0, 3$) and four broadened, with $\Pr[85 \mid s=1] = 0.684$ and $\Pr[86 \mid s=1] = 0.171$. Continued fractions now do real work: $\ell = 171$ returns $r' = 3$, which *fails* verification and must be repaired to $r = 6$. Per-run success ≈ 0.321 directly, ≈ 0.815 allowing the multiple-testing repair.

Reality check. Published $N = 21$ demonstrations use five qubits, not fourteen, by compiling with knowledge of the answer. If your circuit is much smaller than the sizes above, check what you assumed.

PART 10

The elliptic-curve discrete logarithm

The reduction tower



What this part establishes

The ECDLP is solved by the same eigenvalue-to-phase mechanism as factoring, but on a two-dimensional hidden subgroup of $\mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$. From character orthogonality alone, measuring the two index registers returns a pair (y_1, y_2) obeying the linear relation $y_2 \equiv m y_1 \pmod{r}$, from which m follows by one modular inversion. The oracle is then built all the way down to Toffoli gates, and counted.

Notation for this part

Q now denotes the public point $[m]P$, not the counting-register size. Register sizes are 2^{n+1} with $n = \lceil \log_2 p \rceil$.

The discrete logarithm as a hidden subgroup problem

Setup

Let $G = \langle P \rangle$ be a finite cyclic group of *known* order r , written additively. Fix a generator P and an element $Q = [m]P$ for an unknown $m \in \mathbb{Z}/r\mathbb{Z}$. The problem is to recover m .

Definition (hidden subgroup problem)

Given $f : A \rightarrow X$ on a finite abelian group A that is constant on cosets of an unknown subgroup $L \leq A$ and distinct on distinct cosets, find L . We say f *hides* L .

Order finding was the one-dimensional case: $A = \mathbb{Z}$ truncated to $\mathbb{Z}/Q\mathbb{Z}$, $f(x) = a^x \bmod N$, and $L = r\mathbb{Z}$.

Proposition (the hidden subgroup)

Take $A = \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ and $f(k, \ell) = [k]P + [\ell]Q$. Since $Q = [m]P$ and scalar multiplication is a group homomorphism,

$$f(k, \ell) = [k]P + [\ell m]P = [k + m\ell]P,$$

so f depends only on $k + m\ell \bmod r$, and f hides

$$L = \{(k, \ell) : k + m\ell \equiv 0\} = \langle (-m, 1) \rangle,$$

of order r and index r . Knowing L is knowing m .

Proof. $f(k, \ell) = f(k', \ell') \iff (k - k') + m(\ell - \ell') \equiv 0 \iff (k - k', \ell - \ell') \in L$, so f is constant on cosets of L and injective across them. Every element of L is $(-m\ell, \ell) = \ell \cdot (-m, 1)$. $\square$

Geometric reading

The unknown is now the *slope of a line* in $\mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$. The quantum machine will hand over one point of the dual line, which is enough.

The measured relation, derived

Assume for the derivation that the index registers have dimension exactly r ; the finite-register version follows.

Theorem

Prepare the uniform superposition over A , apply f into a point register, apply QFT_r to each index register, and measure. The outcome (y_1, y_2) satisfies $y_2 \equiv m y_1 \pmod{r}$, with each admissible y_1 occurring with probability $1/r$.

Proof

(1) **Superpose and apply the oracle.** $\frac{1}{r} \sum_{k, \ell} |k\rangle |\ell\rangle |\mathcal{O}\rangle \xrightarrow{f} \frac{1}{r} \sum_{k, \ell} |k\rangle |\ell\rangle |[k + m\ell]P\rangle.$

(2) **Measure (or ignore) the point register.** It holds $[c]P$ for a uniformly random coset label c ; the index registers collapse to the uniform superposition over $\{(k, \ell) : k + m\ell \equiv c\}$, parametrised by ℓ (so $k = c - m\ell$): $\frac{1}{\sqrt{r}} \sum_{\ell} |c - m\ell\rangle |\ell\rangle.$

(3) **Apply $\text{QFT}_r \otimes \text{QFT}_r$.**

$$\frac{1}{r^{3/2}} \sum_{y_1, y_2} \omega_r^{c y_1} \underbrace{\left(\sum_{\ell} \omega_r^{\ell(y_2 - m y_1)} \right)}_{= r [y_2 \equiv m y_1] \text{ (Lemma 1)}} |y_1\rangle |y_2\rangle = \frac{1}{\sqrt{r}} \sum_{y_1} \omega_r^{c y_1} |y_1\rangle |m y_1 \bmod r\rangle.$$

(4) **Measure.** The outcome is $(y_1, m y_1 \bmod r)$, uniform over the r values of y_1 . □

The leftover phase

$\omega_r^{c y_1}$ is *not* a global phase — it depends on y_1 . It is a phase attached to each basis state of the register that is measured next, and a computational-basis measurement is insensitive to phases on its own basis states, so $\Pr[(y_1, y_2)] = 1/r$ regardless of c . That is why the unknown coset label does no harm, and why the point register need not be measured at all.

Why two QFTs, and how the two problems compare

The dual group factors

The translations $(k, \ell) \mapsto (k + \alpha, \ell + \beta)$ of $A = \mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$ commute, so they are *simultaneously* diagonalised by the characters of A ,

$$\chi_{y_1, y_2}(k, \ell) = \omega_r^{ky_1 + \ell y_2}.$$

$\text{QFT}_r \otimes \text{QFT}_r$ is precisely the change to that eigenbasis, and it factors over the two coordinates because $\widehat{A} \cong \widehat{\mathbb{Z}/r\mathbb{Z}} \times \widehat{\mathbb{Z}/r\mathbb{Z}}$.

What measurement gives

It samples a character trivial on the hidden subgroup:

$$\chi_{y_1, y_2} \Big|_L \equiv 1 \iff \omega_r^{\ell(y_2 - my_1)} = 1 \ \forall \ell \iff y_2 \equiv my_1.$$

So the measured pair names a point of $L^\perp$, the annihilator of L . Since L is a line, one generic point of $L^\perp$ determines it, hence determines m .

	factoring	DLP / ECDLP
A	$\mathbb{Z}/Q\mathbb{Z}$	$\mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$
L	$r'\mathbb{Z}/Q\mathbb{Z}$	$\langle(-m, 1)\rangle$
rank A	1	2
QFTs	1	2
unknown	the <i>size</i> of L	the <i>slope</i> of L
output	$\ell \approx Qs/r$	(y_1, y_2) , $y_2 \equiv my_1$
post-proc.	continued fractions	one modular inverse
success/run	$\phi(r)/r$	$(r-1)/r$

Recovering m

If $\gcd(y_1, r) = 1$ then $m \equiv y_2 y_1^{-1} \pmod{r}$, by one extended-Euclid inversion. For prime r (the NIST curves, cofactor 1) every $y_1 \neq 0$ is invertible, so a *single* run succeeds with probability $(r-1)/r \approx 1$. Only $y_1 = 0$ is uninformative, and it is uninformative for every m , because $\chi_{0,0} \equiv 1$ is trivial on every subgroup.

The finite-register version, and the algorithm

Why $n + 1$ qubits per index register

By Hasse's theorem $\left| \#E(\mathbb{F}_p) - (p + 1) \right| \leq 2\sqrt{p}$, so

$$r \leq \#E(\mathbb{F}_p) \leq p + 1 + 2\sqrt{p} < 2^{n+1}, \quad n = \lceil \log_2 p \rceil.$$

So $n + 1$ qubits hold any scalar or coordinate.

What changes in the analysis

In practice r is not a power of two, so each index register uses $n + 1$ qubits and one applies $\text{QFT}_{2^{n+1}}$. The exact delta function broadens into a Fejér peak — the same kernel as Part IV — and the measured pair satisfies

$$\frac{y_1}{2^{n+1}} \approx \frac{j_1}{r}, \quad \frac{y_2}{2^{n+1}} \approx \frac{j_2}{r}, \quad j_2 \equiv m j_1 \pmod{r}.$$

Rounding $(y_1/2^{n+1}, y_2/2^{n+1})$ to the nearest rationals *with the known denominator* r — a two-dimensional rounding, not a search over unknown denominators — recovers (j_1, j_2) and hence $m \equiv j_2 j_1^{-1}$.

Algorithm (Shor for the ECDLP)

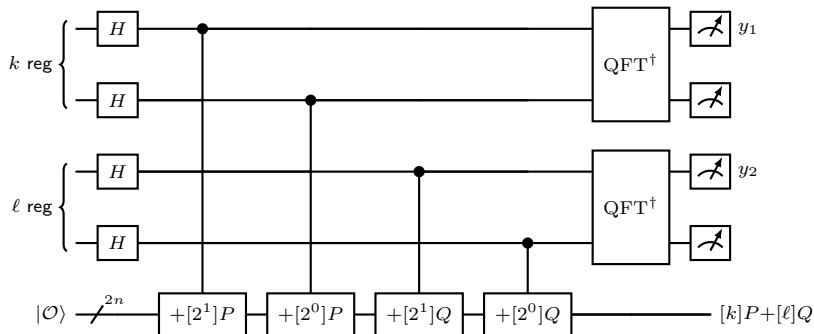
Input: $E(\mathbb{F}_p)$, a point P of order r , and $Q = [m]P$. Set $n = \lceil \log_2 p \rceil$.

1. Prepare two index registers $|0\rangle^{n+1}$ and a point register in $|\mathcal{O}\rangle$.
2. Apply $H^{\otimes(n+1)}$ to each index register.
3. *Double scalar multiplication (the oracle)*: controlled on bit k_j , add the classical point $[2^j]P$; controlled on bit ℓ_j , add $[2^j]Q$. This maps the point register to $[k]P + [\ell]Q$.
4. Apply $\text{QFT}_{2^{n+1}}^\dagger$ to each index register.
5. Measure both index registers, obtaining (y_1, y_2) .
6. Classically: round to (j_1, j_2) with $j_2 \equiv m j_1 \pmod{r}$ and output $m \equiv j_2 j_1^{-1}$.

Post-processing warning

The relation obeyed by the *post-transform* outcomes is $y_2 \equiv m y_1$, not the pre-transform relation $k + m\ell \equiv c$. Applying the pre-transform relation to measured data — e.g. $\hat{m} = (c - k)\ell^{-1}$ — gives a recovery rate at the $1/r$ random baseline even on a perfect simulator. This is a real and easily made implementation error; always verify $[\hat{m}]P = Q$.

The full circuit



Proposition (double scalar multiplication)

For a classically known point R and an $(n+1)$ -qubit scalar register $|s\rangle$,

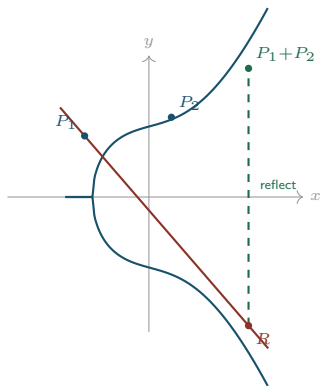
$$\prod_{j=0}^n \left(\text{controlled-} \left(+[2^j]R \right) \text{ on qubit } s_j \right) : |s\rangle |O\rangle \mapsto |s\rangle |[s]R\rangle,$$

since $[s]R = \sum_j s_j [2^j]R$ and the constants $[2^j]R = [2]([2^{j-1}]R)$ are computed by j classical doublings. Running this with $R = P$ on the k -register and $R = Q$ on the ℓ -register realises step 3.

Why every addition is by a constant point

The points $[2^j]P$ and $[2^j]Q$ are precomputed classically, so every box is “add a fixed, known point to a quantum point”, never a quantum–quantum point addition. The constant’s coordinates are folded into the gate pattern *at compile time* (a constant control deletes or fixes a Toffoli, roughly halving the gates); unlike RSA, no quantum–quantum multiplication ever appears; and additions commute, so their order is free. The one irreducible cost inside each box is the modular inversion for λ .

Deriving the group law



The chord meets E in a third point R ; reflecting R in the x -axis gives $P_1 + P_2$.

Definition

For $p > 3$ prime, an elliptic curve in short Weierstrass form is $E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$, with $\Delta = -16(4a^3 + 27b^2) \neq 0$. Its points are $E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p^2 : y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$, a finite abelian group with identity $\mathcal{O}$ and inverse $-(x, y) = (x, -y)$.

Theorem (affine addition formulas)

For $P_1 = (x_1, y_1)$, $P_2 = (x_2, y_2)$ with $P_1 \neq \pm P_2$ set $\lambda = \frac{y_1 - y_2}{x_1 - x_2}$; for

doubling ($P_1 = P_2$, $y_1 \neq 0$) set $\lambda = \frac{3x_1^2 + a}{2y_1}$. Then

$P_3 = P_1 + P_2 = (x_3, y_3)$ with

$$x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1.$$

Proof. The line through P_1, P_2 is $y = \lambda x + \nu$ with $\nu = y_1 - \lambda x_1$; substituting into $y^2 = x^3 + ax + b$ gives

$$x^3 - \lambda^2 x^2 + (a - 2\lambda\nu)x + (b - \nu^2) = 0,$$

a cubic whose roots are the three intersection abscissae. By Viète their sum equals the negated x^2 -coefficient, $x_1 + x_2 + x_3 = \lambda^2$, giving x_3 . The third intersection is $R = (x_3, \lambda x_3 + \nu)$; reflecting in the x -axis gives $y_3 = -(\lambda x_3 + \nu) = \lambda(x_1 - x_3) - y_1$. For doubling, $2yy' = 3x^2 + a$ gives the tangent slope and the same argument applies with a double root. $\square$

The operation that dominates the cost

λ is a quotient in $\mathbb{F}_p$

Both slopes require a modular inversion: $(x_1 - x_2)^{-1}$ or $(2y_1)^{-1}$. Modular inversion is by far the most expensive field operation — worth $\Theta(n)$ multiplications — and it appears in every point addition.

Reversible cost of each field operation

operation	reversible cost
modular add / sub	$O(n \log n)$ Toffoli
modular double	$O(n \log n)$ Toffoli
modular multiply / square	$O(n^2 \log n)$ Toffoli
modular inverse	$O(n^2 \log n)$ Toffoli, $O(n)$ ancilla

The $\log n$ factors come from the constant-incrementer construction; exact leading terms are tabulated later in this part. Textbook treatments quote $\Theta(n)$ / $\Theta(n^2)$, which is the count without that construction.

Consequence, stated early

This is why a 256-bit ECDLP is a comparable quantum target to a 2048–3072-bit RSA modulus *despite* the far smaller key: an elliptic point addition is individually much dearer than a modular multiplication. The smaller n still wins — but by less than the key sizes alone would suggest.

Why affine coordinates, despite the inversion

Projective coordinates avoid almost all inversions classically, by multiplying through with denominators. Two obstacles make that hard here: each point is an equivalence class of coordinate vectors, and the extra temporary variables must all be uncomputed. Since the binding constraint in the published estimates is *qubits*, and the inversion circuit already supplies the ancillas that the multiplier needs, affine coordinates win. Whether projective coordinates help remains open.

The running curve: $E : y^2 = x^3 + 3x + 2$ over $\mathbb{F}_5$

Finding the points

Squares in $\mathbb{F}_5$ are $\{0, 1, 4\}$. Test each x :

- ▶ $x = 0$: 2 — not a square;
- ▶ $x = 1$: $1 + 3 + 2 = 6 \equiv 1$ — square, $y = \pm 1$;
- ▶ $x = 2$: $8 + 6 + 2 = 16 \equiv 1$ — square, $y \in \{1, 4\}$;
- ▶ $x = 3$: $27 + 9 + 2 = 38 \equiv 3$ — not a square;
- ▶ $x = 4$: $64 + 12 + 2 = 78 \equiv 3$ — not a square.

So $E(\mathbb{F}_5) = \{\mathcal{O}, (1, 1), (1, 4), (2, 1), (2, 4)\}$ and $\#E = 5$, prime, so every non-identity point generates. (Discriminant check: $-16(4 \cdot 27 + 27 \cdot 4) = -16 \cdot 216 \equiv 4 \neq 0$.)

The cyclic subgroup

Take $P = (2, 1)$. Doubling ($a = 3$): $\lambda = \frac{3 \cdot 4 + 3}{2 \cdot 1} = \frac{15}{2} \equiv \frac{0}{2} = 0$, so $x_3 = 0 - 2 - 2 = -4 \equiv 1$ and $y_3 = 0 \cdot (2 - 1) - 1 = -1 \equiv 4$: $[2]P = (1, 4)$. Continuing, $[1]P = (2, 1)$, $[2]P = (1, 4)$, $[3]P = (1, 1)$, $[4]P = (2, 4)$, $[5]P =$

The instance to be solved

$P = (2, 1)$, $Q = (1, 1)$. Then $Q = [3]P$, so the answer is $m = 3$ — which the quantum algorithm will produce *without* knowing it in advance. Inverses mod 5: $1^{-1}=1$, $2^{-1}=3$, $3^{-1}=2$, $4^{-1}=4$.

A caveat on small demonstrations

Encoding points by their group index requires enumerating $\langle P \rangle$, which is $O(r)$ classical work — and the index of Q is m . A demonstration built that way has already solved the ECDLP classically before a single qubit is allocated. The path to real sizes is the coordinate oracle built below, where the circuit implements the group law on actual field elements.

Reversible modular arithmetic: the plan



Conventions

All registers hold n -bit integers with $p < 2^n$; operations are in place unless stated. Every constant (p , $2^i a \bmod p$, the coordinates of $[2^j]P$) is folded in at compile time, which roughly halves the gate count.

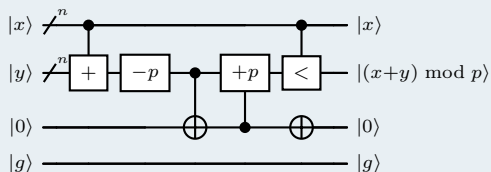
Modular multiplication, squaring and inversion operate *out of place* and xor the result into a separate output register — which is what allows them to be run a second time to clear that register.

Why every block must be garbage-free

A point addition is a fixed sequence of field operations, each of which must be a reversible circuit with all ancillas returned to $|0\rangle$. Leftover scratch would record which-path information and, by rule (R3), destroy the interference that produces $y_2 \equiv my_1$. Concretely this is why the point addition below ends by recomputing λ from its own output and running the inversion backwards.

Modular addition and doubling

Modular addition, $|x\rangle |y\rangle \mapsto |x\rangle |(x+y) \bmod p\rangle$



Add; subtract the constant p unconditionally; copy the resulting sign bit to the reduction flag with a CNOT; add p back conditioned on that flag; finally *uncompute the flag by a comparison of the result with x* , since a reduction happened if and only if $(x+y) \bmod p < x$. Two auxiliary qubits: one flag, and one dirty ancilla $|g\rangle$ used by the constant adders and returned unchanged.

Modular subtraction is the same circuit reversed. To control it, control every operation.

Modular doubling, p odd: $|x\rangle \mapsto |2x \bmod p\rangle$

Two changes make it cheaper than the adder. It works in place on one n -qubit register, so it needs only $n+2$ qubits; and the first integer addition is replaced by a multiplication by 2, implemented as a cyclic bit shift (a chain of SWAPS built from CNOTs).

Because p is odd, the reduction flag is recoverable from the least significant bit of the result: $2x - p$ is odd while $2x$ is even. So a subtraction of the modulus has taken place if and only if that bit is 1, and no persistent ancilla is needed.

The recurring pattern

Every block ends by recomputing its own flag from its output. That, and nothing else, is what makes the whole tower garbage-free.

Modular multiplication, squaring, and inversion

Multiplication by doubling and adding

By Horner's rule,

$$x \cdot y = \sum_{i=0}^{n-1} x_i 2^i y = \left(\cdots \left((x_{n-1} y) \cdot 2 + x_{n-2} y \right) \cdot 2 + \cdots \right) + x_0 y,$$

so with an accumulator z starting at 0, looping $i = n - 1, \dots, 0$ with $z \leftarrow 2z \bmod p$ (modular doubling), $z \leftarrow (z + y) \bmod p$ controlled computes $|x\rangle |y\rangle |0\rangle \mapsto |x\rangle |y\rangle |xy \bmod p\rangle$ in n modular doublings and n controlled modular additions.

Squaring saves the y -register by copying bit x_i into one ancilla each round.

Montgomery multiplication replaces the conditional subtraction by a truncation plus one addition, roughly halving the Toffoli count at the cost of more ancillas — which are free here, because the inversion circuit already supplies them.

Modular inversion: Kaliski's binary extended GCD

Maintain (u, v, r, s) with the loop invariant $p = rv + su$. Start $u = p, v = x, r = 0, s = 1, k = 0$, and iterate

$$\begin{cases} u \text{ even} : & u \leftarrow u/2, s \leftarrow 2s; \\ v \text{ even} : & v \leftarrow v/2, r \leftarrow 2r; \\ u > v : & u \leftarrow (u - v)/2, r \leftarrow r + s, s \leftarrow 2s; \\ \text{else} : & v \leftarrow (v - u)/2, s \leftarrow s + r, r \leftarrow 2r; \end{cases}$$

until $v = 0$; then $r \equiv x^{-1} 2^k \bmod p$ (the “almost inverse”), and $n - k$ controlled modular doublings plus a sign flip convert it to $x^{-1} 2^n$, the Montgomery form.

Obstacle: the loop length depends on the input

k lies between n and $2n$ and depends on x , so the naive loop is irreversible — different inputs would halt at different times.

Making a data-dependent loop reversible

Two devices

1. **Run a fixed $2n$ rounds.** A flag f , initially 1, flips to 0 once $v = 0$ is reached, after which each round merely increments a counter k instead of touching (u, v, r, s) . The circuit shape is therefore independent of x . Since $n \leq k \leq 2n$, the counter needs only $\lceil \log_2 n \rceil + 1$ qubits.
2. **Record the branch.** Each round stores which of the four branches was taken in *one* qubit m_i , so the whole computation can be uncomputed after the result is copied out.

Why one qubit per round suffices

Naively the four branches need two bits, which would cost $4n$ ancillas rather than $2n$. But in each step of Kaliski's algorithm exactly one of r and s is even and the other odd — an inductive consequence of the invariant $p = rv + su$. If the updated r is even the branch was “ v even” or “both odd, $u \leq v$ ”; if the updated s is even it was “ u even” or “both odd, $u > v$ ”. So the second encoding bit is recomputable from the register contents and can be uncomputed immediately by a single CNOT from the least significant bit of r .

Cost, and where the qubits go

Those $2n$ round-record qubits dominate the qubit count of the whole algorithm. The inversion needs

$$7n + 2\lceil \log_2 n \rceil + 9 \text{ qubits, } 32n^2 \log_2 n \text{ TOF,}$$

which is $\Theta(n)$ multiplications' worth of work, and it is why the point addition is expensive.

Round-function structure

Each round: a CNOT testing whether u is even; another testing v ; a subtractor whose most significant qubit decides $u > v$; a five-gate encoder producing the two branch bits ab ('10' for u even, '01' for u odd and v even, '11' for both odd with $u > v$, '00' for both odd with $u \leq v$), where $b = m_i$; a six-gate dispatcher; and a final CNOT from the low bit of r that uncomputes a .

After $2n$ rounds the result is copied out and the entire circuit is run backwards.

Reversible controlled point addition

$P_2 = (x_2, y_2) = [2^j]P$ is a compile-time constant, so every operation involving it is a constant operation

Algorithm: controlled addition of a constant point

Registers hold the quantum point (x_1, y_1) , the control qubit, and auxiliaries λ, t_0 .

	operation	effect
1	sub_const $x_1 - x_2$	$x_1 \leftarrow x_1 - x_2$
2	ctrl_sub_const $y_1 - y_2$	$y_1 \leftarrow [y_1 - y_2]_1$
3	inv $x_1 \rightarrow t_0$	$t_0 \leftarrow 1/(x_1 - x_2)$
4	mul $y_1 t_0 \rightarrow \lambda$	$\lambda \leftarrow [\frac{y_1 - y_2}{x_1 - x_2}]_1$
5	mul $\lambda x_1 \rightarrow y_1$	$y_1 \leftarrow 0$
6	inv $x_1 \rightarrow t_0$	$t_0 \leftarrow 0$
7	squ $\lambda \rightarrow t_0$	$t_0 \leftarrow \lambda^2$
8	ctrl_sub $x_1 - t_0$	$x_1 \leftarrow [x_1 - x_2 - \lambda^2]_1$
9	ctrl_add_const $3x_2$	$x_1 \leftarrow [x_2 - x_3]_1$
10	squ $\lambda \rightarrow t_0$	$t_0 \leftarrow 0$
11	mul $\lambda x_1 \rightarrow y_1$	$y_1 \leftarrow [y_3 + y_2]_1$
12	inv $x_1 \rightarrow t_0$	$t_0 \leftarrow [1/(x_2 - x_3)]_1$
13	mul $t_0 y_1 \rightarrow \lambda$	$\lambda \leftarrow 0$
14	inv $x_1 \rightarrow t_0$	$t_0 \leftarrow 0$
15	ctrl_neg x_1	$x_1 \leftarrow [x_3 - x_2]_1$
16	ctrl_sub_const $y_1 - y_2$	$y_1 \leftarrow [y_3]_1$
17	add_const $x_1 + x_2$	$x_1 \leftarrow [x_3]_1$

$[\cdot]_1$ shows the value when ctrl = 1; when ctrl = 0 every marked step is skipped and the register is returned unchanged.

Checking two steps

Step 9. After step 8 the register holds $x_1 - x_2 - \lambda^2$. Since $x_3 = \lambda^2 - x_1 - x_2$ we have $x_1 - x_2 - \lambda^2 = -x_3 - 2x_2$; adding the constant $3x_2$ gives $x_2 - x_3$.

Step 11. y_1 is 0 after step 5, so it receives $\lambda \cdot (x_2 - x_3)$. By the identity below this equals $y_3 + y_2$, which step 16 turns into y_3 .

Why λ can be uncomputed

The slope is recomputable from the *output* and the constant:

$$\lambda = \frac{y_1 - y_2}{x_1 - x_2} = -\frac{y_3 + y_2}{x_3 - x_2}.$$

Reason: the three collinear points on the chord are P_1, P_2 and $R = (x_3, -y_3)$, so the slope through P_2 and R is $\frac{-y_3 - y_2}{x_3 - x_2}$, and it equals λ .

So the same inversion-and-multiply that computed λ (steps 3–4) recomputes it from the output (steps 12–13) and clears the ancilla. This is Bennett compute-copy-uncompute specialised to the group law, and it is *mandatory*: by rule (R3) leftover garbage would tag the branch and destroy the interference that produces $y_2 \equiv m y_1$.

The three cases the affine formulas do not cover

- ▶ **Doubling** ($P_1 = P_2$) needs the tangent slope $\lambda = (3x_1^2 + a)/(2y_1)$. It does not arise inside a single controlled addition, because the added point is a *constant* generically different from the quantum point — the doublings all happen classically, before the circuit is compiled.
- ▶ **The point at infinity** is carried by a flag qubit (a distinguished bit pattern), with the addition conditioned to pass the other summand through when either point is $\mathcal{O}$.
- ▶ **Collisions** $x_1 = x_2$ (so $\Delta x = 0$ and no inverse exists) occur on a measure-zero fraction of inputs; the published analysis argues they can be avoided by the choice of curve and base point and do not affect the resource count. Complete Edwards or projective formulas remove the exceptions entirely, at extra cost.

Where approximation is legitimate, and where it is not

The *correctness* of the measured relation $y_2 \equiv my_1$ requires the oracle to compute $[k]P + [\ell]Q$ on *every* branch of the superposition. A branch on which the affine formulas fail would contribute a wrong or garbage-tagged term. The literature's position is that such branches carry negligible amplitude for well-chosen parameters — and this is the least rigorous part of the published analysis.

Garbage-freeness is *not* in that category. The interference genuinely depends on it, it must be exactly right, and the λ -uncomputation on the previous slide is how it is achieved.

Worked example over $\mathbb{F}_5$, end to end

$E : y^2 = x^3 + 3x + 2$ over $\mathbb{F}_5$, $P = (2, 1)$ of order $r = 5$, $Q = (1, 1)$, true answer $m = 3$. Using the exact form (index registers of dimension $r = 5$, QFT₅), which shows the mechanism cleanly.

Oracle values and hidden subgroup

$[0]P = \mathcal{O}$, $[1]P = (2, 1)$, $[2]P = (1, 4)$, $[3]P = (1, 1) = Q$, $[4]P = (2, 4)$, so

$$f(k, \ell) = [k]P + [\ell]Q = [k + 3\ell \bmod 5]P.$$

For instance

$$f(2, 4) = [2 + 12]P = [14 \bmod 5]P = [4]P = (2, 4) \checkmark.$$

$$L = \langle (-3, 1) \rangle = \langle (2, 1) \rangle \leq \mathbb{Z}/5\mathbb{Z} \times \mathbb{Z}/5\mathbb{Z},$$

so f is constant exactly when $k + 3\ell$ is constant mod 5.

State evolution

$$\begin{aligned} & \frac{1}{5} \sum_{k, \ell} |k\rangle |\ell\rangle |\mathcal{O}\rangle \xrightarrow{f} \frac{1}{5} \sum_{k, \ell} |k\rangle |\ell\rangle |[k + 3\ell]P\rangle \\ & \xrightarrow{\text{meas. pt}} \frac{1}{\sqrt{5}} \sum_{\ell} |c - 3\ell\rangle |\ell\rangle \xrightarrow{\text{QFT}_5^{\otimes 2}} \frac{1}{\sqrt{5}} \sum_{y_1} \omega_5^{c y_1} |y_1\rangle |3y_1\rangle \end{aligned}$$

outcome (y_1, y_2)	check $y_2 \equiv 3y_1$	$m = y_2 y_1^{-1} \bmod 5$	res
(1, 3)	$3 \equiv 3$	$3 \cdot 1 = 3$	m
(2, 1)	$1 \equiv 6$	$1 \cdot 3 = 3$	m
(3, 4)	$4 \equiv 9$	$4 \cdot 2 = 8 \equiv 3$	m
(4, 2)	$2 \equiv 12$	$2 \cdot 4 = 8 \equiv 3$	m
(0, 0)	—	(undefined)	ret

Reading the table

Since $r = 5$ is prime, every $y_1 \neq 0$ is invertible, so a single run returns $m = 3$ unless $y_1 = 0$ (probability $1/5$). Verification: $[3]P = (1, 1) = Q \checkmark$.

Contrast with factoring

No continued fractions, no divisor ambiguity, no lcm of several runs. For prime-order curves the post-processing is one modular inversion and the success probability is $(r - 1)/r \approx 1$. The quantum part, however, is much more expensive per run.

Resources: the concrete numbers

Per modular-arithmetic circuit, n -bit prime field

circuit	qubits	ancillas	Toffoli gates
add_const_modp	$2n$	n	$16n \log_2 n$
ctrl_add_const_modp	$2n+1$	n	$16n \log_2 n$
ctrl_sub_modp	$2n+4$	3	$16n \log_2 n$
ctrl_neg_modp	$n+3$	2	$8n \log_2 n$
mul_modp (dbl/add)	$3n+2$	2	$32n^2 \log_2 n$
mul_modp (Montgomery)	$5n+4$	$2n+4$	$16n^2 \log_2 n$
squ_modp (dbl/add)	$2n+3$	3	$32n^2 \log_2 n$
squ_modp (Montgomery)	$4n+5$	$2n+5$	$16n^2 \log_2 n$
inv_modp	$7n+2\lceil\log_2 n\rceil+9$	$5n+2\lceil\log_2 n\rceil+9$	$32n^2 \log_2 n$

Simulated counts (Roetteler–Naehrig–Svore–Lauter)

$\lceil\log_2 p\rceil$	qubits	Toffoli	Toffoli depth
110	1 014	9.44×10^9	8.66×10^9
160	1 466	2.97×10^{10}	2.73×10^{10}
192	1 754	5.30×10^{10}	4.86×10^{10}
224	2 042	8.43×10^{10}	7.73×10^{10}
256	2 330	1.26×10^{11}	1.16×10^{11}
384	3 484	4.52×10^{11}	4.15×10^{11}
521	4 719	1.14×10^{12}	1.05×10^{12}

Counts are from a full circuit simulation, not pen-and-paper estimates. The Toffoli *depth* is slightly below the total count, so there is some parallelism to exploit. CNOT and NOT gates are Clifford and do not contribute to depth here.

Per controlled point addition, and for the whole run

The maximum qubit usage occurs during an inversion, so

$$9n + 2\lceil\log_2 n\rceil + 10 \text{ logical qubits}$$

— the $7n + 2\lceil\log_2 n\rceil + 9$ of the inversion, plus one control qubit, plus $2n$ for the two registers holding intermediate results. Algorithm 1 contains 4 inversions, 2 squarings and 4 multiplications, so the leading coefficient is $4 \cdot 32 + 2 \cdot 16 + 4 \cdot 16 = 224$:

$$\lesssim 224 n^2 \log_2 n + 2045 n^2 \text{ TOF per addition,}$$

$(448 \log_2 n + 4090) n^3 \text{ TOF for the whole ECDLP run } (2n \text{ additions, sequential}).$

Where the cost is, precisely

The inversion supplies the $2n$ round-record ancillas that dominate the *qubit* count; the modular multiplications inside the point additions dominate the *Toffoli* count. Not any Fourier transform: the two QFTs are $\Theta(n^2)$ exact, or $O(n \log n)$ approximate — negligible.

	ECDLP, P-256	RSA-3072	ratio
classical security	≈ 128 bit	≈ 128 bit	match
field / modulus size n	256	3 072	$12\times$
logical qubits	2 330	6 146	$0.38\times$
Toffoli gates	1.26×10^{11}	1.86×10^{13}	$0.007\times$

The argument in one paragraph

At comparable classical security a 256-bit elliptic curve matches a 3072-bit RSA modulus, but *both* quantum attacks scale as $\Theta(n^3)$ in the operand size n . The $12\times$ smaller n therefore makes the ECDLP dramatically cheaper — in qubits (2330 versus 6146) and by two orders of magnitude in Toffoli gates — despite each elliptic point addition being individually far harder than a modular multiplication, because of the inversion.

In short: the smaller operand size that makes ECC attractive classically is what makes it the cheaper quantum target.

Matched pairs across the range					
ECC bits		qubits	Toffoli	RSA bits	
					qubits
110		1 014	9.4×10^9	512	1 026
160		1 466	3.0×10^{10}	1 024	2 050
224		2 042	8.4×10^{10}	2 048	4 098
256		2 330	1.3×10^{11}	3 072	6 146
384		3 484	4.5×10^{11}	7 680	15 362
521		4 719	1.1×10^{12}	15 360	30 722

Caveats on the comparison

These are logical-resource estimates for a uniformly good hardware model; physical overheads (error correction, magic-state distillation) multiply both columns similarly. The Ekerå–Håstad argument divides the RSA Toffoli column by 4 but does not apply to the ECC column. And a hybrid quantum-classical number field sieve using $n^{2/3+o(1)}$ qubits would shift the security alignment in a way that complicates the analysis further.

Is the finite-field discrete logarithm the same algorithm?

Yes — identical structure, different oracle

The derivation of $y_2 \equiv my_1$ used only that the group is cyclic of *known* order with an efficiently computable operation. Nothing about elliptic curves entered.

setting	oracle $f(k, \ell)$	cost of the oracle
finite-field DLP ($\mathbb{F}_p^\times$, $r = p - 1$)	$g^k h^\ell \bmod p$	modular exponentiation
ECDLP ($Q = [m]P$)	$[k]P + [\ell]Q$	elliptic scalar multiplication

Practical reading

RSA, finite-field Diffie–Hellman, DSA, ECDH and ECDSA are all broken by the same machine. That is one mechanism, four deployed cryptosystems, and it is the reason the migration to post-quantum algorithms is a single migration rather than several.

What the oracle costs, in each case

For factoring, the oracle is t controlled modular multiplications by classical constants: $\Theta(n^3)$ Toffoli with quantum–classical multiplication only.

For the ECDLP, the oracle is $2n$ controlled point additions, each containing a modular inversion: also $\Theta(n^3)$, but with a much larger constant per operand bit — offset by the $12\times$ smaller operand.

Exercise 11 — the ECDLP measurement relation

10 minutes

Task

In four steps, derive $y_2 \equiv my_1 \pmod{r}$ from the hiding function. State the hidden subgroup and prove it is $\langle(-m, 1)\rangle$. Say why the leftover phase is unobservable, and give the success probability for prime r .

Exercise 11 — the ECDLP measurement relation

10 minutes

Task

In four steps, derive $y_2 \equiv my_1 \pmod{r}$ from the hiding function. State the hidden subgroup and prove it is $\langle(-m, 1)\rangle$. Say why the leftover phase is unobservable, and give the success probability for prime r .

Solution

Hidden subgroup. $f(k, \ell) = [k]P + [\ell]Q = [k + m\ell]P$, so $f(k, \ell) = f(k', \ell') \iff (k - k') + m(\ell - \ell') \equiv 0 \iff (k - k', \ell - \ell') \in L$ where $L = \{k + m\ell \equiv 0\}$. Every element of L is $(-m\ell, \ell) = \ell \cdot (-m, 1)$, so $L = \langle(-m, 1)\rangle$, of order r and index r . Knowing L is knowing m .

(1) Superpose and apply the oracle: $\frac{1}{r} \sum_{k, \ell} |k\rangle |\ell\rangle |\mathcal{O}\rangle \rightarrow \frac{1}{r} \sum_{k, \ell} |k\rangle |\ell\rangle |[k + m\ell]P\rangle$. **(2)** Measure (or ignore) the point register: it holds $[c]P$ for a uniformly random coset label c ; the index registers collapse to $\frac{1}{\sqrt{r}} \sum_{\ell} |c - m\ell\rangle |\ell\rangle$. **(3)** Apply $\text{QFT}_r \otimes \text{QFT}_r$: the sum over the free coset parameter ℓ is $\sum_{\ell} \omega_r^{\ell(y_2 - my_1)} = r [y_2 \equiv my_1]$ by Lemma 1, leaving $\frac{1}{\sqrt{r}} \sum_{y_1} \omega_r^{cy_1} |y_1\rangle |my_1 \bmod r\rangle$. **(4)** Measure: the outcome is $(y_1, my_1 \bmod r)$, uniform over the r values of y_1 .

The phase. $\omega_r^{cy_1}$ is not a global phase — it depends on y_1 — but it is a phase on each basis state of the register measured next, and a computational-basis measurement cannot see such phases. So $\Pr[(y_1, y_2)] = 1/r$ regardless of the unknown coset label c .

Recovery. If $\gcd(y_1, r) = 1$ then $m \equiv y_2 y_1^{-1}$ by one extended-Euclid inversion. For prime r every $y_1 \neq 0$ is invertible, so one run succeeds with probability $(r - 1)/r \approx 1$; only $y_1 = 0$ (probability $1/r$) is uninformative, and it is uninformative for every m because $\chi_{0,0} \equiv 1$ is trivial on every subgroup.

Structural statement: measurement samples a character trivial on the hidden subgroup, i.e. a point of $L^\perp$; since L is a line, one generic point of $L^\perp$ determines it, hence determines m .

Exercise 12 — the group law and the cost it implies

8 minutes

Task

Derive the affine addition formulas by Viète. Then identify which field operation dominates the quantum cost and why, and state the register size from Hasse's bound.

Exercise 12 — the group law and the cost it implies

8 minutes

Task

Derive the affine addition formulas by Viète. Then identify which field operation dominates the quantum cost and why, and state the register size from Hasse's bound.

Solution

$E : y^2 = x^3 + ax + b$ over $\mathbb{F}_p$, $\Delta = -16(4a^3 + 27b^2) \neq 0$. For $P_1 \neq \pm P_2$ the line is $y = \lambda x + \nu$ with $\lambda = \frac{y_2 - y_1}{x_2 - x_1}$, $\nu = y_1 - \lambda x_1$. Substituting,

$$(\lambda x + \nu)^2 = x^3 + ax + b \implies x^3 - \lambda^2 x^2 + (a - 2\lambda\nu)x + (b - \nu^2) = 0,$$

a cubic whose roots are the three intersection abscissae. By Viète the sum of roots equals the negated x^2 -coefficient: $x_1 + x_2 + x_3 = \lambda^2$, so $x_3 = \lambda^2 - x_1 - x_2$. Reflecting the third intersection $(x_3, \lambda x_3 + \nu)$ in the x -axis: $y_3 = -(\lambda x_3 + y_1 - \lambda x_1) = \lambda(x_1 - x_3) - y_1$.

Doubling: $2yy' = 3x^2 + a$ gives $\lambda = \frac{3x_1^2 + a}{2y_1}$, same argument with a double root.

The dominating operation: λ is a quotient in $\mathbb{F}_p$, so every point addition needs a modular inversion. Reversible inversion (Kaliski's binary GCD) is $\Theta(n^2)$ Toffoli with $O(n)$ ancillas — $\Theta(n)$ multiplications' worth of work. Algorithm 1 in fact calls it *four* times (two compute/uncompute pairs), which is why an elliptic point addition is much dearer than a modular multiplication, and why ECC-256 is comparable to RSA-3072 rather than trivially cheaper.

Register size: Hasse gives $|\#E(\mathbb{F}_p) - (p + 1)| \leq 2\sqrt{p}$, so $r \leq \#E \leq p + 1 + 2\sqrt{p} < 2^{n+1}$ for $n = \lceil \log_2 p \rceil$: $n + 1$ qubits per index register.

Exercise 13 — the reversible tower

10 minutes

Task

List the arithmetic blocks bottom-up, with the cost of each and the device that makes each one garbage-free. Then give the structure of the controlled point addition and explain the λ -uncomputation. Finish with the two headline resource formulas.

Exercise 13 — the reversible tower

10 minutes

Task

List the arithmetic blocks bottom-up, with the cost of each and the device that makes each one garbage-free. Then give the structure of the controlled point addition and explain the λ -uncomputation. Finish with the two headline resource formulas.

Solution

Tower. (1) MAJ/UMA blocks — one Toffoli each; MAJ writes the carry, UMA is its partial inverse and leaves the sum bit. (2) Ripple-carry adder — MAJ chain up, one CNOT, UMA chain down: in place, addend preserved, *no garbage*; $2n$ Toffoli, $2n + 2$ qubits; reversed, it subtracts. (3) Modular add — add, $\text{sub}(p)$, copy sign to t , $\text{add}(p)$ controlled on t , then recompute t from the output (a reduction happened iff $(x + y) \bmod p < x$); $\Theta(n)$ Toffoli, one ancilla, cleared. (4) Modular double (p odd) — shift, conditional $\text{sub}(p)$; the flag is recoverable from the least significant bit, so no persistent ancilla. (5) Modular multiply / square — Horner: n modular doublings interleaved with n controlled modular additions; $\Theta(n^2)$ Toffoli (halved by Montgomery form). (6) Modular inverse (Kaliski) — the expensive one: $\Theta(n^2)$ Toffoli, $O(n)$ ancillas.

Making Kaliski reversible. Run a fixed $2n$ rounds with a flag that flips once $v = 0$, after which each round only increments a counter — so the circuit shape is input-independent; and record which of the four branches each round took in one qubit m_i (the second branch bit is recomputable, since exactly one of r, s is even), so the whole computation can be uncomputed once the result is copied out. Those $2n$ round-records dominate the qubit count.

Controlled point addition of the constant $(x_2, y_2) = [2^j]P$: shift by the constant; invert Δx ; form λ and clear the inverse; write x_3 via λ^2 and a constant shift; write y_3 via λ ; then uncompute λ using $\lambda = -\frac{y_3 + y_2}{x_3 - x_2}$, recomputable from the output and the constant; finally undo the constant shifts. *Why it works:* P_1, P_2 and $(x_3, -y_3)$ are collinear, so the slope through P_2 and the third intersection is λ . This is Bennett compute-copy-uncompute specialised to the group law, and it is mandatory: by (R3) leftover garbage would tag the branch and destroy the interference.

Headlines. Per controlled point addition: $9n + 2\lceil \log_2 n \rceil + 10$ logical qubits and $\lesssim 224 n^2 \log_2 n + 2045 n^2$ Toffoli. Whole algorithm ($2n$ sequential additions): $(448 \log_2 n + 4090) n^3$ Toffoli, $\Theta(n)$ qubits.

PART 11

Summary

The mechanism in one line

modular periodicity $\rightarrow$ cyclic shift $\rightarrow$ character eigenvectors $\rightarrow$ roots-of-unity eigenvalues $\rightarrow$ phase kickback
 $\rightarrow$ binary phase ladder $\rightarrow$ inverse-QFT unwinding $\rightarrow \frac{\ell}{Q} \approx \frac{s}{r} \rightarrow$ continued fractions $\rightarrow r \rightarrow$ factors

Reading the chain once more, now with every link a theorem: multiplication by a modulo N restricts, on the orbit of 1, to the cyclic shift S ; S is diagonalised by the characters of $\mathbb{Z}/r\mathbb{Z}$, whose eigenvalues are the r -th roots of unity $e^{2\pi i s/r}$; controlled powers of U_a kick these eigenphases onto the counting register as a binary phase ladder $e^{2\pi i x s/r}$; crucially $|1\rangle$ is the uniform superposition of all r eigenvectors, so one runs phase estimation *without knowing* r ; the inverse QFT unwinds each ladder into a peak at $\ell \approx Qs/r$; continued fractions promote ℓ/Q to s/r ; and r yields the factors.

And the two-dimensional version

$$f(k, \ell) = [k]P + [\ell]Q = [k + m\ell]P \text{ hides } L = \langle (-m, 1) \rangle \xrightarrow{\text{QFT}_r \otimes \text{QFT}_r} \text{samples a character trivial on } L \\ \longrightarrow y_2 \equiv my_1 \pmod{r} \longrightarrow m \equiv y_2 y_1^{-1}, \text{ one run for prime } r.$$

What each ingredient contributes — and what it does not

- ▶ **Superposition** exposes the whole orbit at once as the state $|1\rangle$; it does not, by itself, “try all answers” — an unmeasured uniform superposition is information-free.
- ▶ **Linearity** lets phase estimation act on every eigenbranch of $|1\rangle$ simultaneously; it is why a non-eigenstate input works at all.
- ▶ **Complex relative phase** is the medium carrying the eigenvalue; a real-amplitude theory could not encode s/r in a phase, since eigenvalues of unitaries are points on the unit circle.
- ▶ **Tensor products** give the exponentially large index space and the big-endian ladder $e^{2\pi i x \varphi}$; they do not, alone, create speed-up.
- ▶ **Controlled unitaries** convert an eigenvalue into a relative phase; they do not force the target into an eigenstate.
- ▶ **Entanglement** between counting and work registers tags each ladder with its branch; it suppresses cross-branch interference and is necessary bookkeeping, not the engine. Gottesman–Knill shows it cannot be sufficient.
- ▶ **Interference** — coherent addition of the x -indexed amplitudes — is what concentrates the distribution. Name the two contributions; do not say “the phases interfere”.
- ▶ **Measurement** returns one sample ℓ ; it does not read the whole spectrum and does not uncover a pre-existing classical value.
- ▶ **Reversible modular arithmetic** makes the controlled powers cheap, via classical repeated squaring or double-and-add. This, not the QFT’s gate count, is where the algorithm spends its time.

Three principles

P1 Garbage is which-path information, so it kills interference — hence every arithmetic block must be uncomputed, and the elliptic point addition must recompute λ from its own output. **P2** The cost is arithmetic $\Theta(n^3)$, not Fourier $\Theta(n^2)$ (or $O(n \log n)$ approximate). **P3** The speed-up needs all three of: a structured input, cheap controlled powers, and interference — and it needs the group to be *abelian*.

Six common misstatements, and precise replacements

Imprecise	Precise
It tries all possible answers at once.	It computes on all inputs at once, but a measurement returns a single bitstring — so the algorithm has to arrange interference first. An unmeasured uniform superposition carries zero information.
The QFT computes the whole Fourier spectrum.	It applies the Fourier change of basis to the amplitudes. The Q coefficients are never made classically accessible; one outcome is sampled. That is also why it is not a fast FFT.
Entanglement is the source of the speed-up.	Entanglement appears when the oracle correlates the registers, and its role is to keep the r eigenphase branches from mixing. The advantage comes from interference <i>within</i> each branch, plus the fact that controlled powers are cheap.
Measurement reveals the value the qubit already had.	There was no such value. The relative phases were physically real and had to be converted into magnitudes first — that conversion is what the inverse QFT does.
The phases interfere.	The contributions $V_{yx}\alpha_x$ and $V_{yx'}\alpha_{x'}$ to the outcome $ y\rangle$ add with relative phase $\theta - \phi$. If the two contributions cannot be named, there is no interference to discuss.
A qubit is 0 and 1 at the same time.	A qubit's state is a unit vector in $\mathbb{C}^2$. The two coefficients are complex amplitudes; only their squared moduli are probabilities, and the relative phase between them is physical — that is the part with no classical analogue.

Analogies: which to use, and exactly where each breaks

Analogy	What it gets right	Where it breaks	Safe phrasing
Diffraction grating	Exact: $\Pr[y]$ is the Q -slit intensity pattern, same formula	The slits are basis states, not apertures; nothing propagates	“the outcome distribution is literally a diffraction pattern with Q slits”
Arrows added head to tail	Exact: β_y is a sum of complex numbers	Nothing — use it freely; no significant failure mode	“these are the individual contributions; the probability is the length of the resultant, squared”
Tuning a radio dial	Fourier is a test for frequency match	You get <i>one</i> sample, and only after all the tests happened coherently; there is no spectrum to look at	“it tests every frequency at once but hands you a single sample”
Musical beats / resonance	Constructive vs. destructive addition	The index x is not time; nothing oscillates physically	“the same arithmetic as beats, but the frequency here is an eigenvalue”
Combination lock whose wrong turns cancel	Conveys interference as <i>active</i> cancellation	Suggests checking candidates one at a time	“wrong candidates don’t get checked and rejected — their amplitudes destroy each other”

One mechanism, several broken cryptosystems — and where it stops

The unifying statement

Factoring (hidden group $\mathbb{Z}$, via $\mathbb{Z}/r\mathbb{Z}$), the finite-field discrete logarithm and the ECDLP (both on $\mathbb{Z}/r\mathbb{Z} \times \mathbb{Z}/r\mathbb{Z}$) are all instances of the *abelian* hidden subgroup problem. In every case the group is abelian, so its irreducible representations are one-dimensional, so each eigenvalue is a single phase — the fact underlying every line of Parts IV–IX.

Where it fails

For *non-abelian* groups the irreducible representations have dimension > 1 : an eigenvalue becomes a matrix, and there is no single phase to kick back. No efficient quantum algorithm is known for the non-abelian HSP in general — which is why graph isomorphism (symmetric group) and lattice problems (dihedral group) remain open, and why post-quantum cryptography is built on them.

This is not a footnote. It is the reason the post-quantum standards are believed to work.

Practical reading

- ▶ RSA, finite-field Diffie–Hellman, DSA, ECDH, ECDSA: all broken by the same machine.
- ▶ Symmetric ciphers and hashes: only Grover, i.e. a quadratic speed-up by a different mechanism (amplitude amplification, not eigenvalue estimation). Doubling the key length restores the margin.
- ▶ Lattice, code, isogeny and hash-based schemes: no known abelian-HSP attack.

On timelines

No cryptographically relevant number has been factored. But two things are true at once: the estimates in Parts V and IX are *logical* counts, and physical error correction multiplies them by three or four orders of magnitude; and anything encrypted today that must stay secret for a decade should already be migrating, because of harvest-now-decrypt-later.

Fourteen review questions

1. Why does factoring reduce to order finding?
2. Why is modular multiplication a cyclic translation?
3. Why do roots of unity appear — and why are they forced?
4. Why is a unitary's eigenvalue always a pure phase?
5. Why is the Fourier basis a change of basis, not a trick?
6. Why are all probabilities uniform before the inverse QFT?
7. Why does the free state $|1\rangle$ work as input?
8. Why does $c\text{-}U$ not project the target onto an eigenstate?
9. Why does the inverse QFT produce a peak, and how wide is it?
10. Why does $N = 15$ hide the continued-fraction step, and $N = 21$ not?
11. Why does the ECDLP need two QFTs and no continued fractions?
12. Why is ECC-256 a cheaper quantum target than RSA-3072 despite the harder oracle?
13. Why must the oracle be garbage-free, and what exactly breaks otherwise?
14. Why is the QFT the cheapest part of the algorithm?

Where each answer lives

1–2: Part I. 3–5: Part II. 6–9: Parts III–IV, kickback through the Fejér kernel. 10: Parts VII–VIII. 11–12: Part IX. 13: Parts III and IX. 14: Part V.

Self-assessment

Answering a question never seen before by *re-deriving* rather than searching memory; being able to say, without preparation, what each ingredient contributes *and what it does not*; and being able to catch an error in someone else's explanation and name which link of the chain it broke.

Reference numbers

$N = 15, a = 2$	orbit $\{1, 2, 4, 8\}$, $r = 4$; $Q = 8$ (or 256 with $t = 8$); outcomes $\{0, \frac{Q}{4}, \frac{Q}{2}, \frac{3Q}{4}\}$ each with probability $\frac{1}{4}$ — exact spikes; success outright $\phi(4)/4 = \frac{1}{2}$; $\gcd(3, 15) = 3$, $\gcd(5, 15) = 5$. Dyadic, and misleadingly easy.
$N = 21, a = 2$	orbit $\{1, 2, 4, 8, 16, 11\}$, $r = 6$; $n = 5$, $Q = 512$, $t = 9$ (from $441 \leq Q < 882$); eigenphases $0, \frac{1}{6}, \frac{1}{3}, \frac{1}{2}, \frac{2}{3}, \frac{5}{6}$ — only $s = 0, 3$ dyadic; peaks at $\ell = 0, 85, 171, 256, 341, 427$; branch $s=1$: $\Pr[85] = 0.684$, $\Pr[86] = 0.171$; per-run success ≈ 0.321 direct, ≈ 0.815 with repair; $\gcd(7, 21) = 7$, $\gcd(9, 21) = 3$.
Toy curve	$E : y^2 = x^3 + 3x + 2$ over $\mathbb{F}_5$; $E(\mathbb{F}_5) = \{\mathcal{O}, (1, 1), (1, 4), (2, 1), (2, 4)\}$, $\#E = 5$ prime; $P = (2, 1)$ gives $[1..5]P = (2, 1), (1, 4), (1, 1), (2, 4), \mathcal{O}$; with $Q = (1, 1)$ the answer is $m = 3$; inverses mod 5: 1, 3, 2, 4.
Complexity	modular exponentiation / scalar multiplication $\Theta(n^3)$; QFT $\Theta(n^2)$ exact, $O(n \log n)$ approximate; continued fractions $O(\log^3 q)$; qubits $\Theta(n)$.
Peak bounds	nearest grid point $\Pr \geq 4/\pi^2 \approx 0.405$; exact zeros at $\delta_y = k/Q$, $k \neq 0$; Legendre threshold $1/(2q^2)$; register $t = 2n + O(1)$.
Gate accounting	1 Toffoli = 7 T (T -depth 3), or 4 T per compute/uncompute pair; rotation to precision $\epsilon = 3 \log_2(1/\epsilon)$ T ; CDKM adder $2n$ Toffoli, $2n + 2$ qubits.
ECDLP resources	per point addition $9n + 2\lceil \log_2 n \rceil + 10$ qubits, $\lesssim 224n^2 \log_2 n + 2045n^2$ Toffoli; full run $(448 \log_2 n + 4090)n^3$ Toffoli.
Head-to-head	P-256: 2330 logical qubits, 1.26×10^{11} Toffoli. RSA-3072: 6146 qubits, 1.86×10^{13} Toffoli. Matched security ≈ 128 bits; operand ratio $12\times$.

1. P. W. Shor, *Algorithms for quantum computation: discrete logarithms and factoring*, Proc. 35th FOCS, 124–134 (1994); SIAM J. Comput. 26(5), 1484–1509 (1997).
2. M. Roetteler, M. Naehrig, K. M. Svore, K. Lauter, *Quantum resource estimates for computing elliptic curve discrete logarithms*, arXiv:1706.06752 (2017). *Primary source for Part IX's circuits and counts*.
3. J. Proos, C. Zalka, *Shor's discrete logarithm quantum algorithm for elliptic curves*, Quantum Inf. Comput. 3(4), 317–344 (2003).
4. E. Muñoz-Coreas, H. Thapliyal, *Everything you always wanted to know about quantum circuits*, arXiv:2208.11725 (2022). *Source of the gate set, resource-cost taxonomy and adder gate counts in Part V*.
5. S. A. Cuccaro, T. G. Draper, S. A. Kutin, D. P. Moulton, *A new quantum ripple-carry addition circuit*, arXiv:quant-ph/0410184 (2004).
6. T. G. Draper, *Addition on a quantum computer*, arXiv:quant-ph/0008033 (2000).
7. S. Beauregard, *Circuit for Shor's algorithm using $2n + 3$ qubits*, Quantum Inf. Comput. 3(2), 175–185 (2003).
8. B. S. Kaliski, *The Montgomery inverse and its applications*, IEEE Trans. Computers 44(8), 1064–1065 (1995).
9. C. Gidney, *Halving the cost of quantum addition*, Quantum 2, 74 (2018).
10. C. Gidney, M. Ekerå, *How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits*, Quantum 5, 433 (2021).

11. C. Gidney, *How to factor 2048 bit RSA integers with less than a million noisy qubits*, arXiv:2505.15917 (2025).
12. T. Häner, M. Roetteler, K. M. Svore, *Factoring using $2n + 2$ qubits with Toffoli based modular multiplication*, arXiv:1611.07995 (2016). *Source of the factoring counts in Part V.*
13. R. B. Griffiths, C.-S. Niu, *Semiclassical Fourier transform for quantum computation*, Phys. Rev. Lett. 76, 3228–3231 (1996).
14. A. Yu. Kitaev, *Quantum measurements and the Abelian stabilizer problem*, arXiv:quant-ph/9511026 (1995).
15. N. J. Ross, P. Selinger, *Optimal ancilla-free Clifford+T approximation of z -rotations*, Quantum Inf. Comput. 16(11–12), 901–953 (2016).
16. M. Amy, D. Maslov, M. Mosca, M. Roetteler, *A meet-in-the-middle algorithm for fast synthesis of depth-optimal quantum circuits*, IEEE TCAD 32(6), 818–830 (2013).
17. A. Barenco et al., *Elementary gates for quantum computation*, Phys. Rev. A 52, 3457–3467 (1995).
18. M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary Edition, Cambridge University Press (2010).
19. IBM Quantum Documentation, *Shor's algorithm* tutorial, and J. Watrous, *Fundamentals of quantum algorithms*. *Source of the Qiskit implementation in Part VIII.*
20. M. Amico, Z. H. Saleem, M. Kumph, *Experimental study of Shor's factoring algorithm using the IBM Q Experience*, Phys. Rev. A 100, 012305 (2019); U. Skosana, M. Tame, *Demonstration of Shor's factoring algorithm for $N = 21$ on IBM quantum processors*, Sci. Rep. 11, 16599 (2021).